OHMG: ONE HOT MODULAR GARBLING

A. FUTORANSKY, F. BARBÀRA, R. FERNÁNDEZ, AND G. LAROTONDA

Abstract. We propose a novel mechanism for garbling wires and gates of a logical circuit in a privacy-free environment, focusing on the authenticity of the protocol. It is based on one-hot encodings, tensor products and elliptic curve arithmetic. This scheme is designed to work with arithmetic gates, but we also show gadgets to implement transitions from binary inputs to arithmetic outputs and vice versa. For our scheme, each arithmetic gate takes at most one cyphertext of material to execute its functionality (assuming knowledge of the garbled inputs and their cleartexts). We show an application to blockchain transactions. The security of the scheme is proved in the UC setting.

Keywords. Garbled circuit, cyphertext optimization, one hot product, tensor product.

Contents

Part 1. Cryptographic Foundations

1. Introduction

1.1. **Garbled circuits.** Garbled circuits [35] (GCs) enable secure multiparty computation (MPC) with minimal interaction. Traditional GC schemes use boolean-gates and require at least one cyphertexts per `AND` gate [6, 26, 35]. With the introduction of Groth16 SNARKs [24] and the recent proliferation of ZKVMs [27] that output SNARKs, the problem of verifying large computations on garbled circuits has focused on efficiently encoding SNARK verifiers on boolean circuits. This is enough because SNARKs, combined with other recursion techniques, can prove arbitrary computation.

A Groth16 verification requires a high number of modular exponentiations on large integers, which decompose into an even higher number of CPU word multiplications. When using boolean circuits, each word multiplication requires thousands of gates. The end result is that a Groth16 garbled circuit requires 3 billion gates [29], or over 60 GB of data when garbled (this is after optimizations, see Table 1 below for more crude numbers). When the circuits must be proven correct by cut-and-choose methods, several of these circuits need to be transmitted and stored until evaluation time.

There are currently at least four optimizations being actively researched:

(1) Improving the boolean circuit gadgets that perform arithmetic operations efficiently on the existent garbling schemes.
(2) Replacing the SNARK verifier by an alternate SNARG scheme that requires less computation for SNARK verification, even at the expense of more costly proving.
(3) Replacing the SNARK by a Designated Verifier SNARG.
(4) Replacing prime-field curves with binary curves whose operations require a lower amount of boolean gates.

In this work, we focus on a fifth approach: designing a garbling scheme that can more naturally represent arithmetic operations commonly required by Prime-field curves, resulting in shorter encodings.

Therefore, in this paper we introduce OHMG, a privacy-free garbling scheme using one-hot encodings, tensor products, and elliptic curve arithmetic over $\mathbb{Z}_p$. It optimizes *arithmetic gates* (for instance, modular addition/multiplication in $\mathbb{Z}_n \to \mathbb{Z}_m$), requiring at most one cyphertext per gate, regardless of input size. Privacy-free means that our approach discloses all wiring states, but prioritizes authenticity: verifiers cannot forge outputs without valid input garblings. On the other side, our scheme can still protect secret output labels, so that the circuit can be used for conditional disclosure of secrets. The OHMG scheme suits trustless settings like blockchain zero-knowledge proofs (ZKPs), rewarding honest provers via timeouts and penalizing cheaters.

Our new garbling scheme introduces several important benefits over traditional garbled-circuit constructions. In essence, it replaces Boolean-gate-level garbling with an arithmetic, one-hot, elliptic-curve–based representation that is privacy-free but authenticity-preserving. In terms of circuit garbling material, there is a significant improvement if the circuit is computing a function which is essentially arithmetic to begin with, like a Groth16 verification, simplified to the computation of just one pairing of elliptic

Description	Arithmetic Addition Gates	Arithmetic Multiplication Gates	Boolean XOR Gates	Boolean AND Gates (non-free)
One full emulated BN254 pairing (Miller loop + Final Exp)	2,089,977	1,393,318	∼30 billion	∼9 billion
Full Groth16 BN254 verifier (1 pairing + scalar checks + overhead)	∼2,250,000	∼1,500,00	∼32 billion	∼10 billion

TABLE 1. Gate count for a real Groth16 verifier with one full emulated pairing (BN254 curve). This table does not take into account many possible optimizations in both methods, e.g. stored gates that are re-utilized etc.

Notes: Arithmetic gates are exact $\mathbb{Z}_p$ additions and multiplications, with Montgomery reductions fully expanded. Boolean gate counts are computed roughly as 0 AND - 500 XOR gates per addition gate (ripple carry), while 6500 AND - 20000 XOR gates (Karatsuba) per multiplication gate. Sources: [4, 15, 3, 13].

Description	Arithmetic OHMG	Boolean Yao + optimizations
Cyphertexts per Groth16 verification (garbled circuit material)	∼100 million	∼10 billion

TABLE 2. Cyphertext count for a real Groth16 verifier with one full emulated pairing (BN254 curve).

Notes: The standard Boolean circuit garbling used to compute the pairing (Yao + half-gate optimizations: red) uses 1 cyphertext per AND gate. Our scheme (OHMG: blue) uses 1 cyphertext per addition gate (large integers up to 512 bits), and 1 cyphertext per multiplication gate (small integers up to 16 bits, per Remark 4.45). We use the CRT and Montgomery reduction to compute the total number of $\mathbb{Z}_p$ multiplications of 16-bit numbers; a detailed description of this theoretical method can be found in Appendix A.

curves points in the curve family BN254, see Table 1 for a gate count and Table 2 for a cyphertext count, which shows roughly a **100x** reduction in circuit material (transmission and storage) using OHMG.

These are the key benefits of the OHMG Garbling Scheme:

(1) Optimized for Privacy-Free Applications: OHMG is designed for scenarios where privacy is not required, enabling aggressive efficiency optimizations without compromising correctness.

(2) Multi-Format Wire Support: it supports Boolean (B), Arithmetic (A), and One-Hot-Encoding (O) wire formats, with seamless and efficient conversions
   - Free conversions: $B \to A$, $O \to A$, $O \to B$.
   - Single-cyphertext conversions: $B \to O$, $A \to O$, $A \to B$.

(3) Zero-cyphertext One-Hot computations: any function operating on a single One-Hot wire can be evaluated for free, requiring no cyphertext.

(4) Constant-Cost 1-cyphertext Multi-Wire Computations: any number of functions computed from the same set of two or more wires requires only one cyphertext, independent of gate count. While the communication cost is constant (one cyphertext), the local evaluation cost (counting the number of group operations) is O of the product of sizes of input wires. This provides a tunable computation-communication trade-off.

(5) Adjustable Computation–Communication Trade-off: the scheme allows fine-tuning between wire size (in arithmetic or one-hot encodings) and gate evaluation cost, offering an adaptable communication–computation balance tailored to each application.

Our main design is based on one-hot encoding ideas of Heath and Kolesnikov [21, 20], on elliptic curve operations, and on modular arithmetic. One of the main ingredients in our scheme is the computation of the tensor products of these one-hot garblings, which costs only one cyphertext of material regardless the size of the tensor product. In our scheme, the security properties are subsumed to *authenticity* (see the formal definition in Section 2.3 below), a property that states that the verifier/evaluator of the circuit cannot forge a valid output of the garbled scheme, without knowing the garblings of the corresponding inputs of that output. We exhibit a case of use for proofs of knowledge on a blockchain, which rewards the successful prover.

Since there are several ways to implement a garbling scheme, let us give an overview of our intended presentation and cases of use, that justifies our focus on authenticity, instead of privacy. A garbling of a circuit GC involves two parties P, V (or Patrick and Valery):

(1) P is the *prover/garbler* of a circuit C on which P, V agree, computing a functionality F.

(2) V is the *verifier/evaluator* of the circuit C.

(3) The gates $\mathcal{G}_i$ of the circuit C are (partially) ordered topologically: that is the i-th gate has no input that is an output of $\mathcal{G}_j$ for $j > i$. A topological order (not necessarily unique) can always be put on an acyclic circuit C.

(4) The *input* wires of the circuit C are the wires with no preceding gate, and the *output* wires are those with no subsequent gate.

(5) P claims knowledge of certain inputs a that upon evaluation of circuit C, return the output c, that is $F(a) = c$. Patrick's claim is part of a trustless agreement, such as a blockchain transaction, where Valery needs to confirm the computation's correctness.

(6) In implementations, the final gate is a Boolean gate verifying that the output of the circuit equals c after evaluating it with the inputs provided by Patrick. Therefore, there is only one output wire, carrying a Boolean variable which returns true or false. The mechanism of resolution is then by negation: if the

verifier V obtains the output corresponding to false, then she wins the challenge (the prover's claim is false). Otherwise, after a certain stipulated time-out, the prover's claim is considered to be true, and he wins the challenge.

Remark 1.1. Circuits are modelled as directed acyclic graphs (DAGs), allowing fan-out: a wire can feed multiple gates without recomputation. Mid-states can thus be output without duplicating subcircuits.

1.2. **Comparison with related work.** Table 3 summarizes how OHMG relates to prior garbling schemes. Baum [5] provides a foundational decomposition of privacy-free garbling but does not target arithmetic efficiency. Heath-Kolesnikov [21] introduces one-hot encodings for efficient gate evaluation with privacy, yet relies on Boolean decomposition for arithmetic operations. AuthOr [8] achieves low per-gate cost for Boolean circuits but lacks native arithmetic support. In contrast, OHMG unifies arithmetic, binary, and one-hot wire representations, enabling constant-cost arithmetic gates without bitwidth-dependent blowup. The scheme trades evaluation cost (elliptic-curve operations) for minimal communication (one ciphertext per gate) and provides UC security rather than game-based guarantees alone—properties motivated by our target application of blockchain dispute resolution.

1.3. **Why universal composability?** While game-based authenticity proofs suffice for standalone garbling schemes —as reflected in our own Definition 2.4 in Section 2.3, which captures authenticity without requiring UC— our target application demands stronger guarantees. The OHMG scheme is designed for blockchain dispute resolution (Section 5.4), where the garbled circuit does not operate in isolation. Instead, it interacts with a global ledger functionality $\mathcal{G}_{\mathsf{trust}}$ (Algorithm 5), which we explicitly model as an abstraction of an append-only, publicly readable ledger, such as a blockchain (Remark 5.8). This global setup is shared across all protocol sessions and is directly accessible by the environment $\mathcal{Z}$, enabling commitments posted in one session (such as the garbling phase) to be readable in another (such as the on-chain verification phase) (Remark 5.7).
The UC framework naturally handles this cross-session dependency. Furthermore, our security model allows the environment $\mathcal{Z}$ to adaptively corrupt parties during execution, revealing their internal states (Definition 5.2), capturing scenarios where an adversary schedules challenges across multiple concurrent disputes on the same blockchain. The UC composition theorem then ensures that security is preserved when OHMG is deployed alongside other protocols sharing the same ledger —a guarantee that game-based proofs cannot provide-.
This compositional security, established by Theorem 5.30, justifies the additional complexity of our UC treatment.

1.4. **Notation.** Let us fix the notations used in this paper:
Modular arithmetic. For $n \in \mathbb{N}$, we denote $\mathbb{Z}_n = \{0, 1, \ldots, n-1\}$ the integers modulo n, and $\mathbb{Z}_n^*$ the multiplicative group of invertible elements. We write $a \equiv b \pmod{n}$ if $n \mid (a - b)$. For a function $f : \mathbb{Z}_n \times \mathbb{Z}_m \to \mathbb{Z}_k$, the notation $f(a, b) \pmod{k}$ indicates that the output is reduced modulo k. We identify $\{0, 1\} \simeq \mathbb{Z}_2$.

	Baum [5]	**Heath–Kolesnikov** [21]	**AuthOr** [8]	**OHMG**
Primary goal	Decomposition of privacy-free vs. private GC	Efficient one-hot garbling with privacy	Authenticity-oriented Boolean GC	Arithmetic-efficient, privacy-free garbling
Privacy	Optional	Yes	No	No
Security focus	Context-dependent	Simulation-based	Authenticity	Authenticity only
Formal framework	Compositional	Simulation-based security	Game-based authenticity	UC
Wire semantics	Boolean	One-hot, Boolean	Boolean	Arithmetic, binary, and one-hot wires unified
Arithmetic gates	Via Boolean circuits	Via Boolean decomposition or masking	Via Boolean circuits	Native arithmetic gates
Gate cost	Depends on subcircuit	Scales with bitwidth	$\sim$1 per AND gate	1 ciphertext per arithmetic gate
Dependency on bitwidth	Yes	Yes	Yes	No
Evaluation cost	Moderate	Moderate	Moderate	High, low communication
Use of tensor products	No	Implicit	No	Explicit
Elliptic-curve arithmetic	No	No	No	Yes
Binary $\leftrightarrow$ arithmetic translation	No	Limited	No	Explicit
Target applications	Mixed MPC pipelines	General MPC	Verifiable computation	SNARK verification, blockchain dispute games

TABLE 3. Comparison of OHMG with related garbling schemes

Binary encoding. An integer $a \in \mathbb{Z}_n$ with $n = 2^d$ has little-endian binary encoding

$$\mathsf{bin}(a) = a[0] \cdots a[d-1], \quad \text{where } a = \sum_{j=0}^{d-1} a[j] \cdot 2^j,$$

and $a[0]$ is the least significant bit (LSB). All wires and gates perform modular arithmetic in $\mathbb{Z}_k$ for some $k \leq n$ chosen per wire by the circuit designer; for Boolean operations, $k = 2$.
Vectors and matrices. For a vector $V \in \mathbb{Z}_p^n$, we denote its i-th coordinate by V_i. If $V : \mathbb{Z}_n \to \mathbb{Z}_p^k$ is a vector-valued function, then $V(a)_i$ denotes the i-th coordinate of the vector $V(a)$. We write M^T for the transpose of a matrix M, and identify row vectors v with $1 \times n$ matrices, so that v^T is a column vector.
Booleans. For a bit $z \in \{0, 1\}$, we denote its negation by $\overline{z} = \mathsf{NOT}(z) = 1 - z$.
Conventions. A product over an empty index set equals 1, that is: $\prod_{i \in \emptyset} x_i = 1$.
A sum over an empty index set equals 0, that is: $\sum_{i \in \emptyset} x_i = 0$.

2. Definitions, features and security properties

This section provides a review of garbled circuits, focusing on formal definitions, protocol high-level interactions and security properties relevant to our research. We emphasize a scheme prioritizing authenticity over privacy or obliviousness, exploring their applications and related work.

2.1. Elliptic curves.

For an elliptic curve E over $\mathbb{Z}_p$, let $G \in E(\mathbb{Z}_p)$ generate a prime-order subgroup $\langle G \rangle$ of order q. We write $[s]G$ for scalar multiplication by $s \in \mathbb{Z}_q$. All scalars and multiplicative masks used by the protocol in $\mathbb{Z}_q^*$ and their products and inverses are taken in $\mathbb{Z}_q^*$. When a scalar is displayed as an element of $\mathbb{Z}_p^*$, we use its canonical representative in $\{1, \ldots, q-1\} \subset \mathbb{Z}_p$, but we never perform modular arithmetic mod p on these scalars.

2.2. Formal framework.

We begin with formal definitions to establish the foundation of garbled circuits.

Definition 2.1 (Garbled circuits and consistency)**.** A *garbled circuit* GC of a circuit C (computing functionality F) is a consistent garbling of the wires of the circuit C, together with the necessary *material* that allows the evaluator to obtain a garbled output.

Definition 2.2. A *garbling scheme* is a tuple of probabilistic polynomial-time algorithms $(\mathsf{Gb}, \mathsf{En}, \mathsf{De}, \mathsf{Ev}, \mathsf{ev})$, where:

- $\mathsf{Gb}(1^\kappa, C) \to (GC, f_e, f_d)$: The garbling algorithm takes a security parameter κ and a circuit C, producing a garbled circuit GC, an encoding function f_e, and a decoding function f_d.
- $\mathsf{En}(f_e, x) \to X$: The encoding algorithm transforms an input x into a garbled input X.
- $\mathsf{Ev}(GC, X) \to Y$: The evaluation algorithm computes a garbled output Y.
- $\mathsf{De}(f_d, Y) \to y$: The decoding algorithm recovers the function output y in cleartext.
- $\mathsf{ev}(F, x) \to y$: The evaluation algorithm computes the non-garbled output y from x and the function F represented by the circuit C.

Definition 2.3 (Consistency and correctness)**.** A garbling scheme is *correct* if

$$\Pr[\mathsf{De}(f_d, \mathsf{Ev}(GC, \mathsf{En}(f_e, x))) = \mathsf{ev}(F, x)] = 1$$

for all circuits C and inputs x. Since we garble wires rather than gates, we introduce *consistency*: after evaluating a gate with input wires carrying cleartext x, the evaluator obtains output wires evaluated at $f(x)$, where f is the gate functionality. Consistency at the gate level, using one-hot encodings, ensures the circuit's correctness, supporting OHMG's modularity and authenticity-focused design.

2.3. **Security properties.** In this work we focus on *authenticity* of garbled outputs and on *soundness* against a malicious prover. We do *not* aim to achieve input privacy or obliviousness: in our model the evaluator learns the cleartext input x and only the correctness of the reported output is protected.
For context, standard privacy notions for garbling schemes (indistinguishability-based and simulation-based) can be defined as in Bellare-Hoang-Rogaway [6]. We recall them only informally and do not prove that OHMG satisfies them.

Definition 2.4 (Authenticity against malicious evaluators)**.** Let us consider a grabling scheme given by $\mathcal{G} = (\mathsf{Garble}, \mathsf{En}, \mathsf{Ev}, \mathsf{De})$. Let us consider the following experiment $\mathsf{Auth}^{\mathcal{G}}_{\mathcal{A}}(1^\kappa)$:

(1) The challenger chooses a function F with circuit representation C and an input x (or lets $\mathcal{A}$ choose them). It runs $(GC, f_e, f_d) \leftarrow \mathsf{Garble}(1^\kappa, F)$.
(2) It computes the garbled input $X \leftarrow \mathsf{En}(f_e, x)$ and the honest garbled output $Y \leftarrow \mathsf{Ev}(GC, X)$.
(3) It gives $(F, C, GC, f_e, f_d, x, X, Y)$ to $\mathcal{A}$, which outputs a string Y'.
(4) The experiment outputs 1 iff $Y' \neq Y$ and $y' := \mathsf{De}(f_d, Y') \notin \{\bot, \mathsf{ev}(F, x)\}$.

We say that $\mathcal{G}$ satisfies *authenticity* if for all PPT adversaries $\mathcal{A}$, the probability that $\mathsf{Auth}^{\mathcal{G}}_{\mathcal{A}}(1^\kappa) = 1$ is negligible in κ. See Corollary 5.16 for a UC-based realization of this property by our one-hot gate protocol π_f.

Informally, authenticity guarantees that, given a garbled circuit and a garbled input for some x, no PPT evaluator can produce a different garbled output which still decodes to a valid value $y' \neq \mathsf{ev}(F, x)$.

Definition 2.5 (Correctness against a malicious prover)**.** Consider a two-party protocol between a prover P and evaluator V for computing $y = F(x)$. We say the protocol is *sound against a malicious prover* if for every PPT prover $P^\star$ and every environment $\mathcal{Z}$, the output y_V of the honest evaluator V in the real world is either $\bot$ or the correct value $F(x)$, except with negligible probability. See Corollary 5.31 for a UC-based realization of this property by our cut-and-choose protocol π_{GC}.

We formalize this second property in the UC framework in Section 5, by specifying ideal functionalities $\mathcal{F}_f$ and $\mathcal{F}_{\mathsf{GC}}$ which are sound by construction, and then showing that our protocols π_f and π_{GC} realize them. Authenticity in the sense of Definition 2.4 is recovered as a corollary of these UC realizations.

Remark 2.6 (No privacy or obliviousness)**.** In OHMG the evaluator learns the cleartext input x, and we do not protect against leakage about x beyond what follows from knowing x and $F(x)$. We therefore do not claim indistinguishability-based privacy, simulation-based privacy, or obliviousness as defined in [6]; these are mentioned only for comparison with the classical garbling framework.

2.4. **Background and related work.** Bellare et al. [7] extend the garbling scheme framework from [6] to adaptive security, where inputs may depend on the garbled circuit. For secure outsourcing, authenticity ensures correct output computation without forging, aligning with SFE. The paper provides transforms to upgrade static-secure schemes to adaptive-secure ones, focusing on authenticity and obliviousness. Scenarios like secure outsourcing and one-time programs, where authenticity prevents malicious evaluators from producing invalid outputs, relate directly to SFE. Our protocol's security proofs (Section 5) address adaptive corruption, emphasizing authenticity, and the blockchain use case mirrors secure outsourcing by ensuring correct ZKP verification.
Wang et al. [34] propose optimizations for authenticated garbling schemes, achieving up to 2.6x improvement in communication complexity over semi-honest secure computation. Their focus is on malicious secure two-party computation where authenticity is critical, prioritizing it to prevent output forging while reducing online phase communication to semi-honest levels. Privacy and obliviousness are secondary, suiting scenarios where input privacy is less critical, such as outsourced computation or ZKPs, aligning with our proofs for authenticity under adaptive corruption.
Baum [5] explores garbling schemes with and without privacy, building on [6]. He suggests combining privacy-free (authenticity-oriented) garbling for sub-circuits dependent on one party's input with full SFE for others, reducing overhead. This applies to input validation in SFE, where the evaluator verifies output correctness without hiding inputs, focusing on efficient SFE for decomposable circuits (for instance, signature verification), needing fewer garbled circuits than $O(s)$ for active security (where s is the statistical security parameter). Our blockchain application uses authenticity for UTXO output verification, akin to this use case.
Biçer and Ajorian [8] introduce "AuthOr," an authenticity-oriented garbling scheme for general Boolean circuits, prioritizing authenticity over privacy and obliviousness. It achieves lower communication costs (1 cyphertext per AND gate, compatible with FreeXOR [23]) without extra overhead or assumptions, targeting verifiable computing and ZKPs, similar to our blockchain scenario.

2.5. **Our take on commitments and interactions.** Denote $\mathbb{Z}_p^*$ as the non-zero elements of $\mathbb{Z}_p$ (where p is prime), and fix a global *correlation* $\Delta \in \mathbb{Z}_p^*$ to mask wire encodings securely. More details can be found in Section 2.6.

(1) The circuit C computes F, where the prover claims that if $x = a$ then $F(a) = c$.
(2) The input is binary, that is, $x = x[0] \dots x[d-1]$, with $x[d-1]$ the least significant bit of $x \in \mathbb{Z}_n$.
(3) Input wires w_i, $i \in I$, of the garbled circuit GC are binary arrays of Boolean wires (Definition 4.7 in Section 4).
(4) The prover selects Δ (Remark 4.2) and generates labels for all input wires, picking R_i for all $i \in I$, computing $w_i(0) = R_i$, $w_i(1) = \Delta R_i$ (Definition 4.5), and hashes $h_{i,\sigma} = \mathsf{Hash}(w_i(\sigma))$, where $H : \mathbb{Z}_p \to \{0,1\}^\lambda$.
(5) The prover commits to wire labels using hashes and a Trusted Verification Service $\mathcal{G}_{\mathsf{trust}}$ (Algorithm 5), instantiated via the blockchain as described in Section 5.4.
(6) The output $y \in \mathbb{Z}_m$ (that is, $F : \mathbb{Z}_n \to \mathbb{Z}_m$) has two possible garbled forms, defined in the prover-verifier setup (Definition 4.7), namely: (i) Binary output:

GC produces a binary vector of Boolean wires as a binary garbling, and (ii) Arithmetic one-hot output: *GC* produces a one-hot vector of Boolean wires.

Remark 2.7. We model this trusted verifying service as an instance of the global bulletin-board functionality $\mathcal{G}_{\mathsf{trust}}$ introduced in Section 5.2 (Algorithm 5). Algorithms 1 and 2 should therefore be read as the application-level protocol run by the prover and verifier *on top of* $\mathcal{G}_{\mathsf{trust}}$.

2.5.1. *Commitment and security for OHMG in the UC framework.* The commitment process precedes the cut-and-choose ZKP, ensuring the prover commits wire label hashes to $\mathcal{G}_{\mathsf{trust}}$ before generating s garbled circuits. This order prevents a corrupt prover from adapting labels based on the verifier's subset choice, as bindings are fixed and verifiable. After commitment, the verifier checks a subset, and the evaluator decodes labels using Algorithm De with cleartext inputs, ensuring correctness against inconsistencies. Designed for the UC framework under a corrupt prover/honest verifier scenario, this scheme leverages $\mathcal{G}_{\mathsf{trust}}$'s ideal functionality to detect deviations and maintain authenticity, with binding hashes thwarting alterations. Full UC security proofs, including simulator construction and adaptive corruption handling, are deferred to Section 5.

Algorithm 1 Prover-$\mathcal{G}_{\mathsf{trust}}$ Commitment and Validation Scheme

Require:
Circuit C computing function $F : \mathbb{Z}_n \to \mathbb{Z}_m$. Security parameter κ. Global correlation $\Delta \in \mathbb{Z}_p^*$. Sets of input wires I and output wires O. Cryptographic hash function $H : \mathbb{Z}_p \to \{0,1\}^\lambda$. Security parameter s for cut-and-choose.

▷ Phase 1: Prover generates and commits input wire labels
for each input wire w_i, $i \in I$ **do**
  Prover randomly generates nonce $R_i \in \mathbb{Z}_p^*$.
  Prover computes label for bit 0: $w_i(0) = R_i$.
  Prover computes label for bit 1: $w_i(1) = R_i\Delta$.
  Prover computes hashes $h_{i,0} = \mathsf{Hash}(w_i(0))$, $h_{i,1} = \mathsf{Hash}(w_i(1))$.
  Prover sends $h_{i,\sigma}$ $(i \in I, \sigma \in 0,1)$ to $\mathcal{G}_{\mathsf{trust}}$.
  $\mathcal{G}_{\mathsf{trust}}$ stores hashes in a secure database indexed by prover identity and C.
  $\mathcal{G}_{\mathsf{trust}}$ issues commitment receipt c_{input} to prover.
end for
▷ Phase 2: Prover generates and commits output wire labels
for each output wire w_o, $o \in O$ **do**
  Prover randomly generates nonce $R_o \in \mathbb{Z}_p^*$.
  Prover computes label for bit 0: $w_o(0) = R_o$.
  Prover computes label for bit 1: $w_o(1) = R_o\Delta$.
  Prover computes hashes $h_{o,0} = \mathsf{Hash}(w_o(0))$, $h_{o,1} = \mathsf{Hash}(w_o(1))$.
  Prover sends $\{h_{o,\sigma}\}_{o\in O,\sigma\in 0,1}$ to $\mathcal{G}_{\mathsf{trust}}$.
  $\mathcal{G}_{\mathsf{trust}}$ stores hashes in a secure database indexed by prover identity and C.
  $\mathcal{G}_{\mathsf{trust}}$ issues commitment receipt c_{output} to prover.
end for
▷ Phase 3: Prover prepares garbled circuit and initiates cut-and-choose
Prover computes s garbled circuits $GC_1, \ldots, GC_s$ using labels $w_i(\sigma)$ and $w_o(\sigma)$.
Prover sends $\{GC_j\}_{j=1}^s$ to verifier V.
Prover stores nonces $\{R_i\}_{i\in I}$, $\{R_o\}_{o\in O}$, and receipts $c_{\mathsf{input}}, c_{\mathsf{output}}$.
V chooses random subset $J \subset [s]$ of size $s/2$ and sends J to P.
For each $j \in J$, P opens GC_j by sending keys and randomness to V.
V sends revealed labels to $\mathcal{G}_{\mathsf{trust}}$ for hash validation against $h_{i,\sigma}$ and $h_{o,\sigma}$.
$\mathcal{G}_{\mathsf{trust}}$ verifies hashes; V aborts if any mismatch.
Prover returns GC_j for $j \notin J$, c_{input}, c_{output} to the protocol.
▷ Phase 4: $\mathcal{G}_{\mathsf{trust}}$ validates inputs during evaluation (see Evaluator Interaction)
Upon receiving garbled input g_i for wire i from prover:
$\mathcal{G}_{\mathsf{trust}}$ retrieves stored hash h_{i,z_i} for bit $z_i \in 0,1$ corresponding to g_i.
if $\mathsf{Hash}(g_i) = h_{i,z_i}$ **then** $\mathcal{G}_{\mathsf{trust}}$ accepts g_i and notifies verifier.
else $\mathcal{G}_{\mathsf{trust}}$ rejects g_i, notifies verifier, and evaluator aborts.
end if

Remark 2.8 (Evaluator's Subsequent Role)**.** The evaluator interacts with $\mathcal{G}_{\mathsf{trust}}$ during the evaluation phase (detailed in the Evaluator Interaction Algorithm) to validate garbled inputs and outputs for $j \notin J$, ensuring security against a corrupt garbler. This occurs after cut-and-choose, using cleartext inputs obtained per algorithm De below.

Algorithm 2 Evaluator Interaction with $\mathcal{G}_{\mathsf{trust}}$ for Validation

Require:
Garbled circuits GC_j for $j \notin J$ and receipts c_{input}, c_{output} from prover. Cleartext input $x = x[0] \dots x[d-1]$ obtained from algorithm De. Trusted verification service $\mathcal{G}_{\mathsf{trust}}$ with stored hashes. Subset J from cut-and-choose (validated by $\mathcal{G}_{\mathsf{trust}}$).

▷ Step 1: Verify commitment receipts
Evaluator sends c_{input} and c_{output} to $\mathcal{G}_{\mathsf{trust}}$ for authentication.
$\mathcal{G}_{\mathsf{trust}}$ confirms receipt validity and hash storage; evaluator aborts if invalid.
▷ Step 2: Validate garbled inputs for $j \notin J$
for each input wire $i \in I$ corresponding to bit $x[i]$ **do**
    Prover provides garbled input g_i for w_i in GC_j ($j \notin J$).
    Evaluator sends g_i to $\mathcal{G}_{\mathsf{trust}}$.
    $\mathcal{G}_{\mathsf{trust}}$ retrieves h_{i,z_i} where $z_i = x[i]$.
    if $\mathsf{Hash}(g_i) = h_{i,z_i}$ **then** $\mathcal{G}_{\mathsf{trust}}$ accepts g_i and notifies evaluator.
    else $\mathcal{G}_{\mathsf{trust}}$ rejects g_i, notifies evaluator, and evaluator aborts.
    end if
end for
▷ Step 3: Evaluate GC_j and validate outputs
Evaluator computes garbled output Y using $\mathsf{Ev}(GC_j, g_i i \in I)$ for $j \notin J$.
Evaluator decodes Y to cleartext y using De and hashes garbled output components.
for each output wire $o \in O$ **do**
    Evaluator sends hashed garbled output for w_o to $\mathcal{G}_{\mathsf{trust}}$.
    $\mathcal{G}_{\mathsf{trust}}$ checks against $h_{o,\sigma}$ corresponding to decoded y.
    if mismatch occurs **then** $\mathcal{G}_{\mathsf{trust}}$ notifies evaluator, and evaluator aborts.
    end if
end for
▷ Step 4: Ensure majority consistency
Evaluator checks output consistency across GC_j for $j \notin J$; accepts if a majority agree on y.
return Cleartext output y if all validations and consistency checks pass.

Concretely, when using $\mathcal{G}_{\mathsf{trust}}$, we instantiate keys as tuples $\mathsf{key} = (\mathsf{sid}, \mathsf{pid}, C, i, \sigma)$, where sid is the UC session identifier, pid is the prover's identity, C is the circuit identifier, i is the wire index and $\sigma \in \{0, 1\}$ is the label bit. The commitment receipts c_{input} and c_{output} produced in Algorithms 1 and 2 are simply the collection of such keys together with their associated hashes stored via $\mathcal{G}_{\mathsf{trust}}.\mathsf{Put}$. Thus $\mathcal{G}_{\mathsf{trust}}$ itself does not need a special receipt type beyond first-write-wins storage.

2.6. **Step-by-step overview of OHMG.** Both parties commit to a circuit C computing F. We fix an elliptic curve $E(\mathbb{Z}_p)$ with prime p (specified in Section 4) and describe algorithms $(\mathsf{Gb}, \mathsf{En}, \mathsf{De}, \mathsf{Ev}, \mathsf{ev}) = \mathsf{OHMG}$.

Definition 2.9. Let G be a gate of circuit C; by combining Boolean gates, we consider *arithmetic* gates computing $f : \times_i \mathbb{Z}_{n_i} \to \times_j \mathbb{Z}_{m_j}$.

(1) The circuit's *wires* are primarily Boolean (for binary operations), but arrays of wires can carry arithmetic information (Definition 4.5). Wires carry data

as yes/no values, with some groups representing numbers via binary codes or single active wires for complex calculations.

(2) A *garbling* of a Boolean wire w_i uses two correlated nonces: $w_i^0 = w_i(0) = R_i$ and $w_i^1 = w_i(1) = R_i\Delta \in \mathbb{Z}_p^*$, tied to bits 0 (off) and 1 (on).
(3) Wire array types include: *one-hot tensor products*, *one-hot matrices*, *one-hot vectors*, and *binary vectors*.
(4) For a gate computing $f : \mathbb{Z}_n \to \mathbb{Z}_m$, one-hot vectors with n input wires and m output wires (one active per vector) are useful, contrasting with binary vectors where all 1 wires are on.
(5) A *word* wire carries arithmetic information as $\Omega(a) = r\Delta^a \in \mathbb{Z}_p^*$; for instance, an output wire can encode integer a modulo m.
(6) A garbling of cleartext a is either a word wire or an array with selected active wires.
(7) Input wires are Boolean, grouped to encode arithmetic inputs binary.
(8) The bin2hot primitive converts garbled inputs to one-hot vectors for arithmetic gates.
(9) Instead of garbling gate truth tables, we garble wires consistently, guiding the evaluator to compute garbled outputs $\breve{f}$.
(10) Some gates need no extra data, but others ($\mathcal{G}_k$) require a cyphertext help H_k for the evaluator to compute $\breve{f}_k$.
(11) For multi-variable functions, we construct a tensor product of one-hot input vectors (Definition 4.11), using H_k to evaluate it, enabling consistent computation.
(12) Transmitted data, the *material* of GC, includes generators and helps.
(13) The evaluator knows cleartexts a_i after garbler commitment, crucial for computing $\breve{f}_k$.

Algorithm 3 Garbling Algorithm Gb

Choose $\Delta \in \mathbb{Z}_p^*$.
Select generator G of elliptic curve, compute $G_c = [\Delta^c]G$ for $c \in \{-d, \ldots, -1, 0\}$.
for binary numbers with bits $d_1, d_2, \ldots, d_l \leq d$ at gates **do**
    $k_i \xleftarrow{\$} \mathbb{Z}_p^*$ and compute $G_{i,j_i} = [\Delta^{j_i} k_i]G$ for $i \in \{1, \ldots, l\}$, $j_i \in \{-d_i, \ldots, -1, 0\}$.
end for
Choose generator $G' = [K]G$, compute $G'_i = [\Delta^i]G'$ for $i \in \{\pm 1, \pm 2, \ldots, \pm(n-1)\}$.
Group input wires of C as N ordered arrays of length d, that is, w_i with $i \in \mathbb{Z}_{N \times d}$.
Generate $N \times d$ nonces $R_i \in \mathbb{Z}_p^*$ for input wires, set $w_i(z) = R_i \Delta^z$ for $z \in \{0, 1\}$.
Compute $2 \times I \times d$ hashes:

$$\mathsf{Hash}(w_i(0)) = \mathsf{Hash}(R_i), \quad \mathsf{Hash}(w_i(1)) = \mathsf{Hash}(R_i \Delta) \quad i \in I$$

Compute nonces for subsequent wires topologically per gate functionality.
Compute helps H_k for each gate $\mathcal{G}_k$ as specified.
For output wires w_o ($o \in O$), binary vectors for $y = F(x)$, compute hashes: $\mathsf{Hash}(w_o(0))$, $\mathsf{Hash}(w_o(1))$.
Call $\mathcal{G}_{\mathsf{trust}}.\mathsf{Put}(\mathsf{sid}, \mathsf{out} : o : \sigma, \mathsf{Hash}(w_o(\sigma)))$ for $o \in O$, $\sigma \in \{0, 1\}$.
Evaluator gets $\mathcal{G}_{\mathsf{trust}}.\mathsf{Get}(\mathsf{sid}, \mathsf{out} : o : y[o])$, accepts if $w_o(y[o])$ hashes to retrieved value and labels decode to y.
Send garbled circuit GC's material: G_{i,j_i}, G'_i, H_k, and hashes to evaluator for f_d.

2.6.1. *Algorithm* En. The *encoding function* f_e, handled only by the garbler, uses input wire nonces R_i and Δ to set wires on/off per the binary cleartext inputs.

2.6.2. *Algorithm* De. The decoding function d operates at inputs or outputs: the evaluator hashes garbled inputs, and compares them with $\mathsf{Hash}(w_i(\sigma))$ for $\sigma = 0, 1$ and $i \in I$ to decode binary inputs. Or she hashes garbled outputs, and compares them with $\mathsf{Hash}(w_o(\sigma))$ ($o \in O$) to decode binary outputs.

2.6.3. *Algorithm* Ev. The garbler provides garbled inputs, and the evaluator decodes cleartexts using De. Ev is presented in Algorithm 4 below.

Algorithm 4 Evaluation Algorithm Ev

Compute all gates off-chain using cleartext inputs and garblings (privately on Valery's system).
Use helps H_k to compute gates with one-hot tensor products $T_\otimes$ of input wires for gate $\mathcal{G}_k$.
Use generators G_{i,j_i} for $\mathsf{bin2hot}$ functionality.
Use generators G'_i for $\mathsf{word2hot}$ functionality.
Apply De on output wire garblings to obtain cleartext output.

2.6.4. *Security properties of OHMG.*

Lemma 2.10 (Correctness, honest prover)**.** *The garbling scheme OHMG is correct.*

Proof. All algorithms are deterministic. The claim is that for any circuit C and input x, $\mathsf{De}(f_d, \mathsf{Ev}(GC, \mathsf{En}(f_e, x))) = F(x)$.

Decoding relies on non-colliding hash functions, and evaluation's correctness follows from consistency (Definition 2.1), proved gate-by-gate, completing the proof. □

Remark 2.11.
(1) Obliviousness: This is not a feature in our scheme: most garbled gates require cleartext wire knowledge by design.
(2) Authenticity: Output nonce hashes are known via De, but as one-way functions, nonce recovery is infeasible. See Section 5 for a full proof.
(3) Correctness in the corrupt prover scenario: This is handled by our commitment scheme and the trusted verifying service, the full security proofs can be found in Section 5.3.

3. One hot encodings and cleartexts

This section is an overview of some ideas behind Heath's papers [20] for operations in $\mathbb{Z}_p$ on one-hot encodings, and their approach to garbled circuits. All statements in this section are about cleartexts, however in the approach taken by Heath and Kolesnikov, most of it can be used for cyphertexts, masking the cleartexts x with a uniform mask α. This is possible in their scheme, given certain homomorphic features of their garblings. This masking -as far as we know- is not possible in our OHMG scheme presented in Section 4, which is designed for privacy-free applications. We begin this section with some notations:
We will be dealing with numbers given in its arithmetic expression (that is a word $a \in \mathbb{Z}_n$) but we will also be interested in their binary expression $\mathsf{bin}(a) \in \{0,1\}^d$. Therefore it will be convenient to distinguish two one-hot encodings, first the one for the binary expression (of length d) and then the one for the arithmetic expression (of length $n = 2^d$).

Definition 3.1 (Binary one-hot encoding)**.** Let $\mathcal{H} : \{0,1\}^d \to \{0,1\}^{2^d}$ be the map that assigns to a binary expression $z \in \{0,1\}^d$ the vector with only 1 in the position corresponding to z, and every other entry is 0. For instance for $d = 1$ we map

$$0 \mapsto (1,0) \qquad 1 \mapsto (0,1).$$

While for $d = 2$ we map

$$00 \mapsto (1,0,0) \qquad 01 \mapsto (0,1,0) \qquad 10 \mapsto (0,0,1).$$

It will be convenient to count indices from $i = 0$; that is an element $x \in \{0,1\}^k$ will be denoted $x = (x_0, x_1, \ldots, x_{k-1})$, hence the "first place" is in fact x_0. We will denote

$$\mathcal{H}(z)_k \in \{0,1\} \qquad z \in \{0,1\}^d, \quad k \in \mathbb{Z}_{2^d}$$

to the k-th entry of $\mathcal{H}(z)$.

Remark 3.2. Identifying $\{0,1\} \simeq \mathbb{Z}_2$, note that $\mathcal{H} : \mathbb{Z}_2^d \to \mathbb{Z}_2^{2^d}$ is *not* linear, however its image is the canonical basis of $\mathbb{Z}_2^{2^d}$. This observation will be useful later.

Definition 3.3 (Matrix of a boolean map)**.** For a Boolean function $f : \{0,1\}^d \to \{0,1\}^n$ we denote by $f(z)_j$ to the j-th entry of $f(z)$, $z \in \{0,1\}^d$. We also denote with $\mathcal{T}(f)$ the $2^d \times n$ matrix (2^d rows and n columns) given by its truth table, with inputs and outputs ordered increasingly, so the i-th row is the full bit string $f(z)$.

For instance if $n = 3$ and $d = 2$

$$\mathcal{T}(f) = \begin{pmatrix} f(00)_0 & f(00)_1 & f(00)_2 \\ f(01)_0 & f(01)_1 & f(01)_2 \\ f(10)_0 & f(10)_1 & f(10)_2 \\ f(11)_0 & f(11)_1 & f(11)_2 \end{pmatrix}$$

where each $f(z_i z_j)_k$ is a bit.

Remark 3.4. If $a \in \{0,1\}^n$ and $f : \{0,1\}^n \to \{0,1\}^d$, then $\mathcal{T}(f)^T \cdot \mathcal{H}(a)^T = f(a)^T$. Indeed, if we multiply $\mathcal{T}(f)^T$ on the right by the vector column $\mathcal{H}^T$ (which is only non-zero in the a-th position), we recover its a-th column, which is exactly $f(a)$ written as a column.

Remark 3.5 (Linear interpretation)**.** For $f : \{0,1\}^n \to \{0,1\}^d$ consider the linear mapping $\widehat{f} : \mathbb{Z}_2^{2^n} \to \mathbb{Z}_2^d$ defined on the basis $\{\mathcal{H}(a)\}_{a\in\{0,1\}^n}$ as

$$\widehat{f}(\mathcal{H}(a)) = f(a)$$

(Remark 3.2). Then $\mathcal{T}(f)^T$ is just the matrix of $\widehat{f}$ in the canonical bases that is $M_{\widehat{f}} = \mathcal{T}(f)^T$. Therefore Remark 3.4 can be interpreted as the standard computation of $\widehat{f}$ using its matrix.

Definition 3.6 (Tensor products)**.** Also known as the *outer product* of vectors, for a given ring $\mathbb{F}$ and $a \in \mathbb{F}^n$, $b \in \mathbb{F}^m$, it is defined as the matrix

$$a \otimes b = a^T \cdot b = \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{pmatrix} \begin{pmatrix} b_1 & b_2 & \cdots & b_m \end{pmatrix} = (a_i b_j)_{ij} \in \mathbb{F}^{n\times m}.$$

Note that there is an identification of $a \otimes b$ with a bilinear mapping from $\mathbb{F}^n \times \mathbb{F}^m \to \mathbb{F}$, implemented by $(x, y) \mapsto x \cdot a^T \cdot b \cdot y^T = x \cdot a^T \cdot y \cdot b^T = a \cdot x^T \cdot b \cdot y^T$. The coefficients $a_i b_j$ can be recovered evaluating this bilinear mapping in the canonical bases.

Definition 3.7. In general, if $a_i \in \mathbb{F}^{n_i}$ for $i \in \{0, \ldots, I-1\}$, the entries of their *tensor product* are indexed by the number of inputs as follows:

$$(a_0 \otimes \cdots \otimes a_{I-1})_{k_0 k_2 \cdots k_{I-1}} = \prod_{i=0}^{n_{I-1}} (a_i)_{k_i}$$

where $k_i = 0, 1, \ldots, n_{I-1}$ and $(x)_k$ indicates the k-th entry of x.

As before, we can identify any tensor product $a_1 \otimes \cdots \otimes a_J$ with a multi-linear mapping

$$(x_1, \ldots, x_I) \mapsto x_1 \cdot a_1^T x_2 \cdot a_2^T \ldots x_I \cdot a_I^T \in \mathbb{F}, \qquad x_i \in \mathbb{F}^{n_i}.$$

Lemma 3.8. *Let $x \in \mathbb{F}^k, y \in \mathbb{F}^r$, let $g : \mathbb{F}^k \to \mathbb{F}^d$ and $h : \mathbb{F}^r \to \mathbb{F}^s$ be linear. Let M_g, M_h be their respective matrices in the canonical bases. Then*

$$M_g(x \otimes y) = g(x) \otimes y \qquad \text{and} \qquad (x \otimes y)M_h^T = x \otimes \mathcal{H}(y).$$

Proof. By the associativity of the matrix product

$$M_g \cdot (x^T \cdot y) = (M_g \cdot x^T) \cdot b = g(x)^T \cdot y = g(x) \otimes y,$$

and likewise

$$(x \otimes y)M_h^T = x^T \cdot y \cdot M_h^T = x^T \cdot (M_h \cdot y^T)^T = x^T \cdot (\mathcal{H}(y)^T)^T = x \otimes \mathcal{H}(y)$$

□

Corollary 3.9. *Let $a \in \{0,1\}^n$, $b \in \{0,1\}^m$. If $f : \{0,1\}^n \to \{0,1\}^d$ then*

$$\mathcal{T}(f)^T(\mathcal{H}(a) \otimes b) = f(a) \otimes b$$

while if $f : \{0,1\}^m \to \{0,1\}^k$ then $(a \otimes \mathcal{H}(b))\mathcal{T}(f) = a \otimes f(b)$.
In particular if $f = id_n$ is the identity map of $\{0,1\}^n$ (resp. of $f = id_m$ of $\{0,1\}^m$), then

$$\mathcal{T}(id_n)^T \cdot (\mathcal{H}(a) \otimes b) = a \otimes b = (a \otimes \mathcal{H}(b))\mathcal{T}(id_m).$$

Proof. Let $\mathbb{F} = \mathbb{Z}_2$ and let $g = \widehat{f}$ as in our linear interpretation of Remark 3.5. Take $k = 2^n$, $x = \mathcal{H}(a)$ and $y = b$. Then by the previous lemma

$$\mathcal{T}(f)^T(\mathcal{H}(a) \otimes b) = M_{\widehat{f}}(\mathcal{H}(a) \otimes b) = \widehat{f}(\mathcal{H}(a)) \otimes b = f(a) \otimes b,$$

which proves the first assertion. The second assertion has a similar proof, again using the previous lemma and taking $h = \widehat{f}$, $r = 2^m$, $x = a$, $y = \mathcal{H}(b)$. The assertions for the identity maps are straightforward applications of what we've just proved. □

Definition 3.10. The *arithmetic one-hot encoding* $\delta : \mathbb{Z}_n \to \mathbb{Z}_2^n$ is given by

$$a \mapsto \delta(a) = (\delta_0(a), \ldots, \delta_{n-1}(a)) = (a_0, \ldots, a_{n-1}) \in \{0,1\}^n,$$

where $\delta_i(a) = \delta_{i,a}$ that is $a_i = 1$ in position a, and otherwise $a_i = 0$.

Remark 3.11. Note that for $n = 2$ both one-hot encondings agree, that is $\mathcal{H} = \delta : \{0,1\} = \mathbb{Z}_2 \to \{0,1\}^2 = \mathbb{Z}_2^2$. Moreover, when $n = 2^d$, then from the very definitions we have

$$\delta(a) = \mathcal{H}(\mathsf{bin}(a)) \qquad a \in \mathbb{Z}_n. \tag{1}$$

In the general case, to obtain $\delta(a)$ one has to add zeroes at the end of $\delta(a)$ to obtain this identity; for instance if $n = 6$, then $d = 3$ and we have $\delta(3) = (0,0,0,1,0,0)$ while $\mathcal{H}(011) = (0,0,0,1,0,0,0,0)$.

Remark 3.12. If $f : \mathbb{Z}_n \to \{0,1\}^d$, then we say that

$$\mathcal{T}(f) = \begin{pmatrix} f(0)_0 & f(0)_1 & \cdots & f(0)_{d-1} \\ f(1)_0 & \cdots & \cdots & \cdots \\ \vdots & \vdots & \vdots & \vdots \\ f(n-1)_0 & \cdots & \cdots & f(n-1)_{d-1} \end{pmatrix}$$

is the *truth table* of f. Reasoning as in binary one-hot (with linear $\hat{f}$ defined on the basis as $\hat{f}(\delta(a)) = f(a)$, $a \in \mathbb{Z}_n$) it is plain that for each $a \in \mathbb{Z}_n$ we have

$$\mathcal{T}(f)^T \cdot \delta(a)^T = f(a)^T. \tag{2}$$

In particular for $f = \mathsf{bin} : \mathbb{Z}_n \to \{0,1\}^d$ (with $d = \lceil \log_2 n \rceil$) the map assigning to each $a \in \mathbb{Z}_n$ its binary expression, we have $\mathcal{T}(\mathsf{bin})^T \delta(a)^T = \mathsf{bin}(a)^T$.

Mixing binary encondings with one-hot encondings, we now have an analogue of Corollary 3.9, replacing the binary one-hot encoding $\mathcal{H}$ with the arithmetic one-hot encoding δ, with an identical proof:

Corollary 3.13. *If* $f:\mathbb{Z}_n\to\{0,1\}^d$ *then*

$$\mathcal{T}(f)^T(\delta(a)\otimes b)=f(a)\otimes b \qquad a\in\mathbb{Z}_n,\ b\in\mathbb{Z}_s^l.$$

If $f:\mathbb{Z}_m\to\{0,1\}^k$, *then* $(a\otimes b)\mathcal{T}(f)=a\otimes f(b)$ *for* $a\in\mathbb{Z}_r^l$, $b\in\mathbb{Z}_m$.
In particular if $f=\mathsf{bin}:\mathbb{Z}_*\to\{0,1\}^*$, *then* $\mathcal{T}(\mathsf{bin})^T(\delta(a)\otimes b)=\mathsf{bin}(a)\otimes b$ *and*

$$\mathcal{T}(\mathsf{bin})^T(\delta(a)\otimes\delta(b))\mathcal{T}(\mathsf{bin})=\mathsf{bin}(a)\otimes\mathsf{bin}(b).$$

3.1. **Computing functionalities in one-hot encoding.** In this section we are interested in discussing the computation of a function f when the input is encoded in one-hot, and we *also want a one-hot output* (compare with Remarks 3.4 and 3.12). We begin with the simplest case:

Lemma 3.14 (Arithmetic one-hot to arithmetic one-hot functions)**.** *If* $f:\mathbb{Z}_n\to\mathbb{Z}_m$, *then*

$$\delta_j(f(a))=\sum_{i\in\mathbb{Z}_n\,:\,f(i)=j}\delta_i(a) \qquad\qquad j\in\mathbb{Z}_m.$$

If $f:\mathbb{Z}_n\times\mathbb{Z}_m\to\mathbb{Z}_r$, *then*

$$\delta_k(f(a,b))=\sum_{i,j\,:\,f(i,j)=k}\delta_i(a)\delta_j(b)=\sum_{i,j\,:\,f(i,j)=k}(\delta(a)\otimes\delta(b))_{ij} \qquad\qquad k\in\mathbb{Z}_r$$

Proof. Straightforward computations. □

Remark 3.15.

(1) The ideas we discussed can be extended to functions of the form $f:\mathbb{Z}_{n_1}\times\mathbb{Z}_{n_2}\times\cdots\times\mathbb{Z}_{n_I}\to\mathbb{Z}_{k_1}\times\cdots\times\mathbb{Z}_{k_J}$: first we consider the components of f, which are J maps $f_j:\mathbb{Z}_{n_1}\times\cdots\times\mathbb{Z}_{n_I}\to\mathbb{Z}_{n_j}$, $j=1,\dots J$. Each of these maps can be extended to a multilinear map $\widehat{f_j}:Z_{n_1}\otimes\cdots\otimes\mathbb{Z}_{n_I}\to\mathbb{Z}_{n_j}$, defined on a basis of the tensor product as $\widehat{f_j}(\delta(a_1)\otimes\cdots\otimes\delta(a_I))=f_j(a_1,\dots,a_I)$. With these, it is possible to obtain properties that generalize Corollaries 3.9 and 3.13, and also Lemma 3.14. We will not pursue these generalizations in this paper, with one notable exception that follows, that will be used to obtain a constructive translation from binary encodes to arithmetic one-hot encodes.

(2) For $h_i=((h_i)_0,(h_i)_1)\in\{0,1\}^2$, consider the tensor product

$$(\bigotimes_{i\in\mathbb{Z}_d}h_i)_X=(h_0\otimes\cdots\otimes h_{d-1})_{x_0\dots x_{d-1}}=\prod_{i=0}^{d-1}(h_i)_{x_i} \qquad X=x_0\dots x_{d-1}\in\{0,1\}^d.$$

Note that for each $h=(h_0,\dots,h_{d-1})$ there is exactly one X which makes the product equal to 1, and otherwise the product is zero.

- We adopt the convention that a product over an empty set equals to 1, while the sum over an empty set equals to 0.
- For $k\in\mathbb{Z}_n$, we consider its binary expression $\mathsf{bin}(k)$ as an element of $\{0,1\}^d$, where $d=\lceil\log_2 n\rceil$.

Lemma 3.16 (Binary to arithmetic one-hot/binary functions)**.** *Let* $f:\{0,1\}^d\to\{0,1\}^n$, $Z=z_0\dots z_{d-1}\in\{0,1\}^d$. *Then for* $\mathcal{H}:\{0,1\}\to\{0,1\}^2$ *and each* $k\in\mathbb{Z}_n$ *we have*

$$f(Z)_k=\sum_{X\in\{0,1\}^d\,:\,f(X)_k=1}(\mathcal{H}(z_0)\otimes\cdots\otimes\mathcal{H}(z_{d-1}))_X.$$

In particular if $n = 2^d$ *then for* $a \in \mathbb{Z}_n$ *and each* $k \in \mathbb{Z}_n$ *we have*

$$\delta_k(a) = (\mathcal{H}(a[0]) \otimes \cdots \otimes \mathcal{H}(a[d-1]))_{\mathsf{bin}(k)}$$

where $a[0] \dots a[d-1] = \mathsf{bin}(a)$ *as usual.*

Proof. Note that $(\bigotimes_{i \in \mathbb{Z}_d} h_i)_X = 1$ if and only if $(h_i)_{x_i} = 1$ for all i. In our case $h_i = \mathcal{H}(z_i)$, hence the tensor product is non-zero if and only if $x_i = z_i$, that is $X = Z$. Now if $f(Z)_k = 1$, then both sides are equal to 1 and we are done. If $f(Z)_k = 0$, then for all the X such that $f(X)_k = 1$ the tensor product is zero (being $X \neq Z$), hence both sides are now equal to 0 and we are also done.
For the second assertion, take $f = \mathcal{H} : \{0,1\}^d \to \{0,1\}^{2^d}$ and note first that $\delta_k(a) = (\mathcal{H} \circ \mathsf{bin})(a))_k = f(\mathsf{bin}(a))_k$ for any $a \in \mathbb{Z}_n$. Then note that there is exactly one $X \in \{0,1\}^d$ such that $\mathcal{H}(X)_k = 1$, which is exactly $X = \mathsf{bin}(k)$. □

3.2. **Dictionaries: cleartexts.** In this section we describe the (well-known) implementations that give translations between four expressions of a number $a \in \mathbb{Z}$: the binary expression, the (arithmetic) one-hot expression, the (binary) one-hot expression and the *arithmetic* expression (this is the standard decimal expression of a, also called *word* in [20]).

(1) *Arithmetic to binary*: This one is implemented by the usual algorithm, that is $\mathsf{bin}(a)$ it the binary expression of a.
(2) *Arithmetic one-hot to arithmetic*: This one implements the inverse map of δ, and it is easily implemented by $\delta(a) = (a_0, \cdots, a_{n-1}) \mapsto \sum_{i=0}^{n-1} i \cdot a_i = a$.
(3) *Binary to arithmetic*: It is plain that the map it is given by

$$a[0] \dots a[d-1] = \mathsf{bin}(a) \mapsto \sum_{i=0}^{d-1} a[i] \cdot 2^i = a.$$

(4) *Arithmetic one-hot to binary*: Let $\mathsf{bin} : \mathbb{Z}_n \to \{0,1\}^d$ (with $d = \lceil \log_2 n \rceil$) be the map assigning to each $a \in \mathbb{Z}_n$ its binary expression, let $\mathcal{T}(\mathsf{bin})$ be its truth table, for instance

$$\mathcal{T}(\mathsf{bin})^T = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{pmatrix} \qquad \text{for } n = 4,$$

see Remark 3.12. Then translation from one hot to binary is given by

$$(a_0, \cdots, a_{n-1}) = \delta(a) \mapsto \mathcal{T}(\mathsf{bin})^T \delta(a)^T = \mathsf{bin}(a)^T.$$

Definition 3.17. For $X \in \{0,1\}^d$ we introduce the operator $T_X : \{0,1\}^d \to \{0,1\}^d$ defined as $(T_X Y)[i] = Y[i]X[i] + \overline{Y[i]}\ \overline{X[i]}$ for $Y \in \{0,1\}^d$. Note that T_X flips bits of Y: the i-th bit of Y is flipped if and only if the i-th bit of X is zero (otherwise it is left untouched). Fix n and for $d = \lceil \log_2(n) \rceil$ we let $\mathsf{b2h} : \{0,1\}^d \to \{0,1\}^n$ be given by $\mathsf{b2h} : Y = \mathsf{bin}(a) \mapsto \prod_{i=0}^{d-1} (T_{\mathsf{bin}(k)} Y)[i] = \delta_k(a)$ for $k \in \mathbb{Z}_n$.

Example 3.18. When $n = 3$ we have $d = 2$ hence $\mathsf{b2h} : y_0 y_1 \mapsto (\overline{y_0}\,\overline{y_1}, y_0\,\overline{y_1}, \overline{y_0}\,y_1)$, for $n = 4$ we also have $d = 2$ and $\mathsf{b2h} : y_0 y_1 \mapsto (\overline{y_0}\,\overline{y_1}, y_0\,\overline{y_1}, \overline{y_0}\,y_1, y_0\,y_1)$, while for $n = 5$ we have $d = 3$ with $\mathsf{b2h} : y_0 y_1 y_2 \mapsto (\overline{y_0}\,\overline{y_1}\,\overline{y_2}, y_0\,\overline{y_1}\,\overline{y_2}, \overline{y_0}\,y_1\,\overline{y_2}, y_0\,y_1\,\overline{y_2}, \overline{y_0}\,\overline{y_1}\,y_2)$.

Remark 3.19 (Computing b2h using tensor products)**.** This alternative presentation using tensor products will be useful later: we leave to the reader to check that

$$\prod_{k=0}^{d-1} T_X(Z)[i] = (\mathcal{H}(z_0) \otimes \cdots \otimes \mathcal{H}(z_{d-1}))_X \quad \text{for any } X, Z \in \{0,1\}^d.$$

Then by means of Lemma 3.16 and the previous equality it is easy to check that

Lemma 3.20 (Binary to arithmetic one hot)**.** *The map*

$$\mathsf{b2h} : Z \mapsto (\mathcal{H}(z_0) \otimes \cdots \otimes \mathcal{H}(z_{d-1}))_{\mathsf{bin}(k)} \text{ for } k \in \mathbb{Z}_n$$

implements a translation from the binary expression $Z = \mathsf{bin}(a)$ *of* $a \in \mathbb{Z}_n$ *to the arithmetic one-hot encoding of* a*, that is*

$$\mathsf{b2h}(\mathsf{bin}(a)) = \delta(a), \qquad \mathsf{bin}(a) \in \{0,1\}^d, \quad d = \lceil \log_2 n \rceil.$$

4. OHMG: One hot modular garbling

In this section we describe how to represent numbers and functions in finite fields, and we describe our proposed mechanism for describing a garbled tensor product that will allows to garble *any function* at each gate G_i with message cost $O(1)$. Our garbling scheme OHMG is based on the tensor product of arithmetic one-hot garblings.

Remark 4.1. As we described in the introduction, the evaluator knows the garblings of the input wires, but also their cleartexts. The inputs are chosen by the garbler, who also garbles the circuit. Then the garbler provides to the evaluator a ZKP that this garbled circuit if actually computing the chosen (agreed upon both parties) circuit C.

Remark 4.2 (Elliptic curve)**.** Throughout this paper we will be using the elliptic curve secp256k1 (see https://www.secg.org), given by the Weierstrass equation $y^2 = x^3 + 7$ over the finite field $\mathbb{Z}_p$, with $p = 2^{256} - 2^{32} - 977$. We let G be a generator in the curve, and we use multiplicative notation for the curve operation that is $[2]G = G \cdot G$. For the garbling of all the wires of a full circuit, we fix a *correlation* $\Delta \in \mathbb{Z}_p^*$,

Definition 4.3 (EC-safe corelation, nonces and labels)**.** A value $a \in \mathbb{Z}_p^*$ is *EC-safe* if it satisfies $a \neq 0 \pmod q$, where q is the order of the generator G of the secp256k1 curve. This ensures $[a]G$ is not mapped to the origin of the group. We will ask that all our labels for the wires are EC-safe. For the global correlation Δ, additionally we require $\Delta \neq 1 \pmod q$, ensuring $[\Delta]G \neq G$ is not a trivial hidden correlation.

Remark 4.4. In applications, since the order of the curve is quite similar to p, we have that for $a \in \mathbb{Z}_p^*$, we have that $a \pmod q$ maps to $\{0, \ldots, q-1\}$ with bias $O((p-q)/p) \approx 2^{-224}$. Hence $a = 0 \pmod q$ with probability $\approx 2^{-256}$, which is negligible. A similar reasoning can be applied to the condition $\Delta \neq 1 \pmod q$, so from now we will simply say $R, \Delta \in \mathbb{Z}_p^*$. So all non-zero randomly chosen correlation and nonces are statistically EC-secure.
In protocols requiring field inverses in $\mathbb{Z}_p^*$ (for instance, Δ^{-1} for $G_{-1} = [\Delta^{-1}]G$ in tensor products and not), the effective scalar multiplication is mod q. For $a \in \mathbb{Z}_p^*$, $a^{-1} = b$ mod p satisfies $a \cdot b = 1 + kp$. Then $[(a \cdot b)]G = G + k \cdot ([p]G) = G + [k \cdot r]G$, where $r = p$ mod $q \neq 0$ for secp256k1. This equals G if, and only if, $kr \equiv 0 \pmod q$, that is, $k \equiv 0$ mod q (since r is invertible mod q). For random a, k is distributed such that $k \equiv 0$

mod q with probability $1/q \approx 2^{-256}$, negligible. Thus, field inverses act as expected group inverses with overwhelming probability, ensuring protocols consistency.

4.1. **Wires, nonces, and vectors.**

Definition 4.5 (Types of Wires)**.** Our garbling scheme makes use of two types of garblings:

(1) Boolean wire: conformed by a random nonce $R \in \mathbb{Z}_p^*$ and boolean variable $z \in \{0,1\}$, defined as $w(z) = R\Delta^z$.
(2) Word wire: conformed by a random nonce $r \in \mathbb{Z}_p^*$ and a modular variable $x \in \mathbb{Z}_n$, defined as $\Omega(Z) = r\Delta^x$.

We say that the Boolean wire is *on* (or *hot*) when $z = 1$ and its *off* (or *cold*) when $z = 0$. Let $\mathcal{W}$ be the set of all the binary wires of the circuit C, numbered in the set $\{0, 1, \ldots, W-1\}$. For $k \in \mathcal{W}$, we denote $w_k(z) = R_k\Delta^k$ the k-th wire of the circuit.

Remark 4.6. As we already mentioned, we will not be using the standard notation for a Boolean garbled wire turned off $w_i^0 = R_i$ or turned on $w_i^1 = R_i\Delta$, since the super-index will collide with the exponentiation of numbers in $\mathbb{Z}_p$.

For the following definition, recall that $\delta_i(x) = 0$ for $i \neq x$ and $\delta_x(x) = 1$ that is δ is Dirac's delta function.

Definition 4.7 (Arrays of wires)**.** Wires can be arranged in certain specific ways, for convenience:

(1) A *binary vector of wires* with d-digits is an ordered set of d Boolean wires, denoted $B(Z) = (w_0(z_0), w_1(z_1), \cdots, w_{d-1}(z_{d-1}))$, for $Z = z_0 z_1 \cdots z_{d-1} \in \{0,1\}^d$. We denote with $B(Z)_i = w_i(z_i) = R_i\Delta^{z_i}$ to the i-th wire of the vector.
(2) A *one-hot vector of wires* is the composition of the arithmetic one-hot encoding map $\delta : \mathbb{Z}_n \to \mathbb{Z}_2^n$ with a binary n-vector of wires:

$$\begin{aligned} W(x) = (B \circ \delta)(x) &= (w_0(\delta_0(x)), \ldots, w_{n-1}(\delta_{n-1}(x)) \\ &= (R_0\Delta^{\delta_0(x)}, \ldots, R_{n-1}\Delta^{\delta_{n-1}(x)}), \qquad x \in \mathbb{Z}_n. \end{aligned}$$

Equivalently, it is a vector of wires, with one and only one hot entry. We denote $W(x)_j = R_j\Delta^{\delta_j(x)}$ to the j-th wire of the vector.

Definition 4.8 (Indexing of wires, vectors, and nonces)**.** If $A = \{i_0, \ldots, i_{d-1}\}$ is an *ordered* subset of the index set $\mathcal{W}$ of the binary wires, we indicate with

$$B_A = (w_{i_0}, w_{i_1}, \ldots, w_{i_{d-1}}),$$

and we denote $B_A(Z)_k = w_{i_k}(z_{i_k}) = R_{i_k}\Delta^{z_{i_k}}$. Likewise we let

$$W_A = (w_{i_0} \circ \delta_0\,,\, w_{i_1} \circ \delta_1\,,\, \ldots\,,\, w_{i_{d-1}} \circ \delta_{d-1})$$

and we denote $W_A(x)_k = R_{i_k}\Delta^{\delta_k(x)}$, where it is assumed that $x \in \mathbb{Z}_d$.
For $k = 0, \ldots, d-1$, it will be useful to denote also

$$w_{A,k} := w_{i_k} \qquad R_{A,k} := R_{i_k} \qquad i_k \in A$$

when possible, to make explicit the set A, and avoid the double sub-index notation. With this notation $W_A(x)_k = R_{i_k}\Delta^{\delta_k(x)} = w_{A,k}(x) = R_{A,k}\Delta^{\delta_k(x)}$. Note the two possible notations $W_A(x)_k = w_{A,k}(x)$, each one will be used in the most convenient scenario (that is when we want to emphasize that we are picking a coordinate of the vector, or when we are making computations with coordinates.

Remark 4.9. For the input wires, which are all Boolean, the garbler chooses randomly all of their nonces. The nonces for all the other wires are obtained from these ones, by going through a procedure for each gate G_i, in topological order of the gates. The procedure for each type of gate will be specified below. The following auxiliary notions will also be useful.

Definition 4.10 (Garblings)**.** Let $a \in \mathbb{Z}_n$

(1) A *word garbling* of cleartext $a \in \mathbb{Z}_n$ is obtained by evaluating a word wire $\Omega(a) = r\Delta^a$.
(2) A *binary garbling* of cleartext $a \in \mathbb{Z}_n$ is obtained by evaluating a binary vector of wires at $Z = \mathsf{bin}(a) = a[0] \dots a[d-1]$, that is
$$B(\mathsf{bin}(a)) = (R_0\Delta^{a[0]}, \dots, R_{d-1}\Delta^{a[d-1]}).$$
(3) A *one-hot garbling* of a is obtained by evaluating a one-hot vector of wires, that is $W(a) = (R_0\Delta^{\delta_0(a)}, \dots, R_{n-1}\Delta^{\delta_{n-1}(a)})$.

Definition 4.11 (One-hot garbled tensor products)**.** A one-hot *tensor product* of order 2 is a $n \times m$ matrix (also called *one-hot matrix*) of Boolean wires
$$w_{i,j}(z_{i,j}) = R_{i,j}\Delta^{z_{i,j}}, \quad z_{i,j} \in \{0,1\}, \quad i \in \mathbb{Z}_n, j \in \mathbb{Z}_m$$
with $R_{i,j} \in \mathbb{Z}_p^*$ and all of its entries but one turned off (that is it has only one hot entry).
Given cleartexts $a \in \mathbb{Z}_n$, $b \in \mathbb{Z}_m$, a *one-hot garbled tensor product* is obtained by setting the hot entry of T exactly at position $i = a$, $j = b$; this ordered set of garblings is denoted as $T(a,b)$, and its i,j entry is denoted as $T(a,b)_{ij}$.
In general, a *one-hot tensor product* of order d is a indexed collection of Boolean wires
$$T = R_{i_0,\dots,i_{d-1}}\Delta^{z_{i_0,\dots,i_{d-1}}}, \quad z_* \in \{0,1\}, \quad i_k \in \mathbb{Z}_{n_k}, \quad R_* \in \mathbb{Z}_p^*,$$
with all of its entries but one turned off (that is it has only one hot entry).
Given cleartexts $a_k \in \mathbb{Z}_{n_k}$, we set the hot entry of T at $i_k = a_k$ for all $k = 0, \dots, d-1$ to obtain a *one-hot garbled tensor product* of the cleartexts, denoted $T(a_0, \dots, a_{d-1})$. it is $(i_0, \dots, i_{d-1})$-th entry will be denoted as $T(a_0, \dots, a_{d-1})_{i_0 \dots i_{d-1}}$.

We now describe the available gates, how to compute the nonces of the output wires knowing the nonces of the input wires (done by the garbler), and how to compute the garbled output given the garbled input (done by the evaluator). We will do this for several types of functions.

Definition 4.12. For a given gate G of circuit C computing the functionality f, we will denote with $\breve{f}$ the operation on garblings that produced the garbled output of gate G.

4.2. **Garbling $f : \mathbb{Z}_n \to \mathbb{Z}_m$ with one-hot inputs and outputs.** Let G be the gate computing f, let W_I, W_O be its (one-hot) input and output vectors of wires.

Definition 4.13 ($f : \mathbb{Z}_n \to \mathbb{Z}_m$. Output nonces from input nonces)**.** Let $I, O \subset \mathcal{W}$ be sets of Boolean wires of circuit C, the first of cardinal n and the second of cardinal m. Given the nonces $R_{I,i}$ ($i \in \mathbb{Z}_n$) of the one-hot input vector of wires W_I, the nonces $R_{O,k}$ for the one-hot output vector of wires W_O are defined (computed by the garbler) as follows: $R_{O,k} = \prod_{i \in \mathbb{Z}_n \,:\, f(i)=k} R_{I,i}, \qquad k \in \mathbb{Z}_m.$

Definition 4.14 ($f : \mathbb{Z}_n \to \mathbb{Z}_m$. Computing $\breve{f}$ for free)**.** The garbled output of the gate G computing f with garbled input $W_I(a)$ is $\breve{f} : \mathbb{Z}_p^n \to \mathbb{Z}_p^m$ defined as

$$\breve{f}(W_I(a))_k := \prod_{i\in\mathbb{Z}_n \,:\, f(i)=k} W_I(a)_i = \prod_{i\in\mathbb{Z}_n \,:\, f(i)=k} R_{I,i}\Delta^{\delta_i(a)} \tag{3}$$

For the following theorem (and every time we prove consistency) it is convenient to have in mind Definition 2.3, where consistency its just a way we express correctness in our approach.

Theorem 4.15 ($f : \mathbb{Z}_n \to \mathbb{Z}_m$. Consistency of wire garblings)**.** *The garbled one-hot vector output of gate G computing $f(a)$ is the output vector of wires of gate G evaluated at $f(a)$. That is: $W_O(f(a)) = \breve{f}(W_I(a))$.*

Proof. By definition, since the vector of wires W_O has nonces $R_{O,k}$, we have that $W_O(f(a))_k = R_{O,k}\Delta^{\delta_k(f(a))}$ for each $k \in \mathbb{Z}_m$. That is

$$W_O(f(a))_k = \prod_{i\in\mathbb{Z}_n \,:\, f(i)=k} R_{I,i}\Delta^{\delta_k(f(a))}.$$

But $\delta_k(f(a)) = 0$ unless $k = f(a)$, in which case $\delta_k(f(a)) = 1$, hence

$$W_O(f(a))_k = \begin{cases} \displaystyle\prod_{i\in\mathbb{Z}_n \,:\, f(i)=k} R_{I,i} = \prod_{i\in\mathbb{Z}_n \,:\, f(i)=k} R_{I,i}\Delta^{\delta_i(a)} & k \neq f(a) \\ \displaystyle\prod_{i\in\mathbb{Z}_n \,:\, f(i)=f(a)} R_{I,i}\Delta = \prod_{i\in\mathbb{Z}_n \,:\, f(i)=f(a)} R_{I,i}\Delta^{\delta_i(a)} & k = f(a) \end{cases};$$

the equality in the first line because in this case a does not belong to the product index set, hence $\delta_i(a) = 0$. The equality in the second line is because in this case, a belongs to the product index set, with one occurrence exactly when $i = a$. Comparing with (3), we have completed the proof of the theorem. □

Remark 4.16 ($f : \mathbb{Z}_n \to \mathbb{Z}_m$. Execution by the evaluator)**.** As mentioned, this functionality can be implemented for free (no material is transmitted). It is a multiplicative analogue of (2) and Lemma 3.14. Note that at this point the evaluator, knowing a and f, knows the cleartext $f(a)$ hence she knows the garbling of this output that is she knows the exact *position* $b = f(a)$ which is hot in this garbled vector of wires. Hence she can use this as the input of another functionality.

Lemma 4.17 (Compositional properties of OHMG)**.** *Let $f : \mathbb{Z}_n \to \mathbb{Z}_m$ and $g : \mathbb{Z}_m \to \mathbb{Z}_d$. Then $\widetilde{g \circ f} = \breve{g} \circ \breve{f}$.*

Proof. Let W_I be the input vector of wires of the first gate computing f. First note that $\breve{g}(\breve{f}(W_1(a))_\ell = \displaystyle\prod_{k\,:\,g(k)=\ell} \prod_{i\,:\,f(i)=k} W_I(a)_i$. On the other hand

$$\widetilde{(g \circ f)}(W_1(a))_\ell = \prod_{i\,:\,g(f(i))=\ell} W_I(a)_i.$$

The proof then follows from the elementary set equality:

$$\{i \in \mathbb{Z}_n : g(f(i)) = \ell\} = \bigsqcup_{k\in\mathbb{Z}_m} \{i \in \mathbb{Z}_n : f(i) = k \ \ \textbf{AND} \ \ g(k) = \ell\},$$

where $\sqcup$ indicates disjoint union of sets. □

4.3. **Garbling $f : \mathbb{Z}_n \times \mathbb{Z}_m \to \mathbb{Z}_d$ with one-hot inputs and outputs.** We move on to gates that have two one-hot vectors of wires as inputs W_A, W_B, and produce a one-hot vector of wires as output W_O. This is the most relevant case (conceptually, in terms of presentation and interpretation) of our scheme. The first step will be to define the output nonces, then we will need to define the auxiliary tensor product of the input wires.

Definition 4.18 (The X function: ideal model)**.** In the security analysis, we model $X : \mathcal{G} \to \mathbb{Z}_q^*$ as a random oracle, realized by querying the global functionality $\mathcal{G}_{\mathsf{RO}}^X$ (Algorithm 6 in Section 5.2 below) on inputs $(\mathsf{sid}, \mathsf{tag}, U)$, where $U \in \mathcal{G}$ is a canonically encoded group element, sid is the session identifier, and tag is a domain-separation string specifying the protocol component and role. For $s \in \mathbb{Z}_p$ write $[s]G \in \mathcal{G}$ for scalar multiplication, and set $G_k := [\Delta^k]G$. All security theorems in Section 5 are stated in this $\mathcal{G}_{\mathsf{RO}}^X$-hybrid model.

Definition 4.19 (The X function: concrete instantiation)**.** For implementation, we instantiate X via one of the following constructions:

(1) **Hash-to-field:** $X(U) := H(\mathsf{encode}(U)) \mod q$, where $H : \{0,1\}^* \to \{0,1\}^{2\kappa}$ is a cryptographic hash function (e.g., SHA-256 with output extension) and $\mathsf{encode} : \mathcal{G} \to \{0,1\}^*$ is a canonical injective encoding of group elements.
(2) **Coordinate extraction:** $X(U) := x(U) \mod q$, where $x : \mathcal{G} \setminus \{\mathcal{O}\} \to \mathbb{Z}_p$ extracts the x-coordinate of the affine representation of $U \in E(\mathbb{Z}_p)$.

Remark 4.20. For elliptic curve operations, such as $X([r\Delta^x]G)$, we must ensure that $[\Delta^x]G \neq G$, as this could lead to trivial encodings or repeated generators, potentially compromising indistinguishability. For $[\Delta^x]G = G$, it must be $\Delta^x \equiv 1 \pmod q$. In the multiplicative group $\mathbb{Z}_q^*$, by Lagrange's theorem, this is equivalent to $x \equiv 0 \pmod q$, which has probability $\approx 2^{-256}$, as the bias in mapping $a \in \mathbb{Z}_p^*$ to $\{0, \ldots, q-1\}$ mod q is $O((p-q)/p) \approx 2^{-224}$ (Remark 4.4). Thus the probability of $\Delta^x \equiv 1 \mod q$ is cryptographically negligible, ensuring that word wire encodings remain secure and distinct.

Definition 4.21 (Output nonces from input nonces)**.** Let G be a gate with W_A, W_B as vectors of input wires, with respective nonces $R_{A,i}, R_{B,j}$ $(i \in \mathbb{Z}_n, j \in \mathbb{Z}_m)$. Let nonces $R_{O,k}$ for the output vector of wires W_O, which are defined (computed by garbler) as follows: $R_{O,k} = \prod_{i,j \,:\, f(i,j)=k} X([R_{A,i}R_{B,j}]G)$.

Definition 4.22 ($f : \mathbb{Z}_n \times \mathbb{Z}_m \to \mathbb{Z}_d$ cyphertext help H, circuit material)**.** Let the garbler compute

$$H = \Delta \prod_{i \in \mathbb{Z}_n, j \in \mathbb{Z}_m} X([R_{A,i}R_{B,j}]G) \tag{4}$$

and send it to the evaluator. The number H is called a *help* for the garbled function computation f. It only depends on the nonces of the input wires.

Lemma 4.23 (Repair preserves uniformity)**.** *Let $\{Z_{ij}\}_{i,j}$ be independent uniform in $\mathbb{Z}_q^*$ and fix a hot index (a,b). Define $H := \Delta \cdot \prod_{i,j} Z_{ij}$ and $\widetilde{Z}_{ab} := H \cdot \big(\prod_{(i,j)\neq(a,b)} Z_{ij}\big)^{-1}$. Then $\widetilde{Z}_{ab} = \Delta \cdot Z_{ab}$, and, conditioned on $\{Z_{ij}\}_{(i,j)\neq(a,b)}$, the distribution of $\widetilde{Z}_{ab}$ is uniform in $\mathbb{Z}_q^*$.*

Proof. $H = \Delta\, Z_{ab} \cdot \prod_{(i,j)\neq(a,b)} Z_{ij}$, hence $\widetilde{Z}_{ab} = \Delta \cdot Z_{ab}$; multiplication by $\Delta \in \mathbb{Z}_q^*$ is a permutation of $\mathbb{Z}_q^*$. □

4.3.1. *One-hot tensor product $T_{A\otimes B}$ of one-hot vectors of wires W_A, W_B.* To obtain this auxiliary collection of Boolean wires, we need only to define its nonces. The index set of T is exactly the cartesian product $A \times B$, hence we can use $(i,j) \in \mathbb{Z}_n \times \mathbb{Z}_n$ to indicate these entries

Definition 4.24. Define the nonces of $T_{A\otimes B}$ as $R_{A\otimes B,(i,j)} = X([R_{A,i}R_{B,j}]G)$. These are computed by the garbler since he knows the nonces of the input wires. Then

$$T_{A\otimes B}(x,y)_{ij} = R_{A\otimes B,(i,j)}\Delta^{\delta_i(x)\delta_j(y)} \qquad i \in \mathbb{Z}_n,\ j \in \mathbb{Z}_m.$$

Definition 4.25 ($f : \mathbb{Z}_n \times \mathbb{Z}_m \to \mathbb{Z}_d$. Computing $T_{A\otimes B}(a,b)$)**.** For inputs $a \in \mathbb{Z}_n$ and $b \in \mathbb{Z}_n$, let $W_A(a), W_B(b)$ be their respective garblings on the inputs wires. For $(i,j) \neq (a,b)$ the evaluator computes first

$$w_{ij} = \begin{cases} X([(W_A(a) \otimes W_B(b))_{ij}]G) = X([W_A(a)_i\, W_B(b)_j]G) & i \neq a \text{ and } j \neq b \\ X([(W_A(a) \otimes W_B(b))_{ij}]G_{-1}) = X([W_A(a)_i\, W_B(b)_j]G_{-1}) & i \neq a \text{ or } j \neq b \end{cases}$$

Setting $w_{ab} = 1$, let $h = \prod_{i\in\mathbb{Z}_n, j\in\mathbb{Z}_m} w_{ij}$.

Remark 4.26. Note that w_{ij} and h can be computed by the evaluator, since she holds the garbled wires $W_A(a), W_B(b)$ and she also knows the cleartexts a, b. On the other hand, the help H can only be computed by the garbler, who garbled the wires, but this help was previously sent to the evaluator.

Theorem 4.27 (Tensor product of wires. Evaluation and Consistency)**.** *We have*

$$T_{A\otimes B}(a,b)_{ij} = X([R_{A,i}R_{B,j}]G)\Delta^{\delta_i(a)\delta_j(b)} = \begin{cases} w_{ij} & i \neq a \text{ or } j \neq b \\ Hh^{-1} & i = a \text{ and } j = b \end{cases}.$$

Proof. Note that $w_{ij} = X([R_{A,i}R_{B,j}]G) = R_{A\otimes B,(i,j)} = T_{A\otimes B}(a,b)_{ij}$ for $(i,j) \neq (a,b)$, since $G_{-1} = [\Delta^{-1}]G$ in the second line of w_{ij} cancels out the hot entries with Δ. On the other hand $Hh^{-1} = \Delta\, X([R_{A,a}R_{B,b}]G) = R_{A\otimes B,(a,b)}\Delta = T_{A\otimes B}(a,b)_{ab}$. □

The following definition again is based on Lemma 3.14, replacing the sum with the product and with the addition of the elliptic curve which acts as a hashing function:

Definition 4.28 ($f : \mathbb{Z}_n \times \mathbb{Z}_m \to \mathbb{Z}_d$. Computing the garbled output $\breve{f}$)**.** The one-hot garbled output of gate G computing f in terms of garbled one-hot inputs is defined as

$$\breve{f}(W_A(a), W_B(b))_k := \prod_{i\in\mathbb{Z}_n, j\in\mathbb{Z}_m\,:\, f(i,j)=k} T_{A\otimes B}(a,b)_{ij}.$$

Again, the product over an empty set equals $1 \in \mathbb{Z}_p$.

Remark 4.29. Note that at this point the evaluator, knowing a, b, $T_{A\otimes B}(a,b)$ and f, knows the cleartext $f(a,b)$ hence she knows the garbling of this output and the exact *position* $k = f(a,b)$ in this garbled wire where there is the hot entry Δ, so she can use this as the input of another garbled gate. Also note that with the auxiliary tensor $T_{A\otimes B}(a,b)$ she can obtain the output of any gate with the same garbled inputs $W_A(a), W_B(b)$.

Theorem 4.30 ($f : \mathbb{Z}_n \times \mathbb{Z}_m \to \mathbb{Z}_d$. Consistency)**.** *The computation of $\breve{f}$ returns the one-hot output vector of wires evaluated at $f(a,b)$, that is:*

$$W_O(f(a,b)) = \breve{f}(W_A(a), W_B(b)).$$

Proof. The proof if similar to that of Lemma 4.15: recalling Definition 4.21, if $k = f(a,b)$ then due to Theorem 4.27, both sides are equal to

$$\prod_{i\in\mathbb{Z}_n, j\in\mathbb{Z}_m \,:\, f(i,j)=f(a,b)} X([R_{A,i}R_{B,j}]G)\Delta = R_{O,f(a,b)}\Delta = W_O(f(a,b))_{f(a,b)},$$

while for $k \neq f(a,b)$ both sides are equal to

$$\prod_{i\in\mathbb{Z}_n, j\in\mathbb{Z}_m \,:\, f(i,j)=k} X([R_{A,i}R_{B,j}]G) = R_{O,k} = W_O(f(a,b))_k. \qquad \square$$

Lemma 4.31. *If $f : \mathbb{Z}_n \times \mathbb{Z}_m \to \mathbb{Z}_d$ and $g : \mathbb{Z}_d \to \mathbb{Z}_r$, then $\widetilde{g \circ f} = \breve{g} \circ \breve{f}$.*

Proof. Let W_A, W_B be the input vectors of wires for the first gate computing f. On one hand for each $\ell \in \mathbb{Z}_r$ we have

$$(\widetilde{g \circ f})(W_A(a), W_B(b))_\ell = \prod_{\substack{i,j \text{ s.t.} \\ g(f(i,j))=\ell}} T_{A\otimes B}(a,b)_{ij}$$

and on the other hand

$$\breve{g}(\breve{f}(W_A(a), W_B(b))_\ell = \prod_{k\,:\,g(k)=\ell} \breve{f}(W_A(a), W_A(b))_k = \prod_{k\,:\,g(k)=\ell} \prod_{\substack{i,j \text{ s.t.} \\ f(i,j)=k}} T_{A\otimes B}(a,b)_{ij}.$$

Hence the proof again follows from a set equality:

$$\{i,j \in \mathbb{Z}_n \times \mathbb{Z}_n : g(f(i,j)) = \ell\} = \bigsqcup_{k\in\mathbb{Z}_d} \{i,j \in \mathbb{Z}_n \times \mathbb{Z}_m : f(i,j) = k \ \textbf{AND} \ g(k) = \ell\},$$

where $\sqcup$ indicates disjoint union of sets. $\square$

Remark 4.32. We point out that the composition properties *fail* when there is garbled tensor product *in the middle* of a garbling, that is, if the first gate computes $g : \mathbb{Z}_d \to \mathbb{Z}_{n_1} \times Z_{n_2}$ and $f : \mathbb{Z}_{n_1} \times \mathbb{Z}_{n_2} \to \mathbb{Z}_d$. To see this, note that $g = (g_1, g_2)$ with $g_i : \mathbb{Z}_d \to \mathbb{Z}_{n_i}$. Then the composition $f \circ g : \mathbb{Z}_d \to \mathbb{Z}_k$ is garbled for free (Section 4.2), with $(\widetilde{f \circ g})(W_I(a))$ -here W_I is the vector of input wires of the first gate- involving no tensor products neither elliptic curve operations, while $(\breve{f} \circ \breve{g})$ involves the garbled tensor product of $\breve{g_1}(W_I(a))$ and $\breve{g_2}(W_I(a))$.

Remark 4.33 ($f : \mathbb{Z}_{n_0} \times \cdots \times \mathbb{Z}_{n_{I-1}} \to \mathbb{Z}_m$)**.** This general case can be obtained similarly as we did in Section 4.3. Let $W_{A_0}, \ldots, W_{A_{I-1}}$ be the vectors of input wires of the gate computing f, with $A_j \subset \mathcal{W}$, and let $R_{A_j,i}$ for $j \in \mathbb{Z}_I$, $i \in \mathbb{Z}_{n_j}$ be the respective nonces. Let W_O be the output wires of the gate.

Definition 4.34. The nonces $R_{O,k}$ for W_O ($k \in \mathbb{Z}_m$) are defined (computed by the garbler) as $R_{O,k} = \prod_{\substack{i_j\in\mathbb{Z}_{n_j} \text{ s.t.} \\ f(i_0,\ldots,i_{I-1})=k}} X(\prod_{j\in\mathbb{Z}_I} [R_{A_j,i_j}]G)$.

It is assumed that on the setup, the garbler sent to the evaluator all the generators $G_{-j} = [\Delta^{-j}]G$ for $j \in \mathbb{Z}_I$.
Now the computation of this gate proceeds as in the case of the two-variable function: provided the evaluator holds the actual values of the garbled input wires, that is $W_{A_j}(a_j)$, for $j \in \mathbb{Z}_I$ and $a_j \in \mathbb{Z}_{n_j}$, she now computes the tensor product

$$\bigotimes_{j\in\mathbb{Z}_I} W_{A_j}(a_j) = W_{A_0}(a_0) \otimes W_{A_1}(a_1) \otimes \cdots \otimes W_{A_{I-1}}(a_{I-1}).$$

With the help $H = \Delta \prod_{i_j\in\mathbb{Z}_{n_j}} X([R_{A_0,i_0}R_{A_1,i_1}\dots R_{A_{I-1},i_{i-1}}]G)$, produced by the garbler and handled to the evaluator, she can produce the one-hot garbled tensor product $T_{\otimes jA_j}(a_0,\dots,a_{I-1})$, and with that the computation of garbled f is defined as

$$\breve{f}(W_{A_0}(a_0),\dots,W_{A_{I-1}}(a_{I-1}))_k := \prod_{\substack{i_j\in\mathbb{Z}_{n_j}\ s.t.\\ f(i_0,\dots,i_{I-1})=k}} T_{\otimes jA_j}(a_0,a_1,\dots,a_{I-1})_{i_0 i_1\dots i_{I-1}} \quad k\in\mathbb{Z}_m.$$

We will not give the full details of the general case, but the reader should notice that again this is a well formed garbling: the output of this gate is exactly its output wire $W_O(z)$, evaluated at $z = f(a_0,\dots,a_{I-1})$, that is

$$\breve{f}(W_{A_0}(a_0),\dots,W_{A_{I-1}}(a_{I-1}))_k = R_{O,k}\cdot\Delta^{\delta_k(f(a_0,a_1,\dots,a_{I-1}))} = W_O(f(a_0,\dots,a_{I-1}))_k.$$

We will need a special case of this (with some modifications), when $n_i = 2$ for all i, regarding binary to word functions, see Section 4.3.3 below, which we will develop there in full detail.

4.3.2. *Tuples of functions.* When the output of f (defined in $\prod_j \mathbb{Z}_{n_j}$) is an ordered k-uple of words, its garbling can be dealt by just garbling each component $f_k : \prod_j \mathbb{Z}_{n_j} \to \mathbb{Z}_{m_k}$ with the process just described (as we did in Remark 4.32).
In this case the gate has input wires W_{A_j} ($j \in \mathbb{Z}_I$) and several output wires W_{O_k}. The following is the consistent definition of the output wire nonces for this gate, and it is just a generalization of the cases already discussed.

Definition 4.35. Let $R_{A_j,i}$ be the nonces for input wires W_{A_j} of gate G, then for $\ell \in \mathbb{Z}_{m_k}$, the nonces for the k-th output vector wire W_{O_k} are as in Definition 4.34

$$R_{O_k,\ell} := \prod_{\substack{i_j\in\mathbb{Z}_{n_j}\ s.t.\\ f_k(i_0,\dots,i_{I-1})=\ell}} X(\prod_{j\in\mathbb{Z}_I}[R_{A_j,i_j}]G).$$

With this definition, the computation of the garbled outputs of f can be obtained consistently, and

$$\breve{f}_k(W_{A_0}(a_0),\dots W_{A_{I-1}}(a_{I-1}))_\ell = R_{O_k,\ell}\cdot\Delta^{\delta_\ell(f_k(a_0,a_1,\dots,a_{I-1}))} = W_{O_k}(f(a_0,\dots,a_{I-1}))_\ell.$$

4.3.3. *Garbling binary-to-word functions and some applications.* We want to deal now with a function $f : \mathbb{Z}_2^d \to \mathbb{Z}_n$. In this case we are then dealing with a d-variable function, each variable $z_i \in \{0,1\} = \mathbb{Z}_2$. At this point the evaluator holds $G_c = [\Delta^c]G$ for $c \in \mathbb{Z}_d$ as global material of the garbled circuit. In this section we will need to use the original notation w_{i_k}, R_{i_k} for $i_k \in I$.

Remark 4.36 (Boolean vs. one-hot)**.** Notice that for a bit z, we have $\delta_0(z) = 1-z = \overline{z}$ and $\delta_1(z) = z$. Let $I = \{i_0, i_1\}$ and let $W(z) = (R_{i_0}\Delta^{\overline{z}}, R_{i_1}\Delta^z)$ be a one-hot vector of wires carrying only one bit (that is $z \in \mathbb{Z}_2 \simeq \{0,1\}$). By dropping the first entry we obtain a Boolean wire carrying the same information.

Definition 4.37 (From Boolean to one-hot)**.** For a Boolean wire $w(z) = R\Delta^z$, we denote $r_1 = R\,, r_0 = (R\Delta)^{-1}$ and we build the *auxiliary one-hot vector* of Boolean wires

$$\mathcal{H}(z) = (r_0\Delta^{\delta_0(z)}, r_1\Delta^{\delta_1(z)}) = ((R\Delta)^{-1}\,\Delta^{\overline{z}}, R\,\Delta^z) = (w(z)^{-1}, w(z)) \qquad z \in \{0,1\}.$$

This is the analogue, in our garbling scheme, of the one-hot encoding of a bit $\mathcal{H} : \{0,1\} \mapsto \{0,1\}^2$ described in Definition 3.1. Using the same notation for both cleartexts and garblings will not be a problem (hopefully).

(1) $\mathcal{H}$ is hot in the first coordinate if and only if $z = 0$, and hot in the second if and only if $z = 1$: it is a one-hot vector of two Boolean wires, by Definition 3.2.(2).
(2) If we drop the first entry of $\mathcal{H}$, we recover the original Boolean wire.
(3) If we invert $\pmod p$ one of the wires of $\mathcal{H}$, we obtain the other wire.

Remark 4.38 (Opposite as not)**.** In the process, we have obtained the following behaviour for not in our garbling scheme, when applied to a Boolean garbling:

$$\mathsf{not} : R\Delta^z \mapsto (R\Delta^z)^{-1}.$$

We leave to the reader to check that if we create $\mathcal{H}$ as in Definition 4.37 and drop the first entry of the one-hot output, we recover not as defined above.

Since notation is a bit heavy on this section, let us take a look at how things are for a bunch of Boolean wires:

Definition 4.39 ($f : \mathbb{Z}_2^d \to \mathbb{Z}_n$. Auxiliary bit-to-hot vector wires)**.** For a set of indices $I = \{i_0, \ldots, i_{d-1}\} \subset \mathcal{W}$, with $Z = z_0 \ldots z_{d-1} \in \{0,1\}^d$ and a binary vector of wires

$$B_I(Z) = (w_{i_0}(Z), \ldots, w_{i_{d-1}}(Z)) = (R_{i_0}\Delta^{z_0}, \ldots, R_{i_{d-1}}\Delta^{z_{d-1}})$$

we now have (for each $j \in \mathbb{Z}_d$) the auxiliary one-hot vectors of wires

$$\mathcal{H}_{i_j}(z) = (r_{i_j,0}\Delta^{\overline{z}}, r_{i_j,1}\Delta^z) = (w_{i_j}(z)^{-1}, w_{i_j}(z)) \qquad z \in \{0,1\},$$

with

$$r_{i_j,0} = (R_{i_j}\Delta)^{-1} \qquad r_{i_j,1} = R_{i_j} \qquad j \in \mathbb{Z}_d. \tag{5}$$

Let B_I be the input wires for the gate computing $f : \mathbb{Z}_2^d \to \mathbb{Z}_n$, with a binary entry ($d$-bits) and an arithmetic output. We will think of this gate as having d one-hot input wires $\mathcal{H}_{i_j}$ (one for each bit) and one-hot output wires W_O. Therefore we can apply our already defined algorithm to compute this gate (Section **??**). Lets get into details:

Definition 4.40 ($f : \mathbb{Z}_2^d \to \mathbb{Z}_n$. Output nonces and help)**.** The n nonces for the output wires W_O are obtained as before as $R_{O,\ell} := \prod_{\substack{z_j \in \{0,1\} \text{ s.t.} \\ f(z_0,\ldots,z_{d-1})=\ell}} X(\prod_{j\in\mathbb{Z}_d} [r_{i_j,z_j}]G)$.

For each $k \in \mathbb{Z}_{2^d}$ denote $\mathsf{bin}(k) = k[0] \dots k[d-1]$ as before, with $k[0]$ its LSB, then the help for this gate is

$$H = \Delta \prod_{k \in \mathbb{Z}_{2^d}} X(\prod_{j \in \mathbb{Z}_d} [r_{i_j, k[j]}]G) \tag{6}$$

computed by the garbler and passed on to the evaluator as circuit material.

Definition 4.41 ($f : \mathbb{Z}_2^d \to \mathbb{Z}_n$. One-hot tensor product $T_{\otimes \mathcal{H}}$)**.** We write

$$T_{\otimes \mathcal{H}} = T_{\otimes_{j \in \mathbb{Z}_d} \mathcal{H}_{i_j}}$$

for the tensor product of the auxiliary input wires, to keep notation at bay. To obtain this auxiliary collection of Boolean wires, we need only to define its nonces. The index set of T is exactly $\{0,1\}^d$, hence we can use $\mathsf{bin}(k)$ for $k \in \mathbb{Z}_{2^d}$ to indicate these entries, and define the nonces of $T_{\otimes \mathcal{H}}$ as $R_{\otimes \mathcal{H}, \mathsf{bin}(k)} = X(\prod_{j \in \mathbb{Z}_d} [r_{i_j, k[j]}]G)$ for $k \in \mathbb{Z}_{2^d}$. These are computed by the garbler since he knows the nonces of the input wires.

Definition 4.42 (Hot counting function)**.** For fixed $a_0 a_1 \dots a_{d-1} \in \{0,1\}^d$, and by means of the flip operator T_* introduced in Definition 3.17, we consider the counting function

$$c(k) = \sum_{i=0}^{d-1} (T_{\mathsf{bin}(k)} a_0 a_1 \dots a_{d-1})[i] \qquad k \in \mathbb{Z}_{2^d}. \tag{7}$$

This function returns the number of hot entries (equivalently the number of times Δ appears) in the tensor product of the auxiliary one-hot garblings

$$(\bigotimes_{j=0}^{d-1} \mathcal{H}_{i_j}(a_j))_{\mathsf{bin}(k)} = \prod_{j=0}^{d-1} \mathcal{H}_{i_j}(a_j)_{k[j]} = \prod_{i=0}^{d-1} w_{i_j}(a_j)^{2k[j]-1}.$$

Definition 4.43 ($f : \mathbb{Z}_2^d \to \mathbb{Z}_n$. Computing $T_{\otimes \mathcal{H}}$)**.** Let $w_{i_j} = w_{i_j}(a_j)$ be the binary garbling of cleartext $a_0 \dots a_{d-1} \in \{0,1\}^d$, in the input wires of the gate computing $f : \mathbb{Z}_2^d \to \mathbb{Z}_n$. We first produce their garbled tensor product: for each multi-index $\mathsf{bin}(k) \in \{0,1\}^d$, $k \in \mathbb{Z}_{2^d}$, we let

$$w_k = \begin{cases} X(\prod_{j \in \mathbb{Z}_d} [w_{i_j}^{2k[j]-1}] G_{c(k)}) & \mathsf{bin}(k) \neq a_0 \dots a_{d-1} \\ 1 & \mathsf{bin}(k) = a_0 \dots a_{d-1} \end{cases}$$

Note that the evaluator, in possession of the G_c and the w_{i_j}, can compute all of the w_k and she can also compute $h = \prod_{k \in \mathbb{Z}_{2^d}} w_k$. With a similar proof that of Theorem 4.27, it follows that

$$T_{\otimes \mathcal{H}}(a_0 \dots a_{d-1})_{\mathsf{bin}(k)} = \begin{cases} w_k & \mathsf{bin}(k) \neq a_0 \dots a_{d-1} \\ H h^{-1} & \mathsf{bin}(k) = a_0 \dots a_{d-1} \end{cases}$$

A key observation for this proof is that

$$\begin{array}{ll} w_k = X(\prod_{j=0}^{d-1} [r_{i_j, k[j]}]G) & \mathsf{bin}(k) \neq a_0 \dots a_{d-1} \\ H h^{-1} = \Delta \cdot X(\prod_{j=0}^{d-1} [r_{i_j, a_j}]G) & \mathsf{bin}(k) = a_0 \dots a_{d-1} \end{array}.$$

Definition 4.44 ($f : \mathbb{Z}_2^d \to \mathbb{Z}_n$. Computing $\breve{f}$)**.** Let

$$B = B_I(a_0, \dots, a_{d-1}) = (w_{i_0}, \dots, w_{i_{d-1}})$$

be the binary garbling of the input wires B_I. For each $\ell \in \mathbb{Z}_n$ we let

$$\breve{f}(B)_\ell = \prod_{k \in \mathbb{Z}^{2^d} \,:\, f(\mathsf{bin}(k)) = \ell} T_{\otimes\mathcal{H}}(a_0, \dots, a_{d-1})_{\mathsf{bin}(k)}$$

where as before, the product over the empty set is defined as $1 \in \mathbb{Z}_p$.

We leave to the reader to verify the consistency, that is if $Z = z_0 \cdots z_{d-1}$ then

$$\breve{f}(B_I(Z))_\ell = R_{O,\ell}\, \Delta^{\delta_\ell(f(Z))} = W_O(f(Z))_\ell.$$

As an application, we now use this to translate a binary garbled wire to a one-hot arithmetic garbled wire.

4.4. **Algorithms for translation and more applications.** We now describe how to consistently transform garblings of some type (word, binary, one-hot) into the other two types of garblings. We begin with a direct application of the results of the previous section.

4.4.1. *Binary to one-hot:* bin2hot. Inputs a binary garbling and outputs a one-hot garbling of the same cleartext (Algorithm 13 in the next section). This is an overview with some remarks: taking $n = 2^d$, consider $f : \mathbb{Z}_2^d \to \mathbb{Z}_n$ given by

$$z_0 z_1 \dots z_{d-1} \mapsto \sum_{j=0}^{d-1} z_j\, 2^j.$$

The garbling of this function is a particular instance of the general discussion of the previous section. Clearly $f(\mathsf{bin}(k)) = \ell$ if, and only if, $k = \ell$, and if

$$z_0 z_1 \dots z_{d-1} = a[0] \dots a[d-1] = \mathsf{bin}(a),$$

then $f(z_0, \dots, z_{d-1}) = a$. Therefore if $B_I(a[0] \dots a[d-1]) = (w_{i_0}, \dots, w_{i_{d-1}})$ is a garbling of $\mathsf{bin}(a)$, the one-hot garbled output $W(a = f(\mathsf{bin}(a)))$ of bin2hot is

$$\mathsf{bin2hot} : B_I(\mathsf{bin}(a)) \longmapsto W(a)$$

for $W(a)_k = X(\prod_{j=0}^{d-1} [w_{i_j}^{2k[j]-1}] G_{c(k)})$ $(k \neq a)$, $W(a)_a = H \prod_{k \neq a} W(a)_k^{-1}$, with H derived from (6) and (5) as $H = \Delta \prod_{k \in \mathbb{Z}_{2^d}} X(\prod_{j \in \mathbb{Z}_d} [R_{i_j}^{2k[j]-1} \Delta^{k[j]-1}] G)$.

4.4.2. *One-hot to binary:* hot2bin. The inverse of the previous primitive bin2hot can be obtained combining previous ideas, and it is then computed "for free" (no material transmitted). As usual $d = \lceil \log_2 n \rceil$. We have to think of the operation that sends a one-hot cleartext to a binary cleartext as a collection of mappings $f_j : \mathbb{Z}_n \to \mathbb{Z}_2$ $(j \in \mathbb{Z}_d)$ defined as $f_j[k] = k[j]$, where $k[0] \cdots k[d-1] = \mathsf{bin}(k)$ for $k \in \mathbb{Z}_n$. This gate has then a one-hot vector of input wires W_I (length n) and d Boolean output wires w_{i_j}, $j \in \mathbb{Z}_d$. Hence denoting

$$\mathsf{hot2bin} : W_I(a) \mapsto B_O(\mathsf{bin}(a)) = (w_{i_0}, \dots, w_{i_{d-1}})$$

and looking at Section 4.2 we have

$$w_{i_j} = \prod_{k\in\mathbb{Z}_n\,:\,f_j(k)=1} W_I(a)_k = \prod_{k\in\mathbb{Z}_n\,:\,k[j]=1} W_I(a)_k \qquad j\in\mathbb{Z}_d.$$

In the literature this kind of functionality is also referred as a *BD* (Bit Decomposition) gate.

4.4.3. *Binary to word:* bin2word. Inputs a binary garbling $B_I(a)$, outputs a word garbling $\Omega(a)$ of the same cleartext, for free. If the nonces of the input wires are R_{i_j}, we have

$$B_I(a) = (w_{i_0},\dots,w_{i_{d-1}}) = (R_{i_0}\Delta^{a[0]},\dots,R_{i_{d-1}}\Delta^{a[d-1]}),$$

the nonce of the output wire is obtained as $r = \prod_{j=0}^{d-1} R_{i_j}^{2^j}$, and we compute

$$\prod_{j=0}^{d-1} w_{i_j}^{2^j} = \prod_{j=0}^{d-1} (R_{i_j}\Delta^{a[j]})^{2^j} = \prod_{j=0}^{d-1} R_{i_j}^{2^j}\cdot\Delta^a = r\Delta^a$$

obtaining a word garbling of a, that is

$$\mathsf{bin2word} : (w_{i_0},\dots,w_{i_{d-1}}) \longmapsto \prod_{j=0}^{d-1} w_{i_j}^{2^j} = \Omega(a).$$

4.4.4. *Word to one-hot:* word2hot. Inputs a word garbling, outputs a one-hot garbling of the same cleartext $a\in\mathbb{Z}_n$.
A different generator G of the curve is needed for this algorithm, that is to securely compute it is necessary that the evaluator does not know the generator of the curve. So for each implementation of this algorithm, the garbler randomly chooses $k\in\mathbb{Z}_p^*$, produces a generator $G' = [k]G$, and gives $G'_i = [\Delta^i]G'$ to the evaluator, $-(n-1)\le i\le n-1$, $i\neq 0$. If the (word) input wire has nonce r, and W_I are the output wires with $I=\{i_0,\dots,i_{n-1}\}$, then $R_{i_j} = X([r\Delta^j]G') \qquad j\in\mathbb{Z}_n$ are by definition the nonces for the output wire. Then for any $a\in\mathbb{Z}_n$ and its word garbling $\Omega(a) = r\Delta^a$ in the input wire, the evaluator can compute

$$[\Omega(a)]G'_c = [r\Delta^{a+c}]G'$$

thus obtaining

$$X([\Omega(a)]G'_{j-a}) = X([r\Delta^j]G') = W_I(a)_j \quad j\neq a \qquad \text{and} \qquad h = \prod_{j\in\mathbb{Z}_n\,:\,j\neq a} W_I(a)_j.$$

On the other hand the garbler provides the help

$$H = \Delta\prod_{j=0}^{n-1} X([r\Delta^j]G') \tag{8}$$

to the evaluator and with it she computes $Hh^{-1} = \Delta\, X([r\Delta^a]G') = W_I(a)_a$, obtaining consistently the full one-hot output garbled wire $W_I(a) = \mathsf{word2hot}(\Omega(a))$. Remark 4.20 applies to all these computations.

4.4.5. *One hot to word:* hot2word. Inputs a one-hot garbling, outputs a word garbling of the same cleartext, for free. If the input wires W_I have nonces R_{i_j} ($j \in \mathbb{Z}_n$), the output wire has nonce $r = \prod_j R_{i_j}^j$. Then we define

$$\mathsf{hot2word} : W_I(a) \longmapsto \prod_{j\in\mathbb{Z}_n} (W_I(a)_j)^j = \prod_j R_{i_j}^j \Delta^a := r\Delta^a = \Omega(a)$$

which implements the functionality.

4.4.6. *Word to bin:* word2bin. Inputs a word garbling, outputs a binary garbling. It is implemented by composition: $\mathsf{word2bin}(z) = \mathsf{hot2bin}(\mathsf{word2hot}(z))$. It costs $1 + 0 = 1$ cyphertexts of material, and if r is the nonce of the input word wire, the explicit formula for $\mathsf{word2bin} : \Omega(a) \mapsto B(\mathsf{bin}(a)) = (w_{i_0}, \ldots, w_{i_{d-1}})$ when $a \in \mathbb{Z}_n$ and $n = 2^d$ is given by

$$w_{i_j} = \begin{cases} \prod_{k\in\mathbb{Z}_n\,:\,k[j]=1} X(\Omega(a)G'_{k-a}) & a[j] = 0 \\ \prod_{k\neq a\,:\,k[j]=1} X(\Omega(a)G'_{k-a})Hh^{-1} & a[j] = 1 \end{cases}.$$

where

$$h = \prod_{k\in\mathbb{Z}_n\, k\neq a} X([\Omega(a)]G'_{k-a})$$

and H is the help (8) provided by the garbler. Again, Remark 4.20 applies to this construction.

4.4.7. *More applications: sum and product of integers.* For $f = +$ the sum of two integers modulo n, the truth table of f is a $n \times n$ symmetric matrix, with $(i, k-i) \mapsto k$ for $i \in \mathbb{Z}_n$. Then by the results of Section 4.3 we have

$$\check{+}(W_A(a), W_B(b))_k := \prod_{i\in\mathbb{Z}_n} T_{A\otimes B}(a,b)_{i,k-i} \qquad k \in \mathbb{Z}_n.$$

For $f = \times$ the product of two integers modulo n, with n prime, for $k \neq 0$ we have $(i, ki^{-1}) \mapsto k$ for $i \neq 0$ while $(0, j) \mapsto 0$ and $(i, 0) \mapsto 0$. Hence

$$\check{\times}(W_A(a), W_B(b))_k := \prod_{i\neq 0} T_{A\otimes B}(a,b)_{i,ki^{-1}} \qquad k \neq 0$$

while $\check{\times}(W_A(a), W_B(b))_0 := \prod_{i,j\in\mathbb{Z}_n} T_{A\otimes B}(a,b)_{i,0} T_{A\otimes B}(a,b)_{0,j}$.

Remark 4.45 (Large and small integers)**.** For large integers $n, m \simeq 2^{32}$, these operations are impractical, since the tensor product involves $n \times m$ products that is $\simeq 2^{64}$ elliptic curve multiplications. A possible approach is to take the discrete logarithm, operate with word garblings and go back to one-hot, what follows is a description of this approach.

Let W_A, W_B the the arrays of wires carrying a, b that we want to multiply. Pick the Mersenne prime $m = 2^{31} - 1$, and a generator g of $\mathbb{Z}_m^*$ (say $g = 2$)

(1) Compute the table of values of the discrete logarithm function $l : \mathbb{Z}_m^* \to \mathbb{Z}_m$, $2^{l(x)} = x \pmod m$. Define $l(0) = 2m - 3$ to avoid collisions, since $2^{2m-3} = 2^{-1} = 2^{m-1}$ and all the other discrete logarithms are in $[0, m-2]$.

(2) Apply the basic functionality of Section 4.2 with the discrete logarithm function l to our garbled inputs $W_A(a)$ and $W_B(b)$; the output is $W_O(l(a)), W_O(l_(b))$.
(3) Apply the hot2word functionality (Section 4.4.5) to both of them to obtain word garblings of the logarithms $\Omega_l(l(a)) = R\Delta^{l(a)}, \Omega_l(l(b)) = R\Delta^{l(b)}$.
(4) Multiply to obtain $\Omega(l(ab)) = \Omega_l(l(a))\Omega_l(l(b)) = R^2\Delta^{l(ab)} \pmod p$.
(5) Apply word2hot (Section 4.4.4) to obtain $W_P(l(ab))$ with $P = \{i_0, \dots, i_{m-1}\}$.
(6) If $e : \mathbb{Z}_m \to \mathbb{Z}_m^*$ is the exponentiation function, we again apply the basic functionality of Section 4.2 and we end up with $W_O(ab)$, a valid garbling for the product ab.

Remark 4.46. The functionalities of steps (2), (3), (4), (6) are for free, while the one in step (5) requires one cyphertext. Regarding the number of operations, it is within $O(q)$. Step (2) can be done by brute force given the size of the numbers.

5. Security

This section begins by proving a detailed security proof for the authenticity property of the One Hot Modular Garbling (OHMG) scheme within the Universal Composability (UC) framework. The proof addresses adaptive corruption and leverages the elliptic curve discrete logarithm problem (ECDLP) for cryptographic security. Since the garbling scheme is modular by design, it will suffice to prove the authenticity property for each functionality. This tackles the security in the corrupt verifier/honest prover scenario. To finish the paper, we will dedicate the last section to prove security in the honest verifier/corrupt prover scenario.

Remark 5.1. Throughout this section, the UC security analysis makes use of the global setup introduced in Section 5.2 with $\mathcal{G}_{\mathsf{trust}}$, and $\mathcal{G}_{\mathsf{RO}}^{X}$.

5.1. UC authenticity in the honest prover, corrupt verifier scenario.

Definition 5.2. The UC framework compares a *real-world* execution of a protocol π run by interactive parties (for instance prover P and evaluator V) with an *ideal-world* execution in which a trusted party implements an ideal functionality $\mathcal{F}$. The environment $\mathcal{Z}$ interacts with the parties and with a real-world adversary $\mathcal{A}$ or an ideal-world simulator Sim, and may adaptively corrupt parties during execution, revealing their internal states.

Formalized by Canetti [10], the UC paradigm is the cornerstone of modular security proofs. We use it in the standard way, specialized to our two-party setting.

Theorem 5.3 (UC). *Let $\mathcal{F}$ be an ideal functionality and let π be a protocol with the same party structure, in the $(\mathcal{G}_{\mathsf{trust}}, \mathcal{G}_{\mathsf{RO}}^{X})$-hybrid model. We consider PPT environments $\mathcal{Z}$ and PPT adversaries $\mathcal{A}$, allowing adaptive corruptions, and we allow erasures whenever explicitly assumed. If there exists a PPT simulator* Sim *such that for all PPT $\mathcal{A}, \mathcal{Z}$ the ensembles $\mathsf{REAL}_{\mathcal{A},\pi}(\mathcal{Z})$ and $\mathsf{IDEAL}_{\mathsf{Sim},\mathcal{F}}(\mathcal{Z})$ are computationally indistinguishable, then π securely realizes $\mathcal{F}$ in the UC sense. The concrete security property achieved by π is determined by the behaviour of $\mathcal{F}$. In particular, if $\mathcal{F}$ enforces authenticity in the sense of Definition 2.4, then any protocol that UC-realizes $\mathcal{F}$ inherits this authenticity property.*

Definition 5.4 (Security Assumptions)**.** Recall that $E(\mathbb{Z}_p)$ is an elliptic curve, and $\mathbb{G}$ is the multiplicative group generated by some generator $G \in \mathbb{G}$. We state here our main assumptions.

(1) The elliptic curve discrete logarithm problem (ECDLP) on $E(\mathbb{Z}_p)$ with generator G of order $q \approx 2^\kappa$ is computationally hard. The advantage of any PPT algorithm $\mathcal{A}_{\mathsf{DL}}$ in recovering k from $P = [K]G$ is $\mathsf{Adv}_{\mathsf{DL}}(\kappa) \leq 2^{-\kappa/2}$ (for instance, 2^{-128} for $\kappa = 256$), based on subexponential algorithms like Pollard's Rho adapted to elliptic curves [18, 32, 19].
(2) (Random Oracle Model) The hash function $H : \{0,1\}^* \to \{0,1\}^\kappa$ is modeled as a random oracle. The advantage of distinguishing H from a random function is bounded by $q_H \cdot 2^{-\kappa}$, where q_H is the number of queries (for instance, $q_H \leq 2^{60}$ in practice).
(3) (Random Input Labels) Nonces $R_{I,i} \in \mathbb{Z}_p^*$ are chosen uniformly at random. The advantage of any PPT adversary $\mathcal{A}$ in guessing $R_{I,i}$ without the hash preimage is $\mathsf{Adv}_{\mathsf{guess}}(\kappa) = 1/(p-1) \approx 2^{-\kappa}$.

5.2. **Global model and setup.** We use two global ideal functionalities shared by all sessions: (i) a public bulletin board $\mathcal{G}_{\mathsf{trust}}$ to store commitments for input/output wires and public metadata; (ii) a programmable random oracle for X, denoted $\mathcal{G}_{\mathsf{RO}}^X$. All calls include a session identifier sid. Calls to X also include a domain-separation tag tag describing the component and role. Given a map BB, we introduce the following set: $\mathrm{dom}(\mathsf{BB}) := \{(\mathsf{sid}, \mathsf{key}) \mid \mathsf{BB}[\mathsf{sid}, \mathsf{key}] \text{ is defined}\}$.

Definition 5.5 (Key structure for $\mathcal{G}_{\mathsf{trust}}$)**.** A *valid key* for session sid is a tuple of one of the following forms:

(1) **Input wire commitment:** $\mathsf{key} = (\mathsf{pid}, C, \mathsf{in}, i, \sigma)$ where $\mathsf{pid} \in \mathcal{P}$ is a party identifier, C is a circuit identifier, $i \in \mathbb{N}$ is a wire index, and $\sigma \in \{0,1\}$ is the label bit.
(2) **Output wire commitment:** $\mathsf{key} = (\mathsf{pid}, C, \mathsf{out}, o, \tau)$ where $\mathsf{pid} \in \mathcal{P}$, C is a circuit identifier, $o \in \mathbb{N}$ is an output wire index, and $\tau \in \{0,1\}$ is the label bit.
(3) **Auxiliary commitment:** $\mathsf{key} = (\mathsf{pid}, C, \mathsf{aux}, \mathsf{tag})$ where $\mathsf{pid} \in \mathcal{P}$, C is a circuit identifier, and $\mathsf{tag} \in \{0,1\}^*$ is an application-specific tag.

We write $\mathsf{owner}(\mathsf{key})$ for the first component pid of a valid key, and $\mathsf{Valid}(\mathsf{sid}, \mathsf{key})$ for the predicate that returns true iff key has one of the above forms.

Algorithm 5 Global bulletin board $\mathcal{G}_{\mathsf{trust}}$

1: **State:** A map $\mathsf{BB} : (\mathsf{sid}, \mathsf{key}) \mapsto (\mathsf{val}, \mathsf{time})$, initially empty. A monotonic counter $\mathsf{clock} \leftarrow 0$.
2: **Put:** On input $(\mathsf{Put}, \mathsf{sid}, \mathsf{key}, \mathsf{val})$ from party P:
   (a) **Validity check:** If $\neg\mathsf{Valid}(\mathsf{sid}, \mathsf{key})$, output $(\mathsf{Error}, \mathsf{sid}, \texttt{INVALID\_KEY})$ to P and halt.
   (b) **Ownership check:** If $\mathsf{owner}(\mathsf{key}) \neq \mathsf{id}(P)$, output $(\mathsf{Error}, \mathsf{sid}, \texttt{NOT\_OWNER})$ to P and halt.
   (c) **First-write-wins:** If $(\mathsf{sid}, \mathsf{key}) \in \mathrm{dom}(\mathsf{BB})$, output $(\mathsf{Error}, \mathsf{sid}, \texttt{ALREADY\_SET})$ to P and halt.
   (d) **Commit:** Set $\mathsf{clock} \leftarrow \mathsf{clock} + 1$. Set $\mathsf{BB}[\mathsf{sid}, \mathsf{key}] \leftarrow (\mathsf{val}, \mathsf{clock})$. Broadcast $(\mathsf{Commit}, \mathsf{sid}, \mathsf{key}, \mathsf{clock})$ to all parties. Output $(\mathsf{Ack}, \mathsf{sid}, \mathsf{key}, \mathsf{clock})$ to P.
3: **Get:** On input $(\mathsf{Get}, \mathsf{sid}, \mathsf{key})$ from any party:
   (a) If $(\mathsf{sid}, \mathsf{key}) \in \mathrm{dom}(\mathsf{BB})$, let $(\mathsf{val}, t) \leftarrow \mathsf{BB}[\mathsf{sid}, \mathsf{key}]$ and output $(\mathsf{Value}, \mathsf{sid}, \mathsf{key}, \mathsf{val}, t)$.
   (b) Otherwise, output $(\mathsf{Value}, \mathsf{sid}, \mathsf{key}, \bot, \bot)$.
4: **List:** On input $(\mathsf{List}, \mathsf{sid})$ from any party, output the set $\{(\mathsf{key}, \mathsf{val}, t) : \mathsf{BB}[\mathsf{sid}, \mathsf{key}] = (\mathsf{val}, t)\}$.
5: **Clock:** On input $(\mathsf{Clock}, \mathsf{sid})$ from any party, output $(\mathsf{Time}, \mathsf{sid}, \mathsf{clock})$.

Remark 5.6 (Writer-binding enforcement)**.** Algorithm 5 enforces writer-binding via the ownership check in step 2(b): a Put operation for key key succeeds only if the calling party P satisfies $\mathsf{id}(P) = \mathsf{owner}(\mathsf{key})$. This ensures that:

(1) No party can write commitments under another party's namespace.
(2) The prover P with identity pid has exclusive write access to all keys of the form $(\mathsf{pid}, \cdot, \cdot, \cdot, \cdot)$.
(3) Once a key is written (first-write-wins), no party—including the owner—can modify or overwrite it.

The validity check in step 2(a) additionally ensures that only well-formed keys (per Definition 5.5) are accepted, preventing injection of malformed metadata.

Remark 5.7 (Ideal vs. global functionality)**.** We model $\mathcal{G}_{\mathsf{trust}}$ as a *global setup* in the sense of Canetti et al. [11]: it is shared across all protocol sessions and is directly accessible by the environment $\mathcal{Z}$. This is essential for our application, where commitments posted in one session (e.g., the garbling phase) must be readable in another (e.g., the on-chain verification phase). The global nature of $\mathcal{G}_{\mathsf{trust}}$ is reflected in the session identifier sid being an explicit parameter, allowing the environment to correlate entries across sessions.

Remark 5.8 (Instantiation via a global ledger)**.** The functionality $\mathcal{G}_{\mathsf{trust}}$ should be viewed as an abstraction of an append-only, publicly readable ledger, such as a blockchain. We only rely on its ledger-like properties (global visibility, first-write-wins, and immutability of posted entries) and do not model the underlying consensus protocol. This follows the common approach of modelling blockchains as global ledger functionalities in UC-style analyses [25]. All our UC security theorems are therefore relative to the assumption that the deployment provides such a ledger functionality. The concrete Bitcoin UTXO-based realization of $\mathcal{G}_{\mathsf{trust}}$ is presented in Section 5.5.

Algorithm 6 Global programmable random oracle for X

State: a table $\mathsf{T} : \{0,1\}^* \to \mathbb{Z}_q^*$, initially empty.
Query: On $(\mathsf{Query}, \mathsf{sid}, \mathsf{tag}, U)$ where $U \in \mathbb{G}$ is canonically encoded:
- Let $x \leftarrow \mathsf{enc}(\mathsf{sid}, \mathsf{tag}, U)$.
- If $x \notin \mathrm{dom}(\mathsf{T})$, sample $y \leftarrow \mathbb{Z}_q^*$ and set $\mathsf{T}[x] \leftarrow y$.
- Output $(\mathsf{Ans}, \mathsf{sid}, \mathsf{tag}, U, \mathsf{T}[x])$.

Program: On $(\mathsf{Program}, \mathsf{sid}, \mathsf{tag}, U, y)$ with $x = \mathsf{enc}(\mathsf{sid}, \mathsf{tag}, U)$ and $x \notin \mathrm{dom}(\mathsf{T})$, set $\mathsf{T}[x] \leftarrow y$.

Remark 5.9 (Access to $\mathcal{G}_{\mathsf{RO}}^X$)**.** In the real-world execution, honest parties and the adversary only have access to the Query interface of $\mathcal{G}_{\mathsf{RO}}^X$. The $\mathsf{Program}$ interface is reserved exclusively for the ideal-world simulator in our UC proofs, and is never invoked by real-world code. This matches the standard programmable random-oracle modelling in the GUC-RO literature: programmability is a power of the simulator, not of real-world parties.

Lemma 5.10 (ECDLP Hardness)**.** *The advantage of any PPT algorithm $\mathcal{A}_{\mathsf{DL}}$ in solving ECDLP on $E(\mathbb{Z}_p)$ with order q is $\mathsf{Adv}_{\mathsf{DL}}(\kappa) \leq 2^{-\kappa/2}$, consistent with elliptic curve security implemented for our protocol.*

Proof. The proof follows from Pollard's Rho algorithm adapted to elliptic curves, which requires $O(\sqrt{q})$ time and space. For $\kappa = 256$ and $q \approx 2^{256}$, the work factor is 2^{128}, yielding $\mathsf{Adv}_{\mathsf{DL}}(\kappa) \leq 2^{-128+\epsilon}$ for small ϵ in practice [18, 32]. □

Algorithm 7 Protocol π_f: Garbling and Evaluation of $f : \mathbb{Z}_n \to \mathbb{Z}_m$ in OHMG

Require: Security parameter κ, prime $p \approx 2^\kappa$, circuit C with gate computing $f : \mathbb{Z}_n \to \mathbb{Z}_m$ (agreed by prover and verifier), input wire set $A = \{a_0, \ldots, a_{n-1}\}$, output wire set $O = \{o_0, \ldots, o_{m-1}\}$, cleartext input $x \in \mathbb{Z}_n$, global correlation $\Delta \in \mathbb{Z}_p^*$ (fixed, secret, known only to prover), input nonces $R_{A,i} \in \mathbb{Z}_p^*$ (uniformly random).
Ensure: Garbled input $X = W_A(x)$, garbled output $W_O(f(x))$.
Prover (Garbler): Algorithm $\mathsf{Gb}(1^\kappa, C)$
Use fixed global correlation $\Delta \in \mathbb{Z}_p^*$ (secret, previously chosen for circuit C).
Assume input wire nonces $R_{A,i} \in \mathbb{Z}_p^*$ are uniformly random for each $a_i \in A$.
Prover (Garbler): Algorithm $\mathsf{En}(f_e, x)$
Input cleartext $x \in \mathbb{Z}_n$.
Compute garbled input $X = W_A(x) = (W_A(x)_i)_{i \in \mathbb{Z}_n}$, where $W_A(x)_i = R_{A,i}\Delta^{\delta_i(x)}$ and $\delta_i(x) = 1$ if $i = x$, else 0.
Send $X = W_A(x)$ and cleartext x to verifier.
Verifier: Algorithm $\mathsf{Ev}(C, X)$
Use known function $f : \mathbb{Z}_n \to \mathbb{Z}_m$ (from circuit C).
Receive $X = W_A(x)$ and cleartext $x \in \mathbb{Z}_n$.
Compute garbled output $W_O(f(x)) = \breve{f}(W_A(x))$, where $\breve{f}(W_A(x))_k = \prod_{i \in \mathbb{Z}_n : f(i) = k} W_A(x)_i$ for $k \in \mathbb{Z}_m$.
Output $W_O(f(x)) = (\breve{f}(W_A(x))_k)_{k \in \mathbb{Z}_m}$.

Algorithm 8 Ideal Functionality $\mathcal{F}_f$: Garbling and Evaluation of $f : \mathbb{Z}_n \to \mathbb{Z}_m$

Require: Security parameter κ, prime $p \approx 2^\kappa$, circuit C with gate computing $f : \mathbb{Z}_n \to \mathbb{Z}_m$, input wire set $A = \{a_0, \ldots, a_{n-1}\}$ and output wire set $O = \{o_0, \ldots, o_{m-1}\}$, adversary $\mathcal{A}$, environment $\mathcal{Z}$.

Ensure: Garbled input $W_A(x)$, garbled output $W_O(f(x))$.

Initialization

1: Choose global correlation $\Delta \in \mathbb{Z}_p^*$ (secret, fixed for circuit C) and nonces $R_{A,i} \in \mathbb{Z}_p^*$ for each $a_i \in A$, known only to trusted party.

On input $x \in \mathbb{Z}_n$ from prover P:

2: Compute garbled input $W_A(x) = (W_A(x)_i)_{i \in \mathbb{Z}_n}$, where $W_A(x)_i = R_{A,i}\Delta^{\delta_i(x)}$ and $\delta_i(x) = 1$ if $i = x$, else 0.

3: Send $W_A(x)$ and cleartext x to verifier V.

On request from verifier V to evaluate f:

4: Compute garbled output $W_O(f(x)) = (\breve{f}(W_A(x))_k)_{k \in \mathbb{Z}_m}$, where $\breve{f}(W_A(x))_k = \prod_{i \in \mathbb{Z}_n : f(i) = k} W_A(x)_i$.

5: Send $W_O(f(x))$ to V.

On adaptive corruption of prover P by $\mathcal{A}$:

6: **if** prover P is corrupted **then** Reveal Δ and $\{R_{A,i}\}_{i \in \mathbb{Z}_n}$ to $\mathcal{A}$.

7: **end if**

Lemma 5.11 (Ideal authenticity of $\mathcal{F}_f$)**.** *Let $\mathcal{F}_f$ be the ideal functionality of Algorithm 8. In any ideal execution with honest prover P and potentially malicious verifier $V^\star$, the scheme enjoys perfect authenticity in the sense of Definition 2.4: the adversary's success probability in the authenticity experiment is* 0.

Proof. In the ideal world, the only interaction between $V^\star$ and $\mathcal{F}_f$ is: $V^\star$ receives $(W_A(x), x)$ in line 3, and, upon a single evaluation request, receives the uniquely defined output vector $W_O(f(x))$ in line 5. In particular, $\mathcal{F}_f$ never provides $V^\star$ with garbled labels corresponding to any input $x' \neq x$ or to any output value $y' \neq f(x)$. Thus any string Y' output by $V^\star$ that decodes to a non-trivial value $y' \neq f(x)$ is not derived from $\mathcal{F}_f$ at all. By Definition 2.4, the authenticity experiment run in the ideal world outputs 1 with probability 0. □

Definition 5.12 (Unblinded hot input)**.** In any vector produced by the protocol, let $t^\star$ denote the unique hot index (that is the only position multiplied by Δ *outside* the call to X). Write $U^\star \in \mathbb{G}$ for the group element fed to X at $t^\star$. For the tensor product (Theorem 4.27), we have $U^\star = [R_{A,a}R_{B,b}]G$ at hot index (a, b). For `word2hot`, we have $U^\star = [r\Delta^a]G'$ at hot coordinate a.

Lemma 5.13 (Unpredictability of unblinded hot inputs)**.** *In an honest-prover, corrupt-verifier execution, for each object produced by the prover the corresponding unblinded hot input $U^\star \in \langle G \rangle$ is distributed uniformly in $\langle G \rangle$ from the verifier's point of view, except with negligible probability. In particular, for at most q_X oracle queries,*

$$\Pr[\textit{the verifier queries } \mathcal{G}_{\mathsf{RO}}^X \textit{ on } U^\star] \leq q_X/q.$$

Proof. We analyze each object type from Definition 5.12 separately, showing that U^* has high min-entropy from the verifier's view.

Case 1: One-hot tensor product $T_{A\otimes B}(a, b)$.

By Definition 5.12, $U^* = [R_{A,a}R_{B,b}]G$. The verifier's view $\mathcal{V}$ consists of:

- Garbled inputs $W_A(a), W_B(b)$ where $W_A(a)_i = R_{A,i}\Delta^{\delta_i(a)}$ and $W_B(b)_j = R_{B,j}\Delta^{\delta_j(b)}$;
- Help $H = \Delta \prod_{i,j} X([R_{A,i}R_{B,j}]G)$;
- Public generators $G, G^{-1} = [\Delta^{-1}]G$;
- Random oracle responses for queries made during protocol execution.

Let $\chi := W_A(a)_a = R_{A,a}\Delta$ and $\upsilon := W_B(b)_b = R_{B,b}\Delta$ denote the hot wire values observed by the verifier.

Step 1: Conditional distribution of $R_{A,a}R_{B,b}$ given (χ, υ).

The honest prover samples $(R_{A,a}, R_{B,b}, \Delta)$ independently and uniformly from $(Z_q^*)^3$. Consider the bijection

$$\phi : (R_{A,a}, R_{B,b}, \Delta) \mapsto (\chi, \upsilon, \Delta) = (R_{A,a}\Delta, R_{B,b}\Delta, \Delta).$$

Since ϕ is a bijection and $(R_{A,a}, R_{B,b}, \Delta)$ is uniform over $(Z_q^*)^3$, so is (χ, υ, Δ). In particular, Δ remains uniform over Z_q^* and independent of (χ, υ).

Conditioned on (χ, υ), we have

$$R_{A,a}R_{B,b} = \frac{\chi}{\Delta} \cdot \frac{\upsilon}{\Delta} = \frac{\chi\upsilon}{\Delta^2}.$$

The map $\Delta \mapsto \chi\upsilon/\Delta^2$ is 2-to-1 (since $(-\Delta)^2 = \Delta^2$). Its image is the coset

$$\chi\upsilon \cdot (Z_q^*)^{-2} = \begin{cases} \mathrm{QR}_q & \text{if } \chi\upsilon \in \mathrm{QR}_q, \\ \mathrm{NQR}_q & \text{if } \chi\upsilon \in \mathrm{NQR}_q, \end{cases}$$

where QR_q and NQR_q denote the quadratic residues and non-residues in Z_q^*, respectively. In either case, $|\text{image}| = (q-1)/2$, and $R_{A,a}R_{B,b}$ is uniformly distributed over this image.

Step 2: Independence from the help H.

By Definition 4.25, the verifier queries $\mathcal{G}_{\mathsf{RO}}^X$ on $[R_{A,i}R_{B,j}]G$ for all $(i,j) \neq (a,b)$ and computes

$$h := \prod_{(i,j)\neq(a,b)} X([R_{A,i}R_{B,j}]G).$$

From equation (4), we have $H \cdot h^{-1} = \Delta \cdot X(U^*)$. In the random oracle model, until U^* is queried, the value $X(U^*)$ is uniformly distributed over Z_q^* and independent of all other oracle responses. Hence $\Delta \cdot X(U^*)$ is uniform over Z_q^* and reveals no information about Δ or $X(U^*)$ individually. By Lemma 4.23, this "repair" operation preserves the uniform distribution of the reconstructed hot entry.

Step 3: Computational hardness of determining U^*.

To compute $U^* = [R_{A,a}R_{B,b}]G$ directly, the verifier would need to evaluate

$$\left[\frac{\chi\upsilon}{\Delta^2}\right] G = [\chi\upsilon] \cdot [\Delta^{-2}]G = [\chi\upsilon]G^{-2}.$$

However, the verifier possesses only G and $G^{-1} = [\Delta^{-1}]G$, not $G^{-2} = [\Delta^{-2}]G$. Computing G^{-2} from (G, G^{-1}) is equivalent to computing $[\Delta^{-1}]G^{-1}$, which requires knowledge of Δ^{-1}. Extracting Δ from the pair $(G, [\Delta^{-1}]G)$ constitutes an instance of the ECDLP, which has advantage at most $2^{-\kappa/2}$ by Lemma 5.10.

Step 4: Bound on query probability.

By Steps 1–3, conditioned on $\mathcal{V}$ (excluding the oracle response at U^*), the point U^* is uniformly distributed over a subset $S \subseteq \langle G \rangle$ of cardinality $|S| = (q-1)/2$. For any

(possibly adaptive) query strategy making at most q_X queries to $\mathcal{G}_{\mathsf{RO}}^X$, each query hits U^* with probability $|S|^{-1} = 2/(q-1)$. By a union bound:

$$\Pr[\text{verifier queries } U^*] \leq \frac{2q_X}{q-1} < \frac{2q_X}{q}.$$

Case 2: word2hot conversion (Section 4.4.4).
By Definition 5.12, $U^* = [r\Delta^a]G'$ where $G' = [K]G$ for a uniformly random $K \in Z_q^*$ chosen by the prover. The verifier receives:

- Word garbling $\Omega(a) = r\Delta^a$;
- Cleartext $a \in Z_n$;
- Generators $G'_i = [\Delta^i]G'$ for $i \in \{-(n-1), \ldots, n-1\} \setminus \{0\}$.

Crucially, $G'_0 = G'$ is *not* provided to the verifier. The verifier can compute

$$[\Omega(a)]G'_c = [r\Delta^{a+c}]G' \quad \text{for } c \neq 0,$$

but cannot directly compute $[\Omega(a)]G' = [r\Delta^a]G' = U^*$ without access to G'.
Now, $U^* = [r\Delta^a \cdot K]G = [\Omega(a) \cdot K]G$. Since K is uniform over Z_q^* and independent of the verifier's entire view (which contains only $G'_i = [\Delta^i K]G$ for $i \neq 0$, and $\Omega(a)$, but not K itself), the scalar $\Omega(a) \cdot K$ is uniform over Z_q^*. Therefore U^* is uniformly distributed over all of $\langle G\rangle \setminus \{\mathcal{O}\}$.
By a union bound over q_X queries: $\Pr[\text{verifier queries } U^*] \leq \frac{q_X}{q-1} < \frac{q_X}{q}$.
Combining both cases, for each object produced by the protocol, the unblinded hot input U^* is uniformly distributed over at least $(q-1)/2$ elements of $\langle G\rangle$ from the verifier's perspective. The probability that a verifier making at most q_X oracle queries hits U^* satisfies $\Pr[\text{verifier queries } \mathcal{G}_{\mathsf{RO}}^X \text{ on } U^*] \leq \frac{2q_X}{q}$, which is negligible for polynomial q_X and $q \approx 2^{256}$. □

Lemma 5.14 (Secrecy of Δ)**.** *Consider an honest-prover, corrupt-verifier execution of π_f on input x in the GUC-RO model with global $\mathcal{G}_{\mathrm{RO}}^X$. Let q_X be the total number of X-oracle queries issued by V^* during the session, and let $q = |\mathbb{Z}_q|$ be the group order. Then the view of $V^\star$ with $\Delta \leftarrow \mathbb{Z}_q^*$ is statistically indistinguishable from its view with an independent $\Delta' \leftarrow \mathbb{Z}_q^*$, with distinguishing advantage at most q_X/q.*

Proof. All scalars are in $\mathbb{Z}_q^*$ and X is realized by the global random oracle $\mathcal{G}_{\mathrm{RO}}^X$ with session/domain separation. Write $U_t = [\xi_t]B_t$ for the group elements the prover feeds to X while computing tables/vectors, and let $U^\star$ be, for each such object, the input at its hot index (Definition 5.12). Let h denote the product of all non-hot X-outputs in a given object, and recall $H = \Delta \cdot \prod_t X(U_t)$ from equation (4).

- G_0 (real): Run the real execution with $\Delta \xleftarrow{\$} \mathbb{Z}_q^*$.
- G_1 (lazy RO): Answer all X-queries by lazy sampling: upon first query on $U \in \mathbb{G}$, sample $X(U) \xleftarrow{\$} \mathbb{Z}_q^*$ and memorize. Identical to G_0.
- G_2 (fix all non-hot entries): For each table/vector, for every non-hot index $t \neq t^\star$, set the oracle answer $X(U_t)$ to fresh independent values (via $\mathcal{G}_{\mathrm{RO}}^X$.Program on inputs not yet queried). Let h be their product and set the hot output to $H \cdot h^{-1}$. By Lemma 4.23, this equals $\Delta \cdot X(U^\star)$ and preserves the joint distribution of all entries. Let $\mathsf{HotQuery} = \bigvee V^*$ queries X on an unblinded hot input $U^\star$. Nonce freshness makes each $U^\star$ a uniform group element hidden from V^*.

With q_X total RO queries and session/tag separation, $\Pr[\mathsf{HotQuery}] \le q_X/q$ by a union bound. Hence $\big|\Pr[G_1 = 1] - \Pr[G_2 = 1]\big| \le q_X/q$.

- G_3 (switch Δ): Define G_3 to be the same as G_2, except that we sample an independent $\Delta' \xleftarrow{\$} \mathbb{Z}_q^*$ instead of Δ and use Δ' in the computation of all help values and hot entries. Conditioned on $\neg\mathsf{HotQuery}$, the verifier never queries X on an unblinded hot input $U^\star$. Therefore all X-outputs that appear in its view are either:
  - values $X(U)$ for inputs U that do not involve Δ at all, or
  - (for the hot entries) products of the form $\Delta \cdot Z_{ab}$ where Z_{ab} is an X output on a point $U^\star$ that was never queried.

  In the second case, Z_{ab} is uniform in $\mathbb{Z}_q^*$ and unknown to the verifier, hence $\Delta \cdot Z_{ab}$ is uniform in $\mathbb{Z}_q^*$ and its distribution is independent of the concrete value of Δ. Replacing Δ by Δ' therefore leaves the joint distribution of the verifier's view unchanged: conditioned on $\neg\mathsf{HotQuery}$ we have $\Pr[G_2 = 1 \mid \neg\mathsf{HotQuery}] = \Pr[G_3 = 1 \mid \neg\mathsf{HotQuery}]$, and hence $|\Pr[G_2 = 1] - \Pr[G_3 = 1]| = 0$.

Summing the losses, we get: $\big|\Pr[G_0 = 1] - \Pr[G_3 = 1]\big| \le \Pr[\mathsf{HotQuery}] \le q_X/q$. □

Algorithm 9 Ideal-World Simulator Sim_f, honest prover

Require: Security parameter κ, prime $p \approx 2^\kappa$, circuit C with gate computing $f : \mathbb{Z}_n \to \mathbb{Z}_m$, input wire set $A = \{a_0, \dots, a_{n-1}\}$, output wire set $O = \{o_0, \dots, o_{m-1}\}$, adversary $\mathcal{A}$ controlling corrupt verifier, ideal functionality $\mathcal{F}_f$.

Ensure: Simulated view indistinguishable from real-world execution of $\mathcal{F}_f$.

Simulator Sim_f: Initial Setup

Choose $\Delta' \in \mathbb{Z}_p^*$ uniformly at random (simulating secret global correlation).

Receive input $x \in \mathbb{Z}_n$ from environment $\mathcal{Z}$ and send to $\mathcal{F}_f$.

Simulate $\mathsf{Gb}(1^\kappa, C)$ and $\mathsf{En}(f_e, x)$

For each input wire $a_i \in A$, choose nonce $R'_{A,i} \in \mathbb{Z}_p^*$ uniformly at random.

Compute simulated garbled input: $W'_A(x)_i = R'_{A,i}\Delta'^{\delta_i(x)}$ for $i \in \mathbb{Z}_n$, where $\delta_i(x) = 1$ if $i = x$, else 0.

Send $W'_A(x)$ and cleartext x to $\mathcal{A}$ (corrupt verifier).

Simulate $\mathsf{Ev}(C, W'_A(x))$

Receive garbled output $W_O(f(x)) = (R_{O,k}\Delta^{\delta_k(f(x))})_{k\in\mathbb{Z}_m}$ from $\mathcal{F}_f$.

Send $W_O(f(x))$ to $\mathcal{A}$ (no local computation of $\breve{f}$, rely on $\mathcal{F}_f$).

Handle Adaptive Corruption of Prover

if $\mathcal{A}$ requests corruption of prover **then**

    Reveal Δ' and $\{R'_{A,i}\}_{i\in\mathbb{Z}_n}$ to $\mathcal{A}$, consistent with $W'_A(x)_i = R'_{A,i}\Delta'^{\delta_i(x)}$.

end if

Theorem 5.15 (UC Authenticity of Protocol π_f Realizing $\mathcal{F}_f$)**.** *Let π_f be the protocol for garbling and evaluating the function $f : \mathbb{Z}_n \to \mathbb{Z}_m$ as defined in Algorithm 7, and let $\mathcal{F}_f$ be the corresponding ideal functionality as defined in Algorithm 8. For any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ controlling a corrupt verifier and*

any environment $\mathcal{Z}$, the real-world execution $\mathsf{REAL}_{\mathcal{A},\pi_f}(\mathcal{Z})$ is computationally indistinguishable from the ideal-world execution $\mathsf{IDEAL}_{\mathsf{Sim}_f,\mathcal{F}_f}(\mathcal{Z})$ with simulator Sim_f as defined in Algorithm 9. The advantage $\mathsf{Adv}_{\mathcal{Z}}(\kappa)$ is negligible (that is, $\mathsf{Adv}_{\mathcal{Z}}(\kappa) \leq \mathsf{negl}(\kappa)$) for security parameter κ, ensuring that π_f securely realizes $\mathcal{F}_f$ with UC authenticity in the honest prover, corrupt verifier scenario with adaptive prover corruption.

Proof. We prove that protocol π_f (Algorithm 7) securely realizes ideal functionality $\mathcal{F}_f$ (Algorithm 8) with simulator Sim_f (Algorithm 9), ensuring UC authenticity in the honest prover, corrupt verifier scenario with adaptive prover corruption. Authenticity guarantees that a corrupt verifier cannot forge a valid garbled output $W_O(f(x))$ for $x' \neq x$ without knowing Δ and nonces $R_{A,i}$.
Let $\mathcal{A}$ be a PPT adversary controlling the corrupt verifier, and $\mathcal{Z}$ be the environment. We show that $\mathsf{REAL}_{\mathcal{A},\pi_f}(\mathcal{Z}) \approx_c \mathsf{IDEAL}_{\mathsf{Sim}_f,\mathcal{F}_f}(\mathcal{Z})$ with negligible advantage $\mathsf{Adv}_{\mathcal{Z}}(\kappa)$.
Hybrid Argument: we construct a sequence of hybrid experiments H_0, H_1, H_2 to demonstrate indistinguishability:

(1) H_0 (Real-World Execution): The execution of π_f with $\mathcal{A}$. The prover chooses $\Delta \in \mathbb{Z}_p^*$, nonces $R_{A,i} \in \mathbb{Z}_p^*$, computes $W_A(x)_i = R_{A,i}\Delta^{\delta_i(x)}$ (line 4), sends $W_A(x)$ and x to $\mathcal{A}$ (line 5), and $\mathcal{A}$ computes $W_O(f(x))$ (line 8).
(2) H_1 (Simulated Garbling): The simulator Sim_f chooses $\Delta' \in \mathbb{Z}_p^*$ and $R'_{A,i} \in \mathbb{Z}_p^*$ (lines 1, 3), computes $W'_A(x)_i = R'_{A,i}\Delta'^{\delta_i(x)}$ (line 4), and sends $W'_A(x)$ and x to $\mathcal{A}$ (line 5). The view is identical to H_0 due to the uniform randomness of Δ, Δ', $R_{A,i}$, and $R'_{A,i}$ (probability $1/p \approx 2^{-\kappa}$).
(3) H_2 (Ideal-World Execution): Sim_f sends x to $\mathcal{F}_f$ (line 2), receives $W_O(f(x))$ (line 6), and sends it to $\mathcal{A}$ (line 7). The view matches H_1 since $\mathcal{F}_f$'s $W_O(f(x))$ is computed identically to π_f's $W_O(f(x))$ (lines 4 vs. 8).

Indistinguishability: We show that H_0, H_1, and H_2 are computationally indistinguishable.

- $H_0 \approx_s H_1$: In H_0 the honest prover samples $(\Delta, \{R_{A,i}\})$ uniformly in $\mathbb{Z}_q^*$ and computes all messages according to π_f. In H_1 the simulator instead samples $(\Delta', \{R'_{A,i}\}) \leftarrow (\mathbb{Z}_q^*)^{1+n}$ and computes the same algebraic expressions. By Lemma 5.14, the verifier's view is statistically independent of the particular choice of the global correlation Δ, up to the Bad event whose probability is a most q_X/q. The nonces $R_{A,i}$ appear only as fresh uniform scalars and resampling them does not change their distribution. Hence H_0 and H_1 are statistically indistinguishable, with distinguishing advantage bounded by q_X/q.
- $H_1 \approx_c H_2$: In both hybrids the values $(\Delta', \{R'_{A,i}\})$ are sampled identically, and the garbled input $W'_A(x)$ is computed as $W'_A(x)_i = R'_{A,i}\Delta'^{\delta_i(x)}$. The only difference is whether the garbled output is computed locally by $\breve{f}(W'_A(x))$ or obtained from the ideal functionality $\mathcal{F}_f$ (Algorithm 8). Since $\mathcal{F}_f$ uses the same computation rule, the distributions of $W_O(f(x))$ in H_1 and H_2 are identical, and so are the verifier's views.

We conclude that for all PPT adversaries $\mathcal{A}$ and environments $\mathcal{Z}$ the ensembles $\mathsf{REAL}_{\mathcal{A},\pi_f}(\mathcal{Z})$ and $\mathsf{IDEAL}_{\mathsf{Sim}_f,\mathcal{F}_f}(\mathcal{Z})$ are indistinguishable up to $q_X/q = \mathsf{negl}(\kappa)$. □

Corollary 5.16 (Authenticity of π_f)**.** *In the honest-prover, malicious-verifier setting, protocol π_f satisfies authenticity in the sense of Definition 2.4.*

Proof. By Lemma 5.11, any ideal execution with $\mathcal{F}_f$ enjoys perfect authenticity. If a PPT adversary $\mathcal{A}$ against π_f could win the authenticity game with non-negligible probability, we could build an environment $\mathcal{Z}$ that simply runs that game inside the UC experiment and outputs 1 iff $\mathcal{A}$ produces a successful forgery.
By Theorem 5.15 (and Theorem 5.3), the outputs of $\mathcal{Z}$ in the real and ideal executions must be computationally indistinguishable. But in the ideal execution the forgery probability is 0, whereas in the real execution it is non-negligible by assumption. This contradicts UC security, hence the forging advantage against π_f must be negligible. □

5.2.1. *Tensor products and several functions maps.* In this section we prove authenticity of the tensor product evaluation, and from it, the security of the evaluation of a function, and UC security, it follows the authenticity of evaluation of functions of several variables.

Proposition 5.17 (Randomness of the Outcome)**.** *The outcome*

$$T_{A\otimes B}(a,b)_{ij} = X([R_{A,i}R_{B,j}]G)\Delta^{\delta_i(a)\delta_j(b)}$$

is pseudorandom for a PPT adversary $\mathcal{A}$, given that $R_{A,i}$, $R_{B,j}$ are (pseudo-)random variables and Δ is a fixed secret value.

Proof. The product $R_{A,i}R_{B,j}$ of two (pseudo-)random variables from $\mathbb{Z}_p^*$ remains a (pseudo-)random variable in $\mathbb{Z}_p^*$, as multiplication over a finite field preserves the uniform distribution when inputs are independent and uniform [17]. The operation $[R_{A,i}R_{B,j}]G$ maps the scalar $R_{A,i}R_{B,j}$ to a point on $E(\mathbb{Z}_p)$. Since $R_{A,i}R_{B,j} \neq 0 \mod q$, the resulting point is uniformly distributed over the subgroup generated by G (order q), assuming the Discrete Logarithm Problem (DLP) is hard [19]. First Coordinate Extraction: the function X extracts the first coordinate of $[R_{A,i}R_{B,j}]G$. For a random point in the subgroup of order q, the first coordinate is pseudorandom over $\mathbb{Z}_p$ if the curve is secure, as shown in the following lemma adapted from [18]:

Lemma 5.18 (Statistical properties of coordinate extraction)**.** *Let $\kappa \leq 256$ and define $X_\kappa : \mathcal{G} \setminus \{\mathcal{O}\} \to \{0,1\}^\kappa$ by $X_\kappa(P) := \lfloor x(P)/2^{256-\kappa} \rfloor$, that is, the leading κ bits of the x-coordinate. For a sampled uniformly from $\mathbb{Z}_q^*$, the distribution of $X_\kappa([a]G)$ satisfies:*

$$\Delta(X_\kappa([a]G),\, U_\kappa) \;\leq\; \frac{3\cdot 2^{\kappa-1}\sqrt{p}\log p}{q} \;=\; O(2^{\kappa-119}),$$

where U_κ denotes the uniform distribution on $\{0,1\}^\kappa$ and $\Delta(\cdot,\cdot)$ is the statistical distance. In particular, the bound is negligible for $\kappa = O(\log \lambda)$.

Proof. For a fixed κ-bit prefix $c \in \{0,1\}^\kappa$, define the interval $I_c := \{x \in \mathbb{Z}_p : X_\kappa(x) = c\}$, which has cardinality $|I_c| = 2^{256-\kappa}$ (up to boundary effects of magnitude $O(1)$). The number of points $P = (x,y) \in E(\mathbb{Z}_p)$ with $x \in I_c$ is:

$$N_c := \sum_{x\in I_c}\left(1 + \left(\frac{x^3+7}{p}\right)\right) = |I_c| + \sum_{x\in I_c}\left(\frac{x^3+7}{p}\right),$$

where $\left(\frac{\cdot}{p}\right)$ denotes the Legendre symbol. By the Pólya–Vinogradov inequality [9], for any interval $I \subseteq \mathbb{Z}_p$ and non-trivial multiplicative character χ:

$$\left|\sum_{x \in I} \chi(f(x))\right| \leq d \cdot \sqrt{p} \cdot \log p,$$

where $d = \deg(f)$. Applying this with $f(x) = x^3 + 7$ and $d = 3$:

$$\left|\sum_{x \in I_c} \left(\frac{x^3 + 7}{p}\right)\right| \leq 3\sqrt{p} \log p.$$

Thus $N_c = 2^{256-\kappa} \pm 3\sqrt{p} \log p$. For $a \xleftarrow{\$} \mathbb{Z}_q^*$, the per-prefix probability deviation is:

$$\left|\Pr[X_\kappa([a]G) = c] - 2^{-\kappa}\right| = \frac{|N_c - 2^{256-\kappa}|}{q} \leq \frac{3\sqrt{p} \log p}{q}.$$

Summing over all 2^κ prefixes:

$$\Delta(X_\kappa([a]G), U_\kappa) = \frac{1}{2} \sum_{c \in \{0,1\}^\kappa} \left|\Pr[X_\kappa([a]G) = c] - 2^{-\kappa}\right| \leq 2^{\kappa-1} \cdot \frac{3\sqrt{p} \log p}{q}.$$

For parameters $\sqrt{p} \approx 2^{128}$, $\log p \approx 256$, $q \approx 2^{256}$, this yields $\Delta = O(2^{\kappa-119})$. □

For tensor products, $[\Delta^{\delta_i(a)\delta_j(b)}]G \neq G$ with probability $\approx 2^{-256}$ (Remark 4.20), ensuring the secrecy of Δ when $\delta_i(a)\delta_j(b) = 1$. The full composition yields a pseudorandom output, making $T_{A \otimes B}(a, b)_{ij}$ indistinguishable from a random element in $\mathbb{Z}_p$ for a PPT adversary. □

Remark 5.19. The statistical bound of Lemma 5.18 is meaningful only for small κ. For the full x-coordinate extraction used in our constructions ($\kappa = 256$), we rely on *computational* pseudorandomness: under the ECDLP assumption, no PPT adversary can distinguish $x([a]G)$ from a uniform element of $\mathbb{Z}_p$. In our UC proofs, we model X as a random oracle via the global functionality $\mathcal{G}_{\mathsf{RO}}^X$ (Section 5.2). The above lemma provides statistical justification for small extraction widths, while the ECDLP assumption justifies the random oracle idealization for full coordinate extraction.

Algorithm 10 Protocol π_{TP}: Verifier's Evaluation of Tensor Product

Require: Security parameter κ, prime $p \approx 2^\kappa$, circuit C with gate computing tensor product $T_{A\otimes B} : \mathbb{Z}_n \times \mathbb{Z}_m \to \mathbb{Z}_n \times \mathbb{Z}_m$, input wire sets $A = \{a_0, \ldots, a_{n-1}\}$ and $B = \{b_0, \ldots, b_{m-1}\}$, output wire set $O = \{o_{i,j} \mid i \in \mathbb{Z}_n, j \in \mathbb{Z}_m\}$, garbled inputs $W_A(a) = (W_A(a)_i)_{i\in\mathbb{Z}_n}$, $W_B(b) = (W_B(b)_j)_{j\in\mathbb{Z}_m}$, cleartext inputs $a \in \mathbb{Z}_n$, $b \in \mathbb{Z}_m$, generators $G, G_{-1} \in \mathbb{Z}_p^*$, function $X : \mathbb{Z}_p^* \to \mathbb{Z}_p^*$, help value $H \in \mathbb{Z}_p^*$ from prover.
Ensure: Output $T_{A\otimes B}(a, b)$, encoding the one-hot tensor product.

Initialize empty list $T_{A\otimes B} = []$
for $i \in \mathbb{Z}_n$ **do**
  for $j \in \mathbb{Z}_m$ **do**
    if $i \neq a$ **and** $j \neq b$ **then** $w_{i,j} \leftarrow X([W_A(a)_i \cdot W_B(b)_j]G)$
    else if $i \neq a$ **xor** $j \neq b$ **then** $w_{i,j} \leftarrow X([W_A(a)_i \cdot W_B(b)_j]G_{-1})$
    else $w_{a,b} \leftarrow 1$ ▷ $i = a$ **and** $j = b$
    end if
    Append $w_{i,j} = T_{A\otimes B}(a, b)_{ij}$ to $T_{A\otimes B}$
  end for
end for
Compute $h \leftarrow \prod_{i\in\mathbb{Z}_n, j\in\mathbb{Z}_m} w_{i,j}$
Compute $T_{A\otimes B}(a, b)_{a,b} \leftarrow H \cdot h^{-1} \mod p$ ▷ Update hot wire using help H
return $T_{A\otimes B}(a, b)$ ▷ Return full tensor product output

Algorithm 11 Ideal Functionality $\mathcal{F}_{\mathsf{TP}}$: Evaluation of Tensor Product $T_{A\otimes B}$

Require: Security parameter κ, prime $p \approx 2^\kappa$, circuit C with gate computing tensor product $T_{A\otimes B} : \mathbb{Z}_n \times \mathbb{Z}_m \to \mathbb{Z}_n \times \mathbb{Z}_m$, input wire sets $A = \{a_0, \ldots, a_{n-1}\}$ and $B = \{b_0, \ldots, b_{m-1}\}$, output wire set $O = \{o_{i,j} \mid i \in \mathbb{Z}_n, j \in \mathbb{Z}_m\}$, adversary $\mathcal{A}$, environment $\mathcal{Z}$.
Ensure: Garbled inputs $W_A(a)$, $W_B(b)$, output $T_{A\otimes B}(a, b)$.

Initialization
Choose global correlation $\Delta \in \mathbb{Z}_p^*$ (secret, fixed for circuit C) and nonces $R_{A,i}, R_{B,j} \in \mathbb{Z}_p^*$ for each $a_i \in A$, $b_j \in B$, known only to trusted party.
Compute help $H = \Delta \prod_{i\in\mathbb{Z}_n, j\in\mathbb{Z}_m} X([R_{A,i}R_{B,j}]G)$ (Definition 4.22).
On input $(a, b) \in \mathbb{Z}_n \times \mathbb{Z}_m$ **from prover** P**:**
Compute garbled inputs $W_A(a)_i = R_{A,i}\Delta^{\delta_i(a)}$ and $W_B(b)_j = R_{B,j}\Delta^{\delta_j(b)}$, where $\delta_i(a) = 1$ if $i = a$, else 0, and $\delta_j(b) = 1$ if $j = b$, else 0.
Send $W_A(a)$, $W_B(b)$, cleartext (a, b), and H to verifier V.
On request from verifier V **to evaluate** $T_{A\otimes B}$**:**
Compute $T_{A\otimes B}(a, b)_{i,j} = X([W_A(a)_i \cdot W_B(b)_j]G)$ for $i \neq a$ and $j \neq b$.
Compute $T_{A\otimes B}(a, b)_{i,j} = X([W_A(a)_i \cdot W_B(b)_j]G_{-1})$ for $i \neq a$ xor $j \neq b$.
Set $T_{A\otimes B}(a, b)_{a,b} = 1$.
Compute $h = \prod_{i\in\mathbb{Z}_n, j\in\mathbb{Z}_m} T_{A\otimes B}(a, b)_{i,j}$.
Update $T_{A\otimes B}(a, b)_{a,b} \leftarrow H \cdot h^{-1} \mod p$.
Send $T_{A\otimes B}(a, b)$ to V.
On adaptive corruption of prover P **by** $\mathcal{A}$**:**
if prover P is corrupted **then** Reveal Δ, $\{R_{A,i}\}_{i\in\mathbb{Z}_n}$, $\{R_{B,j}\}_{j\in\mathbb{Z}_m}$ to $\mathcal{A}$.
end if

Lemma 5.20 (Ideal authenticity of $\mathcal{F}_{\mathsf{TP}}$)**.** *Let $\mathcal{F}_{\mathsf{TP}}$ be the ideal functionality of Algorithm 11. In any ideal execution with honest prover P and arbitrary (possibly malicious) verifier $V^\star$, the adversary's success probability in the authenticity experiment of Definition 2.4, applied to the tensor product output $T_{A\otimes B}(a,b)$, is* 0*.*

Proof. Exactly as in Lemma 5.11: in the ideal world $V^\star$ only receives the garbled input $(W_A(a), W_B(b))$ for the actual input (a,b) and a single corresponding tensor-product output $T_{A\otimes B}(a,b)$, and never any labels for different inputs $(a',b') \neq (a,b)$ or different outputs. Thus any forged output that decodes to a valid value for $(a',b') \neq (a,b)$ is impossible, and the authenticity game outputs 1 with probability 0. □

Algorithm 12 Ideal-World Simulator $\mathsf{Sim}_{\mathsf{TP}}$, honest prover

Require: Security parameter κ, prime $p \approx 2^\kappa$, circuit C with gate computing tensor product $T_{A\otimes B} : \mathbb{Z}_n \times \mathbb{Z}_m \to \mathbb{Z}_n \times \mathbb{Z}_m$, input wire sets $A = \{a_0, \ldots, a_{n-1}\}$ and $B = \{b_0, \ldots, b_{m-1}\}$, output wire set $O = \{o_{i,j} \mid i \in \mathbb{Z}_n, j \in \mathbb{Z}_m\}$, adversary $\mathcal{A}$ controlling corrupt verifier, ideal functionality $\mathcal{F}_{\mathsf{TP}}$.

Ensure: Simulated view indistinguishable from real-world execution of π_{TP}.

Simulator $\mathsf{Sim}_{\mathsf{TP}}$: Initial Setup

Choose $\Delta' \in \mathbb{Z}_p^*$ uniformly at random (simulating secret global correlation).

Receive input $(a,b) \in \mathbb{Z}_n \times \mathbb{Z}_m$ from environment $\mathcal{Z}$ and send to $\mathcal{F}_{\mathsf{TP}}$.

Simulate Gb and En for $T_{A\otimes B}$

For each input wire $a_i \in A$ and $b_j \in B$, choose nonces $R'_{A,i}, R'_{B,j} \in \mathbb{Z}_p^*$ uniformly at random.

Compute simulated garbled inputs: $W'_A(a)_i = R'_{A,i}\Delta'^{\delta_i(a)}$ and $W'_B(b)_j = R'_{B,j}\Delta'^{\delta_j(b)}$, where $\delta_i(a) = 1$ if $i = a$, else 0, and $\delta_j(b) = 1$ if $j = b$, else 0.

Compute simulated help $H' = \Delta' \prod_{i\in\mathbb{Z}_n, j\in\mathbb{Z}_m} X([R'_{A,i}R'_{B,j}]G)$.

Send $W'_A(a)$, $W'_B(b)$, cleartext (a,b), and H' to $\mathcal{A}$ (corrupt verifier).

Simulate Ev for $T_{A\otimes B}$

Receive $T_{A\otimes B}(a,b)$ from $\mathcal{F}_{\mathsf{TP}}$.

Send $T_{A\otimes B}(a,b)$ to $\mathcal{A}$.

Handle Adaptive Corruption of Prover

if $\mathcal{A}$ requests corruption of prover **then**

  Reveal Δ', $\{R'_{A,i}\}_{i\in\mathbb{Z}_n}$, $\{R'_{B,j}\}_{j\in\mathbb{Z}_m}$ to $\mathcal{A}$, consistent with $W'_A(a)_i = R'_{A,i}\Delta'^{\delta_i(a)}$ and $W'_B(b)_j = R'_{B,j}\Delta'^{\delta_j(b)}$.

end if

Theorem 5.21 (Indistinguishability Proof for Authenticity of Protocol π_{TP} in UC Framework)**.** *The protocol π_{TP} (Algorithm 10) securely realizes ideal functionality $\mathcal{F}_{\mathsf{TP}}$ (Algorithm 11) with simulator $\mathsf{Sim}_{\mathsf{TP}}$ (Algorithm 12), ensuring UC authenticity in the honest prover, corrupt verifier scenario with adaptive prover corruption. Authenticity guarantees that a corrupt verifier cannot forge a valid tensor product output $T_{A\otimes B}(a',b')$ for $(a',b') \neq (a,b)$ without knowing Δ and nonces $R_{A,i}$, $R_{B,j}$.*

Proof. Let $\mathcal{A}$ be a PPT adversary controlling the corrupt verifier, and $\mathcal{Z}$ be the environment. We show that $\mathsf{REAL}_{\mathcal{A},\pi_{\mathsf{TP}}}(\mathcal{Z}) \approx_c \mathsf{IDEAL}_{\mathsf{Sim}_{\mathsf{TP}},\mathcal{F}_{\mathsf{TP}}}(\mathcal{Z})$ with negligible advantage $\mathsf{Adv}_{\mathcal{Z}}(\kappa)$.

Hybrid Argument: We construct a sequence of hybrid experiments H_0, H_1, H_2 to demonstrate indistinguishability:

(1) H_0 (Real-World Execution): The execution of π_{TP} with $\mathcal{A}$. The prover sends $W_A(a)$, $W_B(b)$, (a, b), and H to $\mathcal{A}$ (implied prior to evaluation). The verifier computes $w_{i,j}$ (lines 10-conditions), h, and $T_{A\otimes B}(a,b)_{a,b} = H \cdot h^{-1} \mod p$ (lines 10-compute), returning $T_{A\otimes B}(a, b)$.

(2) H_1 (Simulated Garbling): The simulator $\mathsf{Sim}_{\mathsf{TP}}$ chooses $\Delta' \in \mathbb{Z}_p^*$ and nonces $R'_{A,i}, R'_{B,j} \in \mathbb{Z}_p^*$ (lines 1, 3), computes $W'_A(a)_i = R'_{A,i}\Delta'^{\delta_i(a)}$, $W'_B(b)_j = R'_{B,j}\Delta'^{\delta_j(b)}$ (line 4), and $H' = \Delta' \prod_{i,j} X([R'_{A,i}R'_{B,j}]G)$ (line 5), sending them to $\mathcal{A}$ (line 6). The view is identical to H_0 due to uniform randomness of Δ, Δ', $R_{A,i}$, $R'_{A,i}$, $R_{B,j}$, and $R'_{B,j}$ (probability $1/p^2 \approx 2^{-2\kappa}$), and the pseudorandomness of $T_{A\otimes B}(a,b)_{i,j} = X([R_{A,i}R_{B,j}]G)\Delta^{\delta_i(a)\delta_j(b)}$ as per Proposition 5.17.

(3) H_2 (Ideal-World Execution): $\mathsf{Sim}_{\mathsf{TP}}$ sends (a, b) to $\mathcal{F}_{\mathsf{TP}}$ (line 2), gets $T_{A\otimes B}(a, b)$ (line 7), and sends it to $\mathcal{A}$ (line 8). The view matches H_1 since $\mathcal{F}_{\mathsf{TP}}$ computes $T_{A\otimes B}(a, b)$ identically to π_{TP} (lines 5-9 vs. 10-conditions), and the output remains pseudorandom (Proposition 5.17).

Indistinguishability:

- $H_0 \approx_c H_1$: The difference is the source of Δ, $R_{A,i}$, $R_{B,j}$, and H. Since these are chosen uniformly at random in π_{TP} and $\mathsf{Sim}_{\mathsf{TP}}$ uses Δ', $R'_{A,i}$, $R'_{B,j}$, and H' with the same distribution, the adversary's view (garbled inputs $W_A(a)$, $W_B(b)$, H) is indistinguishable. The pseudorandomness of $T_{A\otimes B}(a,b)_{i,j}$ (Proposition 5.17) ensures that $\mathcal{A}$ cannot distinguish the simulated garbled inputs from the real ones, with advantage bounded by $1/p^2 \approx 2^{-2\kappa}$ (Lemma 5.10, Definition 5.4).
- $H_1 \approx_c H_2$: The difference is that $T_{A\otimes B}(a, b)$ is computed by $\mathcal{A}$ in H_1 and received from $\mathcal{F}_{\mathsf{TP}}$ in H_2. Since $T_{A\otimes B}(a,b)_{i,j} = w_{i,j}$ (for $i \neq a$ or $j \neq b$) and $T_{A\otimes B}(a,b)_{a,b} = H \cdot h^{-1} \mod p$ are deterministic given $W_A(a)$, $W_B(b)$, H, and the evaluation rules, and $\mathcal{F}_{\mathsf{TP}}$ computes the same, the outputs are identical. The pseudorandomness (Proposition 5.17) further ensures indistinguishability, with advantage negligible $(2^{-\kappa})$.

Authenticity: A corrupt verifier $\mathcal{A}$ attempts to forge $T_{A\otimes B}(a', b')$ for $(a', b') \neq (a, b)$. In π_{TP}, $\mathcal{A}$ receives $W_A(a)$, $W_B(b)$, (a, b), and H, but Δ, $R_{A,i}$, and $R_{B,j}$ are secret. Forging requires computing $T_{A\otimes B}(a', b')_{i,j} = X([R_{A,i}R_{B,j}]G)\Delta^{\delta_i(a')\delta_j(b')}$, needing $R_{A,i}$, $R_{B,j}$ (probability $2^{-2\kappa}$), and Δ (ECDLP hardness, advantage $2^{-\kappa/2}$). The pseudorandomness of $T_{A\otimes B}(a,b)_{i,j}$ (Proposition 5.17) ensures $\mathcal{A}$ cannot infer Δ from $W_A(a)$, $W_B(b)$, or H. In the ideal world, $\mathcal{F}_{\mathsf{TP}}$ only provides $T_{A\otimes B}(a, b)$ for the submitted (a, b), and $\mathsf{Sim}_{\mathsf{TP}}$ reveals Δ', $R'_{A,i}$, $R'_{B,j}$ only on corruption. Thus, $\mathcal{A}$ cannot distinguish or forge, ensuring authenticity.

Adaptive Corruption: If $\mathcal{A}$ corrupts the prover, π_{TP} reveals Δ, $\{R_{A,i}\}$, $\{R_{B,j}\}$ (implicit in state), and $\mathsf{Sim}_{\mathsf{TP}}$ reveals Δ', $\{R'_{A,i}\}$, $\{R'_{B,j}\}$ (line 10), consistent with $W'_A(a)$, $W'_B(b)$. The views remain indistinguishable, with pseudorandomness (Proposition 5.17) preserving security.

Conclusion: By the hybrid argument, $\mathsf{REAL}_{\mathcal{A},\pi_{\mathsf{TP}}}(\mathcal{Z}) \approx_c \mathsf{IDEAL}_{\mathsf{Sim}_{\mathsf{TP}},\mathcal{F}_{\mathsf{TP}}}(\mathcal{Z})$ with advantage $\mathsf{Adv}_{\mathcal{Z}}(\kappa) \leq 2^{-\kappa/2} + 2^{-2\kappa}$, negligible for $\kappa \geq 128$. Thus, π_{TP} securely realizes $\mathcal{F}_{\mathsf{TP}}$ with UC authenticity, supported by the pseudo-randomness of $T_{A\otimes B}(a,b)_{i,j}$ (Proposition 5.17). □

Remark 5.22. The authenticity property for the evaluation of $f : \mathbb{Z}_n \times \mathbb{Z}_n \to \mathbb{Z}_d$ now follows from the authenticity of the evaluation of the tensor product, and an argument very similar to the one used to prove the authenticity of π_f above. The UC composition theorem [10] ensures this holds.

Corollary 5.23 (Authenticity of π_{TP})**.** *In the honest-prover, malicious-verifier setting, protocol π_{TP} satisfies authenticity in the sense of Definition 2.4 for the tensor product output $T_{A\otimes B}(a, b)$.*

Proof. By Lemma 5.20, any ideal execution with $\mathcal{F}_{\mathsf{TP}}$ has perfect authenticity. By Theorem 5.21 and the UC theorem (Theorem 5.3), the real execution of π_{TP} is indistinguishable from the ideal one. A non-negligible authenticity break in the real world would thus yield a distinguisher between the two, contradicting UC security. □

5.2.2. *The* not *and* bin2hot *functionalities.* The not functionality implements, at the garbled level, the "opposite" operation introduced in Remark 4.38: for a Boolean wire $w(z) = R\Delta^z$, the cleartext negation $\mathsf{NOT}(z) = 1 - z$ corresponds to mapping the label $w(z)$ to its opposite label $w(z)^{-1}$. Any forgery would require computing the secret Δ, which is assumed hard under ECDLP (Lemma 5.10). For a Boolean wire w_i with garblings $w_i(0) = R_i$ and $w_i(1) = R_i\Delta$, the NOT gate computes $f(z) = 1 - z$, producing the output garbling $w_o(1-z) = w_i(z)\cdot\Delta^{1-2z}$ without additional cyphertext. A corrupt verifier, lacking knowledge of the secret correlation $\Delta \in \mathbb{Z}_p^*$, cannot forge a valid output garbling $w_o(1 - z')$ for $z' \neq z$, as this requires computing Δ, which is protected by the ECDLP with advantage at most $2^{-\kappa/2}$ (Lemma 5.10). Thus, the NOT gate preserves authenticity in the game-based sense of Definition 2.4: informally, any successful forgery would require computing the secret Δ, which is assumed hard under the ECDLP (Lemma 5.10). The advantage is $\mathsf{Adv}(\kappa) \leq 2^{-\kappa/2} + 2^{-\kappa}$. From here, and by combining it with the authenticity of the tensor product $T_{A\otimes B}$, the bin2hot functionality of Section 4.4.1 also has the authenticity property in the same game-based sense. A fully formal UC treatment would introduce dedicated ideal functionalities for not and bin2hot and then appeal to the UC composition theorem [10]; we omit these details for brevity.

Algorithm 13 Protocol π_{bin2hot}: Garbling and Evaluation of $\mathsf{bin2hot} : \{0,1\}^d \to \mathbb{Z}_n$

Require: Security parameter κ, prime $p \approx 2^\kappa$, elliptic curve $E(\mathbb{Z}_p)$ with generator G, function $X : \mathbb{Z}_p^* \to \mathbb{Z}_p^*$, input wires $W_I = \{w_{i_0}, \ldots, w_{i_{d-1}}\}$ with garbled inputs $w_{i_j}(a_j) = R_{i_j}\Delta^{a_j}$ for bits $a_j \in \{0,1\}$, $j \in \mathbb{Z}_d$, $d = \lceil \log_2 n \rceil$, encoding $a = \sum_{j=0}^{d-1} a_j 2^j \in \mathbb{Z}_n$, output wire set $I = \{i_0, \ldots, i_{n-1}\}$ for $\mathbb{Z}_n$, global correlation $\Delta \in \mathbb{Z}_p^*$ (secret, known only to prover), nonces $R_{i_j} \in \mathbb{Z}_p^*$, generators $G_c = [\Delta^c]G$ for $c \in \mathbb{Z}_d$.

Ensure: Garbled output $W_I(a) = (W_I(a)_k)_{k\in\mathbb{Z}_n}$, where $W_I(a)_k = R_{I,k}\Delta^{\delta_k(a)}$, $\delta_k(a) = 1$ if $k = a$, else 0.

Prover (Garbler): Algorithm $\mathsf{Gb}(1^\kappa, C)$

Compute nonces for tensor product $T_{\otimes H}$: for $k \in \mathbb{Z}_{2^d}$, $R_{\otimes H,\mathsf{bin}(k)} = X(\prod_{j\in\mathbb{Z}_d}[r_{i_j,k[j]}]G)$, where $\mathsf{bin}(k) = k[0]\ldots k[d-1]$, and $r_{i_j,k[j]} = R_{i_j}$ if $k[j] = 1$, else $r_{i_j,k[j]} = (R_{i_j}\Delta)^{-1}$.

Compute help value $H = \Delta \prod_{k\in\mathbb{Z}_{2^d}} X(\prod_{j\in\mathbb{Z}_d} R_{i_j}^{2k[j]-1}[\Delta^{k[j]-1}]G)$.

Send $w_{i_j}(a_j)$, cleartext a_j, generators G_c, and H to verifier.

Verifier (Evaluator): Algorithm $\mathsf{Ev}(C, \{w_{i_j}(a_j)\}_{j\in\mathbb{Z}_d})$

Receive $w_{i_j}(a_j)$, a_j, G_c, and H.

For each $j \in \mathbb{Z}_d$, compute auxiliary one-hot vector $H_{i_j}(a_j) = ((R_{i_j}\Delta)^{-1}\Delta^{1-a_j}, R_{i_j}\Delta^{a_j})$.

For each $k \in \mathbb{Z}_{2^d}$, compute hot counting function $c(k) = \sum_{i=0}^{d-1}(T_{\mathsf{bin}(k),a_0\ldots a_{d-1}})[i]$, where $(T_{\mathsf{bin}(k),a_0\ldots a_{d-1}})[i] = 1$ if $k[i] = a_i$, else 0.

For each $\mathsf{bin}(k) \in \{0,1\}^d$, compute tensor product entry:

if $\mathsf{bin}(k) \neq a_0\ldots a_{d-1}$ **then**

  $w_k = X(\prod_{j\in\mathbb{Z}_d}[w_{i_j}^{2k[j]-1}]G_{c(k)})$, where $w_{i_j}^{2k[j]-1} = w_{i_j}(a_j)$ if $k[j] = a_j$, else $(w_{i_j}(a_j))^{-1}$.

else ▷ $\mathsf{bin}(k) = a_0\ldots a_{d-1}$

  $w_k = 1$.

end if

Compute $h = \prod_{k\in\mathbb{Z}_{2^d}} w_k$.

For each $k \in \mathbb{Z}_n$, set output wire:

if $k \neq a$ **then**

  $W_I(a)_k = w_k$.

else ▷ $k = a$

  $W_I(a)_a = H \cdot h^{-1} \mod p$.

end if

return $W_I(a) = (W_I(a)_k)_{k\in\mathbb{Z}_n}$.

5.2.3. *The* word2hot *functionality.* This is the protocol π_{word2hot} for the word2hot functionality (Section 4.4.4), which converts a word wire garbling $\Omega(a) = r\Delta^a \in \mathbb{Z}_p^*$ into a one-hot vector garbling $W_I(a)_j = R_{i_j}\Delta^{\delta_j(a)}$ for $a \in \mathbb{Z}_n$. The protocol preserves game-based authenticity for the one-hot output: intuitively, a corrupt verifier cannot forge a valid one-hot output for $a' \neq a$ without either recovering Δ (breaking ECDLP) or guessing fresh nonces. The pseudorandomness of X (Proposition 5.17) secures the

intermediate wires. We do not introduce a separate UC functionality for word2hot; we treat its security in this stand-alone, game-based sense.

Algorithm 14 Protocol π_{word2hot}: Garbling and Evaluation of $\mathsf{word2hot} : \mathbb{Z}_n \to \mathbb{Z}_n$

Require: Security parameter κ, prime $p \approx 2^\kappa$, elliptic curve $E(\mathbb{Z}_p)$ with generator G, circuit C with gate computing word2hot, input wire with garbling $\Omega(a) = r\Delta^a \in \mathbb{Z}_p^*$ for cleartext $a \in \mathbb{Z}_n$, output wire set $I = \{i_0, \ldots, i_{n-1}\}$ for $\mathbb{Z}_n$, global correlation $\Delta \in \mathbb{Z}_p^*$ (secret, known only to prover), random nonce $r \in \mathbb{Z}_p^*$, function $X : \mathbb{Z}_p^* \to \mathbb{Z}_p^*$.

Ensure: Garbled output $W_I(a) = (W_I(a)_j)_{j\in\mathbb{Z}_n}$, where $W_I(a)_j = R_{i_j}\Delta^{\delta_j(a)}$.

Prover (Garbler): Algorithm $\mathsf{Gb}(1^\kappa, C)$

Choose random $k \in \mathbb{Z}_p^*$ and compute generator $G' = [K]G$.
Compute generators $G'_i = [\Delta^i]G'$ for $i \in \{-(n-1), \ldots, -1, 1, \ldots, n-1\}$.
Define output nonces $R_{i_j} = X([r\Delta^j]G')$ for $j \in \mathbb{Z}_n$.
Compute help value $H = \Delta \prod_{j=0}^{n-1} X([r\Delta^j]G')$.

Prover (Garbler): Algorithm $\mathsf{En}(f_e, a)$

Input cleartext $a \in \mathbb{Z}_n$.
Compute garbled input $\Omega(a) = r\Delta^a$.
Send $\Omega(a)$, cleartext a, G'_i, and H to verifier.

Verifier: Algorithm $\mathsf{Ev}(C, \Omega(a))$

Receive $\Omega(a)$, cleartext a, G'_i, and H.
For each $j \in \mathbb{Z}_n$, $j \neq a$, compute $W_I(a)_j = X([\Omega(a)]G'_{j-a})$.
Compute $h = \prod_{j\in\mathbb{Z}_n, j\neq a} W_I(a)_j$.
Compute $W_I(a)_a = H \cdot h^{-1} \mod p$.
Output $W_I(a) = (W_I(a)_j)_{j\in\mathbb{Z}_n}$.

The protocol preserves game-based authenticity for the one-hot output: intuitively, forging a valid output for $a' \neq a$ would again require computing Δ or predicting new nonces. The pseudorandomness of X (Proposition 5.17) secures intermediate wires. We do not introduce a separate UC functionality for word2hot.

5.2.4. *Multiplication of large numbers.* This protocol computes $M(a, b) = ab \pmod m$ for large integers $a, b \in \mathbb{Z}_m$ with $m = 2^{31} - 1$, per Remark 4.45 in the previous section. The gate uses discrete logarithms to achieve $O(m)$ complexity, requiring only one cyphertext.

Algorithm 15 Protocol π_{mult}: Garbling and Evaluation of Multiplication Gate for Large Integers

Require: Security parameter κ, prime $p \approx 2^\kappa$, elliptic curve $E(\mathbb{Z}_p)$ with generator G, Mersenne prime $m = 2^{31} - 1$, generator $g = 2$ for $\mathbb{Z}_m^*$, multiplication gate $M : \mathbb{Z}_m \times \mathbb{Z}_m \to \mathbb{Z}_m$, $M(a, b) = ab \pmod m$, input wire sets $A = \{a_0, \ldots, a_{m-1}\}$, $B = \{b_0, \ldots, b_{m-1}\}$ with garbled inputs $W_A(a)_i = R_{A,i}\Delta^{\delta_i(a)}$, $W_B(b)_j = R_{B,j}\Delta^{\delta_j(b)}$ for $i, j \in \mathbb{Z}_m$, $\delta_i(a) = 1$ if $i = a$, else 0, and similarly for $\delta_j(b)$, cleartext inputs $a, b \in \mathbb{Z}_m$ from previous gate, output wire set $O = \{o_0, \ldots, o_{m-1}\}$, global correlation $\Delta \in \mathbb{Z}_p^*$ (secret, known only to prover), random nonces $R_{A,i}, R_{B,j}, r \in \mathbb{Z}_p^*$, function $X : \mathbb{Z}_p^* \to \mathbb{Z}_p^*$, generator $G' = [K]G$ for random $k \in \mathbb{Z}_p^*$, generators $G'_i = [\Delta^i]G'$ for $i \in \{-(m-1), \ldots, -1, 1, \ldots, m-1\}$, precomputed discrete logarithm table $l : \mathbb{Z}_m^* \to \mathbb{Z}_m$ where $2^{l(x)} = x \pmod m$ and $l(0) = 2m - 3$.

Ensure: Garbled output $W_O(ab) = (W_O(ab)_i)_{i \in \mathbb{Z}_m}$, where $W_O(ab)_i = R_{O,i}\Delta^{\delta_i(ab)}$, $\delta_i(ab) = 1$ if $i = ab \pmod m$, else 0.

Prover (Garbler): Algorithm $\mathsf{Gb}(1^\kappa, M)$ **- Setup for Multiplication Gate**

1: Compute help value $H = \Delta \prod_{j=0}^{m-1} X([r\Delta^j]G')$ for word2hot.

2: Define gate M to apply: (1) basic functionality with l to $W_A(a)$, $W_B(b)$, (2) hot2word to outputs, (3) multiplication of word garblings, (4) word2hot using G'_i, H, (5) basic functionality with $e(x) = 2^x \pmod m$.

Verifier (Evaluator): Algorithm $\mathsf{Ev}(M, (W_A(a), W_B(b)))$

3: Apply basic functionality (Section 3.1) with discrete logarithm l to $W_A(a)$, $W_B(b)$, producing one-hot garblings $W_O(l(a))_i = R_{O,i}\Delta^{\delta_i(l(a))}$, $W_O(l(b))_j = R_{O,j}\Delta^{\delta_j(l(b))}$.

4: Apply hot2word (Section 3.4.1) to $W_O(l(a))$, $W_O(l(b))$, yielding word garblings $\Omega_l(l(a)) = R\Delta^{l(a)}$, $\Omega_l(l(b)) = R\Delta^{l(b)}$.

5: Compute $\Omega(l(ab)) = \Omega_l(l(a)) \cdot \Omega_l(l(b)) = R^2\Delta^{l(a)+l(b)} \pmod p$, where $l(a) + l(b) = l(ab) \pmod m$.

6: Apply word2hot (Section 3.4.2, Algorithm 9) to $\Omega(l(ab))$, using G'_i and H, producing $W_P(l(ab))_j = R_{i_j}\Delta^{\delta_j(l(ab))}$ for wire set $P = \{i_0, \ldots, i_{m-1}\}$.

7: Apply basic functionality (Section 3.1) with exponentiation $e : \mathbb{Z}_m \to \mathbb{Z}_m^*$, $e(x) = 2^x \pmod m$, to $W_P(l(ab))$, producing $W_O(ab)_i = R_{O,i}\Delta^{\delta_i(ab)}$, where $ab = e(l(ab)) \pmod m$.

8: **Output:** $W_O(ab) = (W_O(ab)_i)_{i \in \mathbb{Z}_m}$ for further processing in the circuit.

The protocol preserves authenticity for the product gate in a game-based sense: intuitively, a corrupt verifier cannot forge a different valid garbled product $W_O(ab')$ for $ab' \neq ab \pmod m$ without either guessing fresh nonces or breaking the ECDLP assumption (Lemma 5.10) and the pseudorandomness guarantees of Proposition 5.17. By modularity, this game-based authenticity extends to circuits that use the multiplication gate, but we do not formalize a dedicated UC functionality for it.

5.2.5. *The verification function.* The protocol π_{ver} for the verification function $V : \mathbb{Z}_m \to \mathbb{Z}_2$ (Section 5.7) follows, which verifies whether the garbled output $W_C(c')$ of circuit C matches the claimed value $c \in \mathbb{Z}_m$.

Definition 5.24 (On-chain fairness of π_{ver})**.** Consider the stand-alone execution of the verification protocol π_{ver} (Algorithm 16) between a prover P and an evaluator V, interacting with a ledger that maintains the coins and UTXOs used in the Bitcoin

integration example of Figure 1. Fix a function $F : \mathbb{Z}_n \to \mathbb{Z}_m$ and an input $x \in \mathbb{Z}_n$, and let $z^\star = F(x)$ be the correct output. Let $\mathsf{ver} : \mathbb{Z}_m \to \{0,1\}$ be the verification predicate implemented on-chain by π_{ver}, where $\mathsf{ver}(z^\star) = 1$ if and only if the intended success condition for $F(x)$ holds.

We model the ledger's final state by a bit $\mathsf{pay} \in \{0,1\}$ which is 1 if the ledger confirms a transaction that spends UTXO_1 via the prover's timeout path (transaction TX_2 creating UTXO_3 paying the prover), and 0 if the verifier successfully challenges via transaction TX_1 before timeout, or if the prover never spends UTXO_0.

We say that π_{ver} is *fair against a malicious prover* if for every PPT adversary $\mathcal{A}$ controlling P it holds that $\Pr\left[\mathsf{pay} = 1 \ \wedge \ \mathsf{ver}(z^\star) = 0\right] \leq \mathsf{negl}(\kappa)$, where the probability is over the randomness of $\mathcal{A}$, the honest party V, and the ledger. In words: except with negligible probability, the ledger never outputs a successful on-chain claim for P when the underlying cleartext computation $F(x)$ does not satisfy the verification predicate.

Algorithm 16 Protocol π_{ver}: Garbling and Evaluation of $\mathsf{ver} : \mathbb{Z}_m \to \mathbb{Z}_2$

Require: Security parameter κ, prime $p \approx 2^\kappa$, circuit C with output wires $W_C = (W_C(c')_i)_{i \in \mathbb{Z}_m}$ for cleartext $c' \in \mathbb{Z}_m$, claimed value $c \in \mathbb{Z}_m$, input wire nonces $R_{C,i} \in \mathbb{Z}_p^*$, global correlation $\Delta \in \mathbb{Z}_p^*$ (secret, known only to prover), output wire w_{ver}, hash function $H : \mathbb{Z}_p \to \{0,1\}^\lambda$.

Ensure: Garbled output $w_{\mathsf{ver}}(z) = R_f \Delta^z$, where $z = V(c') = c' - c + 1 \pmod 2$, and $R_f = R_{C,c}$.

Prover (Garbler): Algorithm $\mathsf{Gb}(1^\kappa, C)$

1: Set output nonce $R_f = R_{C,c}$.
2: Compute hash $h_f = \mathsf{Hash}(w_{\mathsf{ver}}(0)) = \mathsf{Hash}(R_f)$.
3: Embed h_f in UTXO_1 with script comparing $\mathsf{Hash}(r)$ for verifier's input nonce r.

Prover (Garbler): Algorithm $\mathsf{En}(f_e, c')$

4: Input cleartext $c' \in \mathbb{Z}_m$ (output of circuit C).
5: Send one-hot garbled output $W_C(c') = (W_C(c')_i)_{i \in \mathbb{Z}_m}$, where $W_C(c')_i = R_{C,i} \Delta^{\delta_i(c')}$, and cleartext c' to verifier.

Verifier: Algorithm $\mathsf{Ev}(C, W_C(c'))$

6: Receive $W_C(c')$ and cleartext c'.
7: Compute $z = V(c') = c' - c + 1 \pmod 2$.
8: Set $w_{\mathsf{ver}}(z) = W_C(c')_c$.
9: Compute hash $\mathsf{Hash}(w_{\mathsf{ver}}(z))$ and input nonce $r = w_{\mathsf{ver}}(z)$ to UTXO_1.

Blockchain Verification

10: If $\mathsf{Hash}(r) = \mathsf{Hash}(R_f)$ and $t \leq t_0$, verifier claims funds from UTXO_1, indicating $z = 0$ ($c' \neq c$).
11: If $t > t_0$, prover claims funds from UTXO_1, indicating $z = 1$ ($c' = c$).

The protocol achieves a stand-alone (game-based) authenticity guarantee: given the hash commitment $H(R_f)$ embedded in UTXO_1, a cheating party can cause an inconsistent on-chain claim (for instance claiming funds for $z = 1$ when $c' \neq c$) only with negligible probability, under the ECDLP assumption (Lemma 5.10) and the preimage resistance of H. The random-oracle modelling for H captures its preimage resistance

in our analysis. We do not introduce a dedicated UC functionality for the on-chain verification; our guarantees for π_{ver} are stand-alone and game-based.
We assume that the on-chain hash h_f embedded in the locking output is bound to the committed label hash for the verifier wire $w_v(0)$ posted to $\mathcal{G}_{\mathsf{trust}}$ with $Hash(w_v(0))$ validated via $\mathcal{G}_{\mathsf{trust}}$).

Theorem 5.25 (Fairness of π_{ver})**.** *Assume that the hash function H used in π_{ver} is preimage-resistant and collision-resistant, and that the underlying garbling scheme for F satisfies authenticity in the sense of Definition 2.4. Then, against a malicious prover, π_{ver} satisfies on-chain fairness in the sense of Definition 5.24.*

Proof. Fix any PPT adversary $\mathcal{A}$ controlling P and an input x with correct output $z^\star = F(x)$. By Definition 5.24, unfairness corresponds to the event that the ledger executes the "success" branch ($\mathsf{pay} = 1$) while $\mathsf{ver}(z^\star) = 0$.
In protocol π_{ver}, the ledger moves to the success branch (pay $= 1$) iff the prover successfully confirms the timeout transaction TX_2 after the timelock, and no valid challenge transaction TX_1 spending the same locked output was confirmed before the timelock. Equivalently, an unfair execution requires that the verifier did *not* manage to produce the correct preimage R_f in time (or did not publish it), even though the correct output fails verification.
If $\mathcal{A}$ can produce such an r with $H(r) = H(R_f)$ but $r \neq R_f$, then it breaks the collision/preimage resistance of H, against our assumption.
Otherwise, with overwhelming probability, we must have $r = R_f$. In this case, a successful unfair execution would require that $\mathsf{ver}(z) = 1$ while $\mathsf{ver}(z^\star) = 0$. Since ver is a deterministic predicate on the cleartext output of F, this means that z is an incorrect output—that is, the garbled output opened on-chain corresponds to a value different from $F(x)$. By the authenticity of the underlying garbling scheme (Definition 2.4 and Corollary 5.16), the probability that $\mathcal{A}$ can produce such a forgeable garbled output is negligible.
Combining the two cases, the probability that $\mathsf{pay} = 1 \wedge \mathsf{ver}(z^\star) = 0$ is bounded by the negligible advantage of breaking either the authenticity of the garbling or the preimage/collision resistance of H. Hence π_{ver} is fair in the sense of Definition 5.24. □

Remark 5.26. The functionalities in Subsections 4.4.6 and 4.4.5 (that is $\mathsf{word2bin}$ and $\mathsf{hot2word}$) inherit authenticity via Universal Composability.

5.3. **UC correctness in the honest verifier, corrupt prover scenario.** This subsection proves that the OHMG protocol π_{GC} securely realizes the ideal functionality $\mathcal{F}_{\mathsf{GC}}$ with UC correctness in the honest verifier, corrupt prover scenario with adaptive corruption, leveraging the cut-and-choose technique and $\mathcal{G}_{\mathsf{trust}}$'s ideal functionality, as defined by consistency and correctness in 2.3.

Remark 5.27 (Corruption model in the cut-and-choose analysis)**.** In the analysis of protocol π_{GC} we consider an honest evaluator V and a potentially malicious prover $P^\star$. We work with *adaptive corruption* of the prover *after the commitment phase*: that is, the adversary may corrupt P at any time once all garbled circuits and hash commitments have been posted to the global bulletin board $\mathcal{G}_{\mathsf{trust}}$.

This matches the standard cut-and-choose setting in which the binding property of the

commitments, together with the consistency checks on the opened subset of circuits, prevents a corrupted prover from changing its garblings retroactively. Corruption before the commitment phase would simply reveal P's randomness and is outside the scope of Theorem 5.30.

Algorithm 17 Protocol π_{GC}: Garbling and Evaluation of Circuit C Computing F

Require: Security parameter κ, prime $p \approx 2^\kappa$, circuit C computing functionality $F : \mathbb{Z}_n \to \mathbb{Z}_m$, input wire set $A = \{a_0, \ldots, a_{d-1}\}$ for binary input, output wire set $O = \{o_0, \ldots, o_{m-1}\}$, cleartext input $x \in \mathbb{Z}_n$ in binary form $x[0], \ldots, x[d-1]$, global correlation $\Delta \in \mathbb{Z}_p^*$ (fixed, secret), input nonces $R_{A,i} \in \mathbb{Z}_p^*$, output nonces $R_{O,j} \in \mathbb{Z}_p^*$, statistical security parameter s (for instance, $s = 80$ for 2^{-40} security).

Ensure: Garbled input $B_I(x)$, garbled output $W_O(F(x))$, cleartext output y.

Prover (Garbler): Algorithm $\mathsf{Gb}(1^\kappa, C)$

1: Use fixed global correlation $\Delta \in \mathbb{Z}_p^*$ (secret).
2: For each input wire $a_i \in A$, choose nonce $R_{A,i} \in \mathbb{Z}_p^*$ uniformly at random.
3: For each output wire $o_j \in O$, choose nonce $R_{O,j} \in \mathbb{Z}_p^*$ uniformly at random.
4: Generate garbled circuit GC for C using Δ, $R_{A,i}$, and $R_{O,j}$.
5: Commit to wire labels $w_{A,i}(\sigma)$ for $a_i \in A$ and $w_{O,j}(\tau)$ for $o_j \in O$ via $\mathcal{G}_{\mathsf{trust}}$ with hashes $h_{A,i,\sigma} = \mathsf{Hash}(w_{A,i}(\sigma))$ and $h_{O,j,\tau} = \mathsf{Hash}(w_{O,j}(\tau))$ (Section 2.4.1).
6: Generate s garbled circuits $GC_1, \ldots, GC_s$ and send $\{GC_j\}_{j=1}^s$ to verifier V.

Prover (Garbler): Algorithm $\mathsf{En}(f_e, x)$

7: Input cleartext $x \in \mathbb{Z}_n$ as binary $x[0], \ldots, x[d-1]$.
8: Compute garbled input $B_I(x) = (b_{A,i}(x[i]))_{i \in \mathbb{Z}_d}$, where $b_{A,i}(x[i]) = R_{A,i}\Delta^{x[i]}$ for each bit $x[i] \in \{0, 1\}$.
9: Send $B_I(x)$ and cleartext x to verifier.

Cut-and-Choose ZKP

10: V chooses random subset $J \subset [s]$ of size $s/2$ and sends J to P.
11: For each $j \in J$, P opens GC_j by sending keys and randomness to V.
12: V sends revealed labels to $\mathcal{G}_{\mathsf{trust}}$ for hash validation against $h_{A,i,\sigma}$ and $h_{O,j,\tau}$.
13: $\mathcal{G}_{\mathsf{trust}}$ verifies hashes; V aborts if any mismatch.
14: Prover returns GC_j for $j \notin J$ to V.

Verifier: Algorithm $\mathsf{Ev}(C, B_I(x))$ **and** $\mathsf{De}(f_d, Y)$

15: Use known circuit C computing F.
16: Receive $B_I(x)$ and cleartext x in binary form.
17: For each $j \notin J$, evaluate GC_j with $B_I(x)$ to compute garbled output $W_O^j(F(x)) = (w_{O,j}(F(x)_j))_{j \in \mathbb{Z}_m}$.
18: For each $j \notin J$, decode $W_O^j(F(x))$ using De by hashing components and comparing with $h_{O,j,\tau}$ via $\mathcal{G}_{\mathsf{trust}}$ to obtain cleartext y^j.
19: Check majority consistency across $\{y^j\}_{j \notin J}$; accept y if a majority agree, abort otherwise.
20: Output cleartext y if all validations pass.

Remark 5.28. We work in the $\mathcal{F}_{\mathsf{ZK}}$-hybrid model for zero-knowledge proofs: on common input a statement stmt from prover and verifier, and a witness w from the prover, the ideal functionality $\mathcal{F}_{\mathsf{ZK}}$ either returns accept to both parties (if (stmt, w) is in the NP relation) or reject to both, and leaks nothing else to the verifier. Any UC-secure

instantiation of such a ZK proof system can be plugged in, and UC composition then lifts our analysis in the $(\mathcal{G}_{\mathsf{trust}}, \mathcal{G}_{\mathsf{RO}}^{X}, \mathcal{F}_{\mathsf{ZK}})$-hybrid model to the fully concrete protocol.

Algorithm 18 Ideal Functionality $\mathcal{F}_{\mathsf{GC}}$: Garbling and Evaluation of Circuit C Computing F

Require: Security parameter κ, prime $p \approx 2^{\kappa}$, circuit C computing $F : \mathbb{Z}_n \to \mathbb{Z}_m$, input wire set $A = \{a_0, \ldots, a_{d-1}\}$, output wire set $O = \{o_0, \ldots, o_{m-1}\}$, adversary $\mathcal{A}$, environment $\mathcal{Z}$.

Ensure: Garbled input $B_I(x)$, garbled output $W_O(F(x))$, cleartext output y.

Initialization

Choose global correlation $\Delta \in \mathbb{Z}_p^*$ (secret) and nonces $R_{A,i}, R_{O,j} \in \mathbb{Z}_p^*$ for each $a_i \in A$, $o_j \in O$, known only to trusted party.

On input $x \in \mathbb{Z}_n$ from prover P (originating from the environment $\mathcal{Z}$) in binary form $x[0], \ldots, x[d-1]$:

Compute garbled input $B_I(x) = (b_{A,i}(x[i]))_{i \in \mathbb{Z}_d}$, where $b_{A,i}(x[i]) = R_{A,i}\Delta^{x[i]}$ for each bit $x[i] \in \{0, 1\}$.

Send $B_I(x)$ and cleartext x to verifier V.

On request from verifier V to evaluate F:

Compute garbled output $W_O(F(x)) = (w_{O,j}(F(x)_j))_{j \in \mathbb{Z}_m}$, where $w_{O,j}(F(x)_j) = R_{O,j}\Delta^{\delta_j(F(x))}$.

Decode $W_O(F(x))$ to cleartext y using stored nonces and Δ.

Send $W_O(F(x))$ and y to V.

On adaptive corruption of prover P by $\mathcal{A}$:

if prover P is corrupted **then**

    Reveal Δ, $\{R_{A,i}\}_{i \in \mathbb{Z}_d}$, and $\{R_{O,j}\}_{j \in \mathbb{Z}_m}$ to $\mathcal{A}$.

end if

We stress that $\mathcal{F}_{\mathsf{GC}}$ sends both the clear input x and the clear output $y = F(x)$ to the verifier V, in addition to the corresponding garbled output vector $W_O(F(x))$. This reflects our explicit design choice not to target input or output privacy (Remark 2.6): $\mathcal{F}_{\mathsf{GC}}$ only enforces correctness and authenticity of the reported output.

Lemma 5.29 (Ideal soundness of $\mathcal{F}_{\mathsf{GC}}$)**.** *Let $\mathcal{F}_{\mathsf{GC}}$ be the ideal functionality of Algorithm 18. In any ideal execution with honest evaluator V and arbitrary (possibly malicious) prover $P^{\star}$, the output y_V of V is either $\perp$ or the correct value $F(x)$. In particular, a malicious prover cannot cause V to accept an incorrect value $y_V \notin \{\perp, F(x)\}$.*

Proof. By inspection of Algorithm 18, $\mathcal{F}_{\mathsf{GC}}$ maintains an internal correlation Δ and nonce sets $\{R_{A,i}\}$, $\{R_{O,j}\}$, and on input x from the environment it always computes the output vector $W_O(F(x))$ according to the specification of F. The evaluator V only receives $W_O(F(x))$ (and possibly x), and no other value is ever delivered that could decode to a different output y_V. Hence, regardless of $P^{\star}$'s behaviour, V's output is either this prescribed value $F(x)$ or $\perp$ if the protocol aborts. □

Theorem 5.30 (UC Correctness of π_{GC})**.** *Let π_{GC} be the protocol for garbling and evaluating the circuit C computing $F : \mathbb{Z}_n \to \mathbb{Z}_m$ as defined in Algorithm 17, and let $\mathcal{F}_{\mathsf{GC}}$ be the corresponding ideal functionality as defined in Algorithm 18. Consider the*

Algorithm 19 Ideal-World Simulator $\mathsf{Sim}_{\mathsf{GC}}$, honest verifier

Require: : Security parameter κ, circuit C computing F, global functionalities $\mathcal{G}_{\mathsf{trust}}$ and $\mathcal{F}_{\mathsf{ZK}}$, ideal functionality $\mathcal{F}_{\mathsf{GC}}$, adversary $\mathcal{A}$ controlling the corrupt prover.

Ensure: : Simulated view indistinguishable from the real execution of π_{GC}.

Simulator $\mathsf{Sim}_{\mathsf{GC}}$ (emulates the honest verifier V)

1: Receive from $\mathcal{A}$ all messages that a corrupt prover would send to V in π_{GC}: commitments posted to $\mathcal{G}_{\mathsf{trust}}$, the s garbled circuits, and the claimed clear input x (and $B_I(x)$ if applicable).
2: Choose the cut-and-choose challenge subset $J \subset [s]$ uniformly at random with $|J| = s/2$, and send J to $\mathcal{A}$.
3: Upon receiving openings for circuits in J, forward the corresponding statement/witness to $\mathcal{F}_{\mathsf{ZK}}$ and query $\mathcal{G}_{\mathsf{trust}}$ as in Algorithm 2 to validate all revealed hashes. If any check fails, output $\perp$ for V and halt.
4: If all checks pass, send the clear input x to $\mathcal{F}_{\mathsf{GC}}$ and request evaluation.
5: Receive $(W_O(F(x)), y)$ from $\mathcal{F}_{\mathsf{GC}}$ and output y on behalf of V.
6: If $\mathcal{A}$ adaptively corrupts the prover after commitment, hand $\mathcal{A}$ the prover state that is consistent with the commitments it already posted (or abort if $\mathcal{A}$ requests an inconsistent opening).

$(\mathcal{G}_{\mathsf{trust}}, \mathcal{G}_{\mathsf{RO}}^{X}, \mathcal{F}_{\mathsf{ZK}})$-hybrid model with an honest evaluator V and a potentially malicious prover $P^\star$ (with adaptive corruption of P after the commitment phase, cf. Remark 5.27). For any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ controlling $P^\star$ and any PPT environment $\mathcal{Z}$, the real-world execution $\mathsf{REAL}_{\mathcal{A},\pi_{\mathsf{GC}}}(\mathcal{Z})$ is computationally indistinguishable from the ideal-world execution $\mathsf{IDEAL}_{\mathcal{F}_{\mathsf{GC}},\mathcal{F}_{\mathsf{GC}}}(\mathcal{Z})$, where $\mathcal{F}_{\mathsf{GC}}$ is the simulator defined in Algorithm 19. The distinguishing advantage $\mathsf{Adv}_{\mathcal{Z}}(\kappa)$ is negligible in the security parameter κ. In particular, π_{GC} securely realizes $\mathcal{F}_{\mathsf{GC}}$ with UC correctness in the honest-verifier, corrupt-prover scenario.

Proof. Fix any PPT adversary $\mathcal{A}$ controlling the prover $P^\star$ and any PPT environment $\mathcal{Z}$. We prove that

$$\mathsf{REAL}_{\mathcal{A},\pi_{\mathsf{GC}}}(\mathcal{Z}) \approx_c \mathsf{IDEAL}_{\mathsf{Sim}_{\mathsf{GC}},\mathcal{F}_{\mathsf{GC}}}(\mathcal{Z})$$

in the $(\mathcal{G}_{\mathsf{trust}}, \mathcal{G}_{\mathsf{RO}}^{X}, \mathcal{F}_{\mathsf{ZK}})$-hybrid model, where $\mathsf{Sim}_{\mathsf{GC}}$ is Algorithm 19 (emulating the honest verifier V). We define hybrids H_0, H_1, H_2:

- H_0 (real execution): This is the real execution $\mathsf{REAL}_{\mathcal{A},\pi_{\mathsf{GC}}}(\mathcal{Z})$ with honest verifier V running π_{GC} and prover controlled by $\mathcal{A}$.
- H_1 (output switch to $\mathcal{F}_{\mathsf{GC}}$ upon acceptance): This hybrid is identical to H_0 up to and including: (i) the prover's messages, (ii) the verifier's sampling of the cut-and-choose challenge subset $J \subseteq [s]$ of size $|J| = s/2$, and (iii) all verifier-side validation checks (using $\mathcal{G}_{\mathsf{trust}}$ for commitment consistency and $\mathcal{F}_{\mathsf{ZK}}$ for the cut-and-choose/ZK check). If the verifier aborts in H_0, then H_1 aborts as well and outputs $\perp$. Otherwise, H_1 does *not* compute the output by locally evaluating and decoding the unopened garbled circuits. Instead, it sends the clear input x (as received in the transcript) to $\mathcal{F}_{\mathsf{GC}}$ as the prover's input, requests evaluation, receives $(WO(F(x)), y)$ from $\mathcal{F}_{\mathsf{GC}}$, and outputs y.

- H_2 (ideal execution): This is the ideal execution $\mathsf{IDEAL}_{\mathsf{Sim}_{\mathsf{GC}},\mathcal{F}_{\mathsf{GC}}}(\mathcal{Z})$, where $\mathsf{Sim}_{\mathsf{GC}}$ emulates the honest verifier: it interacts with $\mathcal{A}$ exactly as V would in π_{GC}, samples and sends J, performs the same checks via $\mathcal{G}_{\mathsf{trust}}$ and $\mathcal{F}_{\mathsf{ZK}}$, and if the checks pass sends the same x to $\mathcal{F}_{\mathsf{GC}}$ and outputs the resulting y.

We show $H_0 \approx_c H_1$ and $H_1 \equiv H_2$. Indeed:
Step 1: $H_1 \equiv H_2$**.** By construction, the joint distribution of views in H_1 and H_2 is identical: in both hybrids the verifier-side logic samples the same distribution of J, applies the same accept/abort checks using the same ideal functionalities $\mathcal{G}_{\mathsf{trust}}$ and $\mathcal{F}_{\mathsf{ZK}}$, and upon acceptance sends the same x to $\mathcal{F}_{\mathsf{GC}}$ and outputs the same y returned by $\mathcal{F}_{\mathsf{GC}}$. Hence $H_1 \equiv H_2$.
Step 2: Bounding the distance between H_0 **and** H_1**.** The hybrids H_0 and H_1 are identical except for the verifier output in the case where the verifier does *not* abort. Define the event

$$\mathsf{IncorrectOutput} := \{\text{the verifier does not abort and outputs } y \neq F(x) \text{ in } H_0\}.$$

In H_1, whenever the verifier does not abort, it outputs $y = F(x)$ by inspection of $\mathcal{F}_{\mathsf{GC}}$. Therefore, for any environment $\mathcal{Z}$,

$$|\Pr[H_0(\kappa) = 1] - \Pr[H_1(\kappa) = 1]| \leq \Pr[\mathsf{IncorrectOutput}].$$

It remains to upper bound $\Pr[\mathsf{IncorrectOutput}]$.
Step 3: Cut-and-choose soundness bound. Let $R := [s] \setminus J$ be the unopened set, so $|R| = s/2$. Let

$$t := \left\lfloor \frac{|R|}{2} \right\rfloor + 1 = \left\lfloor \frac{s}{4} \right\rfloor + 1$$

be the strict majority threshold among the circuits in R.
Let Coll be the event that the verifier's commitment validation fails to be binding. [1]
Since we are in the $\mathcal{F}_{\mathsf{ZK}}$-hybrid model, verification is perfectly sound: any opened circuit inconsistent with the recorded commitments and the claimed relation causes the verifier to abort.
Condition on $\neg\mathsf{Coll}$. If $\mathsf{IncorrectOutput}$ occurs, then the verifier did not abort and the majority decoding over the circuits in R produced an incorrect value $\hat{y} \neq F(x)$. This implies that there exist at least t indices in R whose circuits decode to $\hat{y}$ while being consistent with the posted commitments. Fix any such set $B \subseteq [s]$ of size $|B| = t$. For the verifier not to abort under $\neg\mathsf{Coll}$, none of the indices in B can be opened (otherwise an opened inconsistent/bad circuit would be detected by the checks). Thus,

$$\mathsf{IncorrectOutput} \wedge \neg\mathsf{Coll} \implies B \subseteq R.$$

Since J is uniform among all subsets of size $s/2$, for any fixed B of size t we have

$$\Pr[B \subseteq R] = \frac{\binom{s-t}{(s/2)-t}}{\binom{s}{s/2}} = \prod_{i=0}^{t-1} \frac{(s/2) - i}{s - i} \leq \left(\frac{1}{2}\right)^t.$$

Therefore,

[1] Under the random-oracle model for Hash (or the binding assumption stated for Hash), $\Pr[\mathsf{Coll}]$ is negligible in κ.

$$\begin{aligned}\Pr[\mathsf{IncorrectOutput}] \leq \\ \Pr[\mathsf{IncorrectOutput} \wedge \neg\mathsf{Coll}] + \Pr[\mathsf{Coll}] \leq \\ 2^{-t} + \mathsf{negl}(\kappa) \leq 2^{-(\lfloor s/4 \rfloor + 1)} + \mathsf{negl}(\kappa).\end{aligned}$$

In particular, choosing $s = s(\kappa)$ such that $2^{-s/4} = \mathsf{negl}(\kappa)$ makes the probability $\Pr[\mathsf{IncorrectOutput}]$ negligible.

Step 4: Conclusion. Combining the above and using $H_1 \equiv H_2$,

$$|\Pr[H_0(\kappa) = 1] - \Pr[H_2(\kappa) = 1]| \leq 2^{-(\lfloor s/4 \rfloor + 1)} + \mathsf{negl}(\kappa),$$

so $H_0 \approx_c H_2$. Hence π_{GC} securely realizes $\mathcal{F}_{\mathsf{GC}}$.

5.3.1. *Adaptive corruption (post-commitment).* In this theorem, the adversary $\mathcal{A}$ already controls the prover $P^\star$ throughout the execution, so "corrupting the prover" reveals no additional information beyond the adversary's own internal state. If the environment is restricted to issuing corruption queries only after the prover has posted the commitments and circuits (as discussed in the paper), then these posted values are already fixed by $\mathcal{G}_{\mathsf{trust}}$ in both the real and ideal executions, and the simulator's behaviour is unchanged. □

Corollary 5.31. *In the honest-evaluator, malicious-prover setting, protocol π_{GC} satisfies soundness in the sense of Definition 2.5: for every PPT prover $P^\star$ and every environment $\mathcal{Z}$, the output y_V of the honest evaluator V in a real execution of π_{GC} is either $\perp$ or $F(x)$, except with negligible probability.*

Proof. By Theorem 5.30, for every PPT adversary $\mathcal{A}$ and environment $\mathcal{Z}$, the real-world execution of π_{GC} is computationally indistinguishable from the ideal execution with $\mathcal{F}_{\mathsf{GC}}$. By Lemma 5.29, in the ideal world V's output is always in $\{\perp, F(x)\}$. If a PPT prover $P^\star$ could cause V to output a value $y_V \notin \{\perp, F(x)\}$ with non-negligible probability in the real world, we could use $P^\star$ and $\mathcal{Z}$ to distinguish the real and ideal executions with non-negligible advantage, contradicting Theorem 5.30. □

Part 2. Blockchain applications

We now give a high-level description of the implementation of a garbling in a multi-party protocol.

5.4. **Integration with the blockchain.** We omit most of the fine detail involving how to garble the wires and compute the gate functionalities. This description corresponds to Figure 1 below. The claim of the prover is that he holds an input a such that $F(a) = c$. Let the circuit C compute the functionality F. The data F, C and c are known by both parties. The prover claims knowledge of a such that the public circuit C (computing F) yields $F(a) = c$.

We want payments to be constrained by the result of computations. We use the disputable computing paradigm, also known as optimistic computing, where the computation is assumed correct if the verifier cannot submit a fraud-proof during a challenge time window. The verifier locally runs the computation on the circuit with a public input, and if the circuit correctness check ($F(a) = c$) fails, the circuit discloses a hidden secret that can be used to prove fraud to a Bitcoin smart contract. In the Bitcoin

blockchain, funds are always locked in UTXOs (Unspent Transaction Outputs). Each UTXO holds funds, like a locked prize box, using a smart contract model based on simple predicates that can disallow spending before a future time (time-locks) or allow spending when presented with certain hash pre-image (hash-locks). Both constrains can be combined in what is know as HTLCs (Hash-time lock contracts).
When a party cheats and submits and invalid input a, the circuit discloses a secret that penalizes that party. Additionally, and to to keep verifiers incentivized to watch for fraud, the verifier can usually claim locked funds as a reward for detecting the cheater. A spending timeout (hashlock) rewards the prover for truthfulness and forces a timely resolution of the protocol. This protocol enables two-party trustless blockchain transactions based on off-chain computations, ensuring computations are correct without needing a trusted third party, as long a the verifier is live and has the funds to pay the required transaction fees.

5.5. **Realizing $\mathcal{G}_{\mathsf{trust}}$ via Bitcoin UTXOs.** The abstract bulletin board functionality $\mathcal{G}_{\mathsf{trust}}$ (Algorithm 5) can be instantiated using Bitcoin's UTXO model. This subsection describes the concrete implementation $\Pi_{\mathcal{L}}$ and proves it realizes $\mathcal{G}_{\mathsf{trust}}$. The blockchain realization of $\mathcal{G}_{\mathsf{trust}}$ introduces timing considerations absent from the ideal functionality. We formalize these via the following definitions.

Definition 5.32 (Confirmation time)**.** Let $\mathcal{L}$ be a ledger (blockchain) and let tx be a transaction. The *confirmation time* $\mathsf{conf}_{\mathcal{L}}(\mathsf{tx})$ is the earliest time t at which tx has received at least k_{conf} confirmations on $\mathcal{L}$, where $k_{\mathsf{conf}} \geq 1$ is a security parameter (e.g., $k_{\mathsf{conf}} = 6$ for Bitcoin). If tx is never confirmed, we set $\mathsf{conf}_{\mathcal{L}}(\mathsf{tx}) = \infty$.

Definition 5.33 (Ledger synchronization)**.** We say that parties are δ-*synchronized* with respect to ledger $\mathcal{L}$ if, for any transaction tx and any two honest parties P_1, P_2:

$$|\mathsf{view}_{\mathcal{L}}^{P_1}(t) - \mathsf{view}_{\mathcal{L}}^{P_2}(t)| \leq \delta$$

where $\mathsf{view}_{\mathcal{L}}^{P}(t)$ denotes P's local view of the ledger at time t, and the difference is measured in block height. This captures network delay and temporary chain disagreements.

Definition 5.34 (Ledger properties)**.** We assume the underlying ledger $\mathcal{L}$ satisfies the following standard properties from the blockchain literature [**?**, **?**]:

(1) **Persistence:** Once a transaction tx is confirmed in an honest party's view at depth k_{conf}, it remains in all honest parties' views at the same position, except with probability $\mathsf{negl}(\kappa)$.
(2) **Liveness:** If an honest party broadcasts a valid transaction tx at time t, then tx is included in all honest parties' views by time $t + \Delta_{\mathsf{live}}$, except with probability $\mathsf{negl}(\kappa)$, where Δ_{live} is a known upper bound.
(3) **Chain quality:** In any window of ℓ consecutive blocks in an honest party's view, at least $\mu \cdot \ell$ blocks were mined by honest parties, for some constant $\mu > 0$.

Definition 5.35 (Confirmation event)**.** For a session sid and key key, we define the event $\mathsf{Confirmed}(\mathsf{sid}, \mathsf{key}, t)$ to hold at time t if and only if there exists a transaction tx encoding $(\mathsf{sid}, \mathsf{key}, \mathsf{val})$ for some val, such that $\mathsf{conf}_{\mathcal{L}}(\mathsf{tx}) \leq t$.

Definition 5.36 (Query-after-confirmation)**.** We say an environment $\mathcal{Z}$ is *query-after-confirmation compliant* if, for every $\mathsf{Get}(\mathsf{sid}, \mathsf{key})$ query issued by $\mathcal{Z}$ or by an honest party at time t, either:

(a) No honest party has issued $\mathsf{Put}(\mathsf{sid}, \mathsf{key}, \cdot)$ prior to time t, or
(b) $\mathsf{Confirmed}(\mathsf{sid}, \mathsf{key}, t - \delta)$ holds for some $\delta \geq \Delta_{\mathsf{sync}}$, where $\Delta_{\mathsf{sync}} := k_{\mathsf{conf}} \cdot \mathbb{E}[\text{block time}] + \Delta_{\mathsf{live}}$ is the synchronization delay.

This ensures that queries only occur after the ledger state has stabilized.

Let $\mathcal{L}$ be a ledger satisfying Definition 5.34 with parameters $(k_{\mathsf{conf}}, \Delta_{\mathsf{live}}, \mu)$. Consider the following UTXO-based implementation $\Pi_{\mathcal{L}}$ of $\mathcal{G}_{\mathsf{trust}}$:

Data encoding: Each commitment is encoded in a transaction $\mathsf{TX}_{\mathsf{commit}}$ with:

- Inputs: One or more UTXOs controlled by the committing party P, providing funds for transaction fees.
- Outputs: A commitment output encoding the tuple $(\mathsf{sid}, \mathsf{key}, \mathsf{val})$ via one of:
  (1) OP_RETURN output containing $\mathsf{Encode}(\mathsf{sid}, \mathsf{key}, \mathsf{val})$, or
  (2) Pay-to-Contract commitment in the output's public key, or
  (3) Taproot witness commitment (SegWit v1).
- Authentication: The transaction is signed by P, binding the commitment to P's identity.

State extraction: For session sid, define:

$$\mathsf{TXO}^P_{\mathsf{sid}}(t) := \left\{u \in \mathcal{L}^P(t) : u \text{ encodes } (\mathsf{sid}, \cdot, \cdot) \text{ with } \geq k_{\mathsf{conf}} \text{ confirmations}\right\}$$

where $\mathcal{L}^P(t)$ is party P's local ledger view at time t.
For each key key, if there exist outputs in $\mathsf{TXO}^P_{\mathsf{sid}}(t)$ encoding $(\mathsf{sid}, \mathsf{key}, \cdot)$:

- Let $\mathsf{tx}^*(P, t, \mathsf{key})$ be the transaction with the earliest block height (ties broken by lexicographic txid ordering).
- Define $T^P_{\mathsf{sid}}(\mathsf{key}, t) := \mathsf{val}$ where val is the value encoded in tx^*.

Otherwise, set $T^P_{\mathsf{sid}}(\mathsf{key}, t) := \bot$.

Protocol operations:

(1) $\mathsf{Put}(\mathsf{sid}, \mathsf{key}, \mathsf{val})$ by party P:
  (a) Verify $\mathsf{Valid}(\mathsf{sid}, \mathsf{key})$ and $\mathsf{owner}(\mathsf{key}) = \mathsf{id}(P)$; abort if either fails.
  (b) Check $T^P_{\mathsf{sid}}(\mathsf{key}, t_{\mathsf{now}}) = \bot$; if not, return ALREADY_SET.
  (c) Construct and broadcast $\mathsf{TX}_{\mathsf{commit}}$ encoding $(\mathsf{sid}, \mathsf{key}, \mathsf{val})$.
  (d) Wait for k_{conf} confirmations; return $(\mathsf{Ack}, \mathsf{sid}, \mathsf{key}, \mathsf{block\_height})$.
(2) $\mathsf{Get}(\mathsf{sid}, \mathsf{key})$ by any party P:
  (a) Compute $\mathsf{val} := T^P_{\mathsf{sid}}(\mathsf{key}, t_{\mathsf{now}})$.
  (b) If $\mathsf{val} \neq \bot$, return $(\mathsf{Value}, \mathsf{sid}, \mathsf{key}, \mathsf{val})$; else return $(\mathsf{Value}, \mathsf{sid}, \mathsf{key}, \bot)$.
(3) $\mathsf{List}(\mathsf{sid})$ by any party P: Return $\{(\mathsf{key}, T^P_{\mathsf{sid}}(\mathsf{key}, t_{\mathsf{now}})) : T^P_{\mathsf{sid}}(\mathsf{key}, t_{\mathsf{now}}) \neq \bot\}$.

Lemma 5.37 (Realization of $\mathcal{G}_{\mathsf{trust}}$ by a UTXO-based ledger)**.** *For any query-after-confirmation environment $\mathcal{Z}$ (Definition 5.36), and any set of honest parties $\mathcal{H}$, the joint distribution of inputs and outputs of parties in $\mathcal{H}$ under $\Pi_{\mathcal{L}}$ is statistically indistinguishable from their distribution in an ideal execution with $\mathcal{G}_{\mathsf{trust}}$ (Algorithm 5):*

$$\left|\Pr\left[\mathsf{REAL}_{\Pi_{\mathcal{L}}, \mathcal{A}, \mathcal{Z}}(\kappa) = 1\right] - \Pr\left[\mathsf{IDEAL}_{\mathcal{G}_{\mathsf{trust}}, \mathcal{S}, \mathcal{Z}}(\kappa) = 1\right]\right| \leq \mathsf{negl}(\kappa)$$

where $\mathcal{S}$ is the canonical simulator that forwards all operations to $\mathcal{G}_{\mathsf{trust}}$.

Proof. We verify that $\Pi_{\mathcal{L}}$ correctly implements each operation of $\mathcal{G}_{\mathsf{trust}}$.

(1) Put operation. The validity check (step 1a) and ownership check (step 1b) in $\Pi_{\mathcal{L}}$ directly mirror steps 2(a)–2(b) of Algorithm 5, producing identical accept/reject decisions. For first-write-wins: if two transactions $\mathsf{tx}_1, \mathsf{tx}_2$ both encode $(\mathsf{sid}, \mathsf{key}, \cdot)$, the state extraction rule selects the one with earlier block height (ties broken lexicographically by txid), ensuring all honest parties agree on a unique first writer. By Definition 5.34, once confirmed, this transaction remains in all honest views except with probability $\mathsf{negl}(\kappa)$.
(2) Get and List operations. By Definition 5.36, queries occur only after relevant Put operations have been confirmed for at least Δ_{sync} time. Combined with Definition 5.34 and Definition 5.33 (δ-synchronization), all honest parties compute identical values for $T_{\mathsf{sid}}(\mathsf{key})$, matching $\mathcal{G}_{\mathsf{trust}}$.
(3) Adversarial writes. Transaction authentication (signatures) ensures that only party P with $\mathsf{id}(P) = \mathsf{owner}(\mathsf{key})$ can create valid commitments for key, matching the ownership check in $\mathcal{G}_{\mathsf{trust}}$.

The only discrepancy, confirmation delay, is masked by query-after-confirmation compliance. Defining $\mathsf{LedgerMismatch}$ as the event that any honest party's query returns a different value in $\Pi_{\mathcal{L}}$ versus $\mathcal{G}_{\mathsf{trust}}$, we have $\Pr[\mathsf{LedgerMismatch}] \leq \mathsf{negl}(\kappa)$ by persistence. Conditioned on $\neg\mathsf{LedgerMismatch}$, the distributions are identical. □

5.6. **Blockchain protocol.** Before describing the protocol, we note that some wires in the circuit are arithmetic, others are one-hot, and others are binary. The descriptions of these and the most relevant gate functionalities are given in Chapter 4.

(i) The prover garbles the input wires w_i ($i \in I$) of circuit C.
(ii) The hashes $h_{i,0} = \mathsf{Hash}(w_i(0)), h_{i,1} = \mathsf{Hash}(w_i(1))$ are sent to the verifier. These hashes commit the prover to the wire labels before sending the circuit, preventing adaptive attacks.
(iii) All other garbled wires are created consistently by the prover, until the final Boolean output that we call verification wire w_v (see Section 5.7).
(iv) The prover sends to the verifier, for each gate $\mathcal{G}_k$ that is not free, one cyphertext (the *help* H_k), computed as specified for each gate type in Chapter 4.
(v) For the output wire, we associate $z = 1$ the value true and $z = 0$ the value false. The hash $h_f = \mathsf{Hash}(w_v(0))$ is sent to the verifier. The hash h_f is a fingerprint of an incorrect output, used later to catch a false claim.
(vi) The garbled circuit C is sent to the verifier (see Section 2.2 for details).
(vii) A ZKP is constructed by the prover, showing the verifier that: the garbled circuit C indeed computes the agreed circuit F, that the hashes $h_{i,z}$ correspond to $w_i(z)$ for $z = \{0, 1\}$ and that the hash h_f corresponds to $w_v(0)$.
(viii) Once these ZKPs convince the verifier, both parties co-sign a setup transaction $\mathsf{TX}_{\mathsf{setup}}$ that creates four UTXOs: UTXO_0, UTXO_1, UTXO_2, UTXO_3, each locked with the locking scripts described below. The setup transaction is broadcast to the Bitcoin network. UTXO_0 is an output of $\mathsf{TX}_{\mathsf{setup}}$ with the following locking script conditions:
- It has a relative time-lock t_0 (implemented via `OP_CSV`) that begins when UTXO_0 is confirmed on-chain. This time-lock is measured in blocks.
- UTXO_0's locking script can be satisfied by the prover providing witness data consisting of:

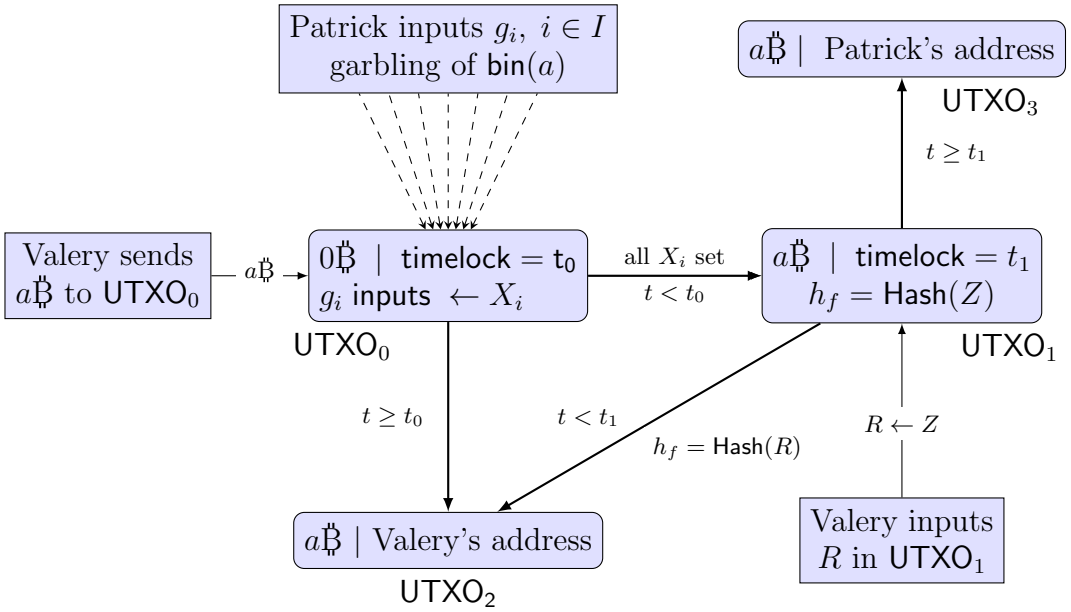

FIGURE 1. Mechanism for a ZKP with on-chain commitments. Conditions on both sides of an arrow should be understood as AND. The hash h_f corresponds to the output false of the final verifying gate of the garbled circuit C computing F, which checked wether $F(a) = c$.

  - The garbled input labels $g_i = w_i(z_i)$ for each input wire $i \in \mathcal{I}$, where $z_i \in \{0, 1\}$ and $z_i = a[i]$
  - Satisfaction requires that $\mathsf{Hash}(g_i) = h_{i,z_i}$ for all i, where h_{i,z_i} are the hash commitments made in step (ii)
- UTXO_0 has an alternative spending path: if t_0 blocks pass without the prover spending UTXO_0, the verifier can spend it using her own signature, recovering the locked funds.
- The prover can spend UTXO_0 by broadcasting transaction TX_0, which must satisfy:
  - Inputs: Consumes UTXO_0 with witness data containing garbled inputs $\{g_i\}_{i \in \mathcal{I}}$
  - Outputs: Creates exactly one output UTXO_1 carrying the full value from UTXO_0 (minus transaction fees)
  - The output script of UTXO_1 is enforced by a covenant construction or by pre-signing: during setup, both parties co-sign TX_0 with a specific template for UTXO_1, ensuring the prover cannot create alternate outputs

(ix) UTXO_1 (created when TX_0 is confirmed) has a locking script with two alternative spending conditions (implemented via `OP_IF/OP_ELSE`):

(a) **Prover success path**: After relative time-lock t_1 (implemented via `OP_CSV`, for instance, $t_1 = 144$ blocks) measured from UTXO_1's confirmation, if no valid challenge has been submitted, the prover can spend UTXO_1 by broadcasting transaction TX_2 that:

- Inputs: Consumes UTXO_1 with prover's signature

- Outputs: Creates UTXO_3 paying to the prover's address
- This represents successful proof verification by timeout

(b) **Verifier challenge path**: At any time before timeout t_1, the verifier can spend UTXO_1 by broadcasting transaction TX_1 that:

- Inputs: Consumes UTXO_1 with witness data containing nonce R
- Script validation: Checks that $\mathsf{Hash}(R) = h_f$, where h_f is the commitment to the "false" output wire from step (v)
- Outputs: Creates UTXO_2 paying to the verifier's address
- This represents catching a fraudulent proof

(x) The prover broadcasts transaction TX_0 that spends UTXO_0 by providing garbled inputs g_i as witness data. Once TX_0 is confirmed on-chain:

- The witness data $\{g_i\}$ becomes publicly visible in the blockchain
- The verifier retrieves $\{g_i\}$ from TX_0's witness
- The verifier computes $\mathsf{Hash}(g_i)$ and compares with the committed values $h_{i,z}$ for $z \in \{0, 1\}$
- For each wire i, exactly one of $\{\mathsf{Hash}(g_i) = h_{i,0}\}$ or $\{\mathsf{Hash}(g_i) = h_{i,1}\}$ holds, revealing bit $z_i = a[i]$
- The verifier thereby learns the cleartext input $a = \sum_{i=0}^{d-1} z_i \cdot 2^i$

Sharing a after commitment is intentional to simplify verification, as the goal is to ensure correctness and authenticity, not hide inputs.

(xi) The verifier now performs off-chain computation:

- Using the garbled inputs $\{g_i\}$ retrieved from TX_0 and cleartext a
- Evaluates all gates of garbled circuit C topologically using the helps $\{H_k\}$ and generators provided in the garbling material
- Computes the garbled output of the verification gate (Section 5.7), obtaining nonce $R = w_v(z)$ where $z \in \{0, 1\}$ is the verification bit
- If $z = 0$ (computation failed), R equals the committed nonce R_f satisfying $\mathsf{Hash}(R) = h_f$
- If $z = 1$ (computation succeeded), $R \neq R_f$

This computation is performed locally by the verifier; no blockchain interaction is required.

(xii) If the verifier determines that $\mathsf{Hash}(R) = h_f$ (that is, the computation returned false), she broadcasts transaction TX_1 that:

- Inputs: Spends UTXO_1 before timeout t_1
- Witness: Contains nonce R as challenge evidence
- Script execution: UTXO_1's locking script validates that $\mathsf{Hash}(R) = h_f$ (using `OP_SHA256` and `OP_EQUAL`)
- Outputs: Creates UTXO_2 paying to the verifier's address
- Effect: The hash equality proves the prover's claim was incorrect, and the verifier successfully challenges the proof, claiming the locked funds as reward

Once TX_1 is confirmed, UTXO_1 is consumed, and the protocol terminates with verifier success.

(xiii) If the verifier does not broadcast a valid challenge transaction TX_1 before timeout t_1 expires:

- The prover broadcasts TX_2 spending UTXO_1 via the time-lock path

- TX_2 creates UTXO_3 paying to the prover's address
- Effect: The protocol terminates with prover success (no successful challenge implies the proof was correct)

The verifier's failure to challenge can result from either: (1) the computation genuinely succeeded ($F(a) = c$), or (2) verifier was offline. Rational verifiers monitor actively to claim rewards for catching fraud.

(xiv) The protocol satisfies *on-chain fairness* (Definition 5.24) against a malicious prover through the following enforcement mechanism:

- If $F(a) \neq c$ (invalid computation), the verifier computes $R = R_f$ satisfying $\mathsf{Hash}(R_f) = h_f$
- The verifier can then spend UTXO_1 via transaction TX_1 at any time before timeout t_1
- Conversely, the prover can only collect funds via TX_2 *after* timeout t_1
- Therefore, a cheating prover who submits invalid a cannot collect funds unless either:
  - (a) The verifier fails to challenge (offline/unavailable), or
  - (b) The prover forges $R' \neq R_f$ with $\mathsf{Hash}(R') = h_f$ (breaks hash preimage resistance), or
  - (c) The prover forges a valid garbled output for incorrect computation (breaks authenticity, Theorem 5.15)
- Under cryptographic assumptions (collision resistance of Hash, ECDLP), options (2) and (3) have negligible probability
- Thus, assuming verifier liveness, dishonest provers cannot successfully claim payouts

Remark 5.38. Since our garbling scheme involves modular arithmetic in $\mathbb{Z}_p$ and operations on an elliptic curve $E(\mathbb{Z}_p)$, we will replace the usual super-index notation for the garbling of a wire w_i^0, w_i^1 (wire i is off or on) with $w_i(z)$ for $z \in \{0, 1\}$. With this we avoid notation collisions with powers and exponentiation in $\mathbb{Z}_p$.

5.7. **The verification function.** This gate is used in our case of use for the blockchain, see Section 5.4. Its input wires W_C are the output wires of the circuit C computing functionality F. The value c is the one such that the garbler/prover claims to have a pre-image, that is circuit C evaluated in the prover inputs a_i, is claimed to be equal to c.

Definition 5.39. For fixed $c \in \mathbb{Z}_m$, consider the boolean verification function $V : \mathbb{Z}_d \to \mathbb{Z}_2$, with $V(x) = x - c + 1 \pmod 2$. The gate computing V has a one-hot input array of wires W_C, and a Boolean output wire w_v, $v \in \mathcal{W}$. We say that $V(x)$ is false if the output bit equals to 0, or equivalently, if $x \neq c$.
If $R_{C,i}$, $i \in \mathbb{Z}_m$ are the nonces of the input wires, then we set $R_f = R_{C,c}$ as the nonce for the output wire.

Note that $\breve{V}(W_C(c')) = w_v(V(c')) = R_f$ if and only if $c' \neq c \pmod d$ if and only if $z := V(c') = 0 \pmod 2$, otherwise it equals $R_f\Delta$. The hash of the output corresponding to false, $\mathsf{Hash}(R_f)$ is embedded in an UTXO by the prover, with a script that compares it with the hash of any other nonce r that the evaluator can input in this UTXO. A match of these two hashes is proof that the claim of the prover was false.

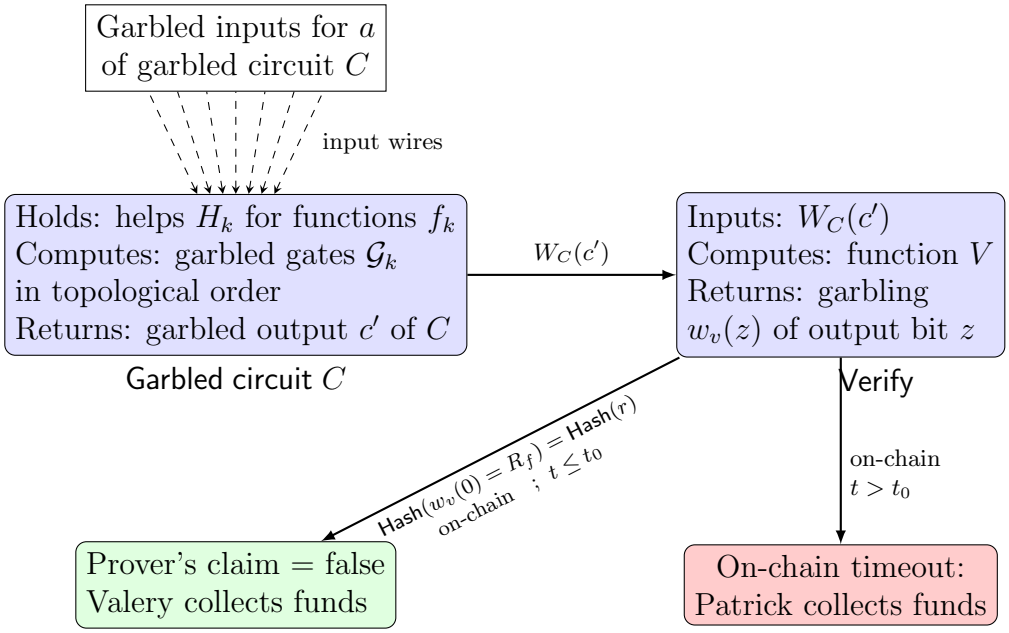

FIGURE 2. Circuit/verification module, each gate $\mathcal{G}_k$ of circuit C computes function f_k. Each help H_k is specific for gate $\mathcal{G}_k$. Garbled output bit $z = 0$ is proof that the prover claim $F(a) = c$ is false.

REFERENCES

[1] A. Afshar; P. Mohassel; B. Pinkas; M. Riva. *Non-interactive secure computation based on cut-and-choose*. In Advances in Cryptology–EUROCRYPT 2014, pages 387-404, Springer, 2014.

[2] M. Andrychowicz; S. Dziembowski; D. Malinowski; L. Mazurek. *Secure multiparty computations on Bitcoin*. 2014 IEEE Symposium on Security and Privacy, 443–458, IEEE, 2014. https://eprint.iacr.org/2013/784.

[3] Axiom Team,
Halo2-lib BN254 pairing chip and verifier benchmarks,
GitHub repository (v0.3.2), 2024.
https://github.com/axiom-crypto/halo2-lib.

[4] F. Barbara. *Cost estimator for emulated BN254 pairings* (Python),
GitHub repository, 2025.
https://github.com/disnocen/garbriel_bilinear.

[5] C. Baum. *On garbling schemes with and without privacy*. In: Security and Cryptography for Networks (SCN 2016), Lecture Notes in Comput. Sci. 9841, pp. 468–485, Springer, Cham (2016).

[6] M. Bellare, V. T. Hoang, P. Rogaway. *Foundations of garbled circuits*. CCS'12: Proceedings of the 2012 ACM conference on Computer and communications security, pages 784–796 https://eprint.iacr.org/2012/265.pdf

[7] M. Bellare; V. T. Hoang; P. Rogaway. *Adaptively secure garbling with applications to one-time programs and secure outsourcing*. IACR Cryptol. ePrint Arch. 2012/323, 38 pp. (2012).

[8] O. Biçer, Osman; A. Ajorian. *AuthOr: Lower cost authenticity-oriented garbling for arbitrary Boolean circuits*. arXiv:2501.18387 [cs.CR], 33 pp. (2025) https://arxiv.org/html/2501.18387

[9] D. A. Burgess. *On Dirichlet characters of polynomials*. Proc. London Math. Soc. (3), 13:537–548, 1963.

[10] R. Canetti. *Universally composable security*. J. ACM 67 (2020), no. 5, Art. 29, 94 pp.

[11] R. Canetti; Y. Dodis; R. Pass; S. Walfish. *Universally composable security with global setup*. In: Theory of Cryptography (TCC 2007), Lecture Notes in Comput. Sci. 4392, pp. 61–85, Springer, Berlin, Heidelberg (2007). https://eprint.iacr.org/2006/432.

[12] ChainwayXYZ,
Garbled SNARK verifier: Streaming garbled circuit for Groth16 over BN254,
GitHub repository, 2025.
https://github.com/chainwayxyz/garbled-snark-verifier.

[13] ConsenSys,
gnark BN254 Groth16 verifier (emulated field mode),
GitHub repository (v0.10.0), 2024, Zenodo DOI: 10.5281/zenodo.11034183.
https://github.com/ConsenSys/gnark.

[14] B. Donald, S. Micali, P. Rogaway. *The round complexity of secure protocols (extended abstract)*. In 22nd ACM STOC, pages 503–513. ACM Press, 1990 https://www.cs.ucdavis.edu/rogaway/papers/bmr90

[15] Y. El Housni and V. Iovino,
Optimized emulated pairing circuits for ZK-SNARKs,
in: *Applied Cryptography and Network Security (ACNS 2023)*,
Lecture Notes in Comput. Sci., vol. 13905, Springer, 2023, pp. 1–20.
ePrint https://eprint.iacr.org/2022/1234.

[16] T. K. Frederiksen; J. B. Nielsen; C. Orlandi. *Privacy-free garbled circuits with applications to efficient zero-knowledge*. In: Advances in Cryptology – EUROCRYPT 2015, Lecture Notes in Comput. Sci. 9057, pp. 191–219, Springer, Cham (2015).

[17] G. R. Grimmett, D. R. Stirzaker. *Probability and random processes*. Third edition. Oxford University Press, Oxford, 2001.

[18] N. Gürel. *Extracting bits from coordinates of a point of an elliptic curve*. Cryptology ePrint Archive, Report 2005/324, 2005. 9 pp. https://eprint.iacr.org/2005/324.

[19] D. Hankerson; A. Menezes, S. Vanstone. *Guide to elliptic curve cryptography*. Springer Professional Computing. Springer-Verlag, New York, 2004.

[20] D. Heath *Efficient arithmetic in garbled circuits*. Advances in cryptology—EUROCRYPT 2024. Part V, 3–31, Lecture Notes in Comput. Sci., 14655, Springer, Cham, 2024 https://eprint.iacr.org/2024/139.pdf.

[21] D. Heath, V. Kolesnikov. *One hot garbling*. In Giovanni Vigna and Elaine Shi, editors, ACM CCS 2021, pages 574–593. ACM Press, 2021 https://eprint.iacr.org/2022/798

[22] Y. Huang, J. Katz, and D. Evans. Efficient secure two-party computation using symmetric cut-and-choose. In Advances in Cryptology–CRYPTO 2013, pages 18-35, Springer, 2013.

[23] V. Kolesnikov, T. Schneider. *Improved garbled circuit: Free XOR gates and applications*. ICALP 2008, Part II, volume 5126 of LNCS, pages 486–498. Springer, Heidelberg, July 2008. https://www.cs.toronto.edu/ vlad/papers/XOR_ICALP08.pdf Kosba et al.

[24] J. Groth. *On the size of pairing-based non-interactive arguments*. In Annual international conference on the theory and applications of cryptographic techniques (pp. 305-326). Berlin, Heidelberg: Springer Berlin Heidelberg.

[25] A. Kosba; S. Miller; E. Shi; Z. Wen; C. Papamanthou. *Hawk: The blockchain model of cryptography and privacy-preserving smart contracts*. 2016 IEEE Symposium on Security and Privacy, 839–858, IEEE, 2016. https://eprint.iacr.org/2015/675.

[26] Y. Lindell, B. Pinkas, Benny. *A proof of security of Yao's protocol for two-party computation*. J. Cryptology 22 (2009), no. 2, 161–188 https://eprint.iacr.org/2004/175.pdf

[27] R. Lavin, X Liu, H. Mohanty, L. Norman, G. Zaarour, and B. Krishnamachari. *A survey on the applications of zero-knowledge proofs*. arXiv preprint arXiv:2408.00243 (2024).

[28] Y. Lindell. *Fast cut-and-choose based protocols for malicious and covert adversaries*. In Advances in Cryptology–CRYPTO 2013, pages 1-17, Springer, 2013.

[29] R.Linus. *BitVM 3s–Garbled Circuits for Efficient Computation on Bitcoin* (2025).

[30] S. Maxwell. *Zero knowledge contingent payment*. Bitcoin Wiki, 2011. https://en.bitcoin.it/wiki/Zero_Knowledge_Contingent_Payment.

[31] A. Saleem; A. Khan, F. Shahid; M. M.Alam; M. K. Khan. *Recent advancements in garbled computing: How far have we come towards achieving secure, efficient and reusable garbled circuits*. Journal of Network and Computer Applications, Volume 108, 2018, Pages 1–19.

[32] I. Semaev. *Extracting bits from coordinates of a point of an elliptic curve.* Cryptology ePrint Archive, Report 2005/324, 2005. 10 pp. https://eprint.iacr.org/2005/324.
[33] X. Wang, S. Ranellucci, and J. Katz. *Duplo: Unifying cut-and-choose for garbled circuits.* In Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, pages 3-20, 2017.
[34] X. Wang; S. Ranellucci; J. Katz. *Optimizing authenticated garbling for faster secure two-party computation.* IACR Cryptol. ePrint Arch. 2018/123, 37 pp. (2018).
[35] A. Yao. *How to generate and exchange secrets (extended abstract).* In 27th FOCS, pages 162–167. IEEE Computer Society Press, October 1986.
[36] R. Zhu and Y. Huang. The cut-and-choose game and its application to cryptographic protocols. In 25th USENIX Security Symposium, pages 1085-1100, 2016.

Appendix A. Detailed BN254 pairing computation

The BN254 curve pairing computation involves the Miller loop and final exponentiation. We use Montgomery reduction, hybrid Karatsuba for multiplications, with 16-bit small multiplications and 512-bit additions.

A.1. **Miller loop derivation.** For a point $P = (x_P, y_P) \in E(\mathbb{Z}_p)$, the quadratic twist map is:

$$\text{twist}(P) = (x_P \cdot \omega, y_P \cdot \omega^{3/2}) \in E'(\mathbb{Z}_{p^2})$$

where ω is a fixed non-cube in $\mathbb{Z}_{p^2}$ (for BN254, $\omega = u + 1$ with $u^2 + 1 = 0$). This maps P to the twisted curve E' over $\mathbb{Z}_{p^2}$. The Miller loop computes

$$f = \prod_{i=0}^{65-2} \ell_{\text{twist}(R)}[6x + 2](\text{twist}(R), \text{twist}(P))$$

and updates $R = 2R$ for each bit of the exponent $6x + 2$ in base $\{-1, 0, 1\}$. Starting with $f = 1$, $R = Q$, each bit b_i of the exponent triggers:

(1) If $b_i = 1$: $f \leftarrow f \cdot \ell_{\text{twist}(R)}[6x + 2](\text{twist}(R), \text{twist}(Q))$, $R \leftarrow R + Q$
(2) If $b_i = -1$: $f \leftarrow f \cdot \ell_{\text{twist}(R)}[6x + 2](\text{twist}(R), \text{twist}(-Q))$, $R \leftarrow R - Q$
(3) If $b_i = 0$: $f \leftarrow f \cdot f \cdot \ell_{\text{twist}(R)}[6x + 2](\text{twist}(R), \text{twist}(R))$, $R \leftarrow 2R$

After the main loop, two correction steps are appended:

$$f \leftarrow f \cdot \ell_{\text{twist}(R)}[6x + 2](\text{twist}(R), \text{twist}(Q')), \quad R \leftarrow R + Q'$$

$$f \leftarrow f \cdot \ell_{\text{twist}(R)}[6x + 2](\text{twist}(R), \text{twist}(Q'')), \quad R \leftarrow R + Q''$$

The line function through distinct points $R = (x_R, y_R)$ and $S = (x'_S, y'_S)$ evaluated at P is:

$$\ell(R, S)(P) = \frac{y'_S - y_R}{x'_S - x_R}(x_P - x_R) + y_R$$

For the tangent case $R = S$:

$$\ell(R, R)(P) = \frac{3x_R^2 + A}{2y_R}(x_P - x_R) + y_R$$

This yields 2 million $\mathbb{Z}_p$ multiplications in the main chain.

A.1.1. *Miller loop CRT optimization (no Montgomery in between steps).* For the Miller loop's chained multiplications, we use the Chinese Reminder Theorem representations with $\sim 32 \times 16$-bit moduli, avoiding intermediate Montgomery reductions:

Step	**16-bit muls**	**512-bit adds**
Binary $\to$ CRT (once)	$\sim$2,000	$\sim$10,000
Main chain (2M $\times$ 32)	$\sim$64M	negligible
CRT $\to$ binary (once)	$\sim$1,000	$\sim$5,000
Total	$\sim$66M	$\sim$15,000

This yields roughly 30$\times$ fewer multiplications than a Montgomery chain.

A.2. **Final exponentiation derivation.** The Miller loop outputs $f \in \mathbb{Z}_{p^{12}}^*$. The goal is to compute the final pairing value $f^{(p^{12}-1)/r}$. The exponent is decomposed as:

$$\frac{p^{12}-1}{r} = (p^6-1)(p^2+1) \cdot \frac{p^4-p^2+1}{r}$$

- **Easy part**: Compute $f^{p^6-1} \cdot f^{p^2+1}$.
  - $f^{p^6-1} = f \cdot \overline{f}^{-1}$ (conjugation and inversion).
  - Then raise to p^2+1 using Frobenius maps.
- **Hard part**: Let $m = f^{p^6+1}$ (i.e., apply the easy part to the original Miller result f). Then compute $m^{(p^4-p^2+1)/r}$.

To compute the hard part, raise m to the exponent using an optimal addition chain. First compute the Frobenius powers $y_i = m^{p^i}$ for $i = 0, \ldots, 6$:

$$y_0 = m, \quad y_1 = m^p, \quad y_2 = m^{p^2}, \quad y_3 = m^{p^3}, \quad y_4 = m^{p^4}, \quad y_5 = m^{p^5}, \quad y_6 = m^{p^6}.$$

Then evaluate the polynomial using the following addition chain:

- $T_{0,1} = y_6^2 \cdot y_4 \cdot y_5$
- $T_{1,1} = T_{0,1} \cdot y_3 \cdot y_5$
- $T_{0,2} = T_{0,1} \cdot y_2$
- $T_{1,2} = T_{1,1}^2 \cdot T_{0,2}$
- $T_{1,3} = T_{1,2}^2$
- $T_{1,4} = T_{1,3} \cdot y_0$
- $T_{0,3} = T_{1,3} \cdot y_1$
- $T_{0,4} = T_{0,3}^2 \cdot T_{1,4}$

The value $T_{0,4}$ is exactly $m^{(p^4-p^2+1)/r}$. The final pairing result is the easy part multiplied by $T_{0,4}$.

A.3. **Final exponentiation CRT optimization.** We consider no Montgomery, and 5 chains of $\sim$200k muls each. In this case, $\mathbb{Z}_p$ multiplications in short chains: $\sim$1 million in 5-10 chains.

Step	**16-bit muls**	**512-bit adds**
Binary → CRT	∼2,000	∼10,000
Chain (200k × 32)	∼6.4M	negligible
CRT → binary	∼1,000	∼5,000
Per chain	∼6.4M	∼15,000
All 5 chains	∼32M	∼75,000

A.4. **Total operations.**

A.4.1. *Standard (Montgomery + Karatsuba).*

Scope	**16-bit muls**	**512-bit adds**
Per $\mathbb{Z}_p$ multiplication (254-bit)	∼1,050	∼7,000
Full pairing (∼3.5M $\mathbb{Z}_p$ muls)	∼3.675T	∼24.5T

A.4.2. *CRT-optimized for chains (no intermediate Montgomery).*

Step	**16-bit muls**	**512-bit adds**
Binary → CRT	∼2,000	∼10,000
Main chain (2M × 32)	∼64M	negligible
CRT → binary	∼1,000	∼5,000
Subtotal	∼66M	∼15,000

A.4.3. *Final exponentiation (∼1M $\mathbb{Z}_p$ multiplications, 5 chains of ∼200k each).*

Component	**16-bit muls**	**512-bit adds**
Final exponentiation (5 chains)	∼32M	∼75,000

A.4.4. *Summary.*

Method	**16-bit muls**	**512-bit adds**
Standard (Montgomery + Karatsuba)	∼3.675T	∼24.5T
CRT-optimized	∼98M	∼90,000
Speedup	∼37,500× fewer multiplications	

Ariel Futoransky. Fairgate Labs
Email address: futo@fairgate.io

Fadi Barbàra. Università di Roma La Sapienza (Italy) and Fairgate Labs
Email address: fadi.barbara@fairgate.io

Ramsés Fernández. Fairgate Labs
Email address: ramses.fernandez@fairgate.io

Gabriel Larotonda. Universidad de Buenos Aires (Argentina) and CONICET
Email address: glaroton@dm.uba.ar